

DÉDICACE

À mes très chers parents,

pour leur amour inconditionnel, leur patience infinie
et les innombrables sacrifices consentis tout au long de mon parcours.

Vous êtes ma force et mon inspiration.



À mes frères et sœurs,

pour votre soutien sans faille, votre présence réconfortante
et les moments de joie partagés qui ont illuminé ce chemin.



À mes amis et collègues,

pour les échanges enrichissants, les fous rires partagés
et votre précieux soutien dans les moments difficiles.



*À tous ceux qui croient en la puissance
du savoir et de la persévérance.*

« Le savoir est un trésor que personne ne peut vous voler. »

Mabrouk Zoubair

Projet de Fin d'Études — Génie Informatique · IIT Sfax · 2024–2025

REMERCIEMENTS



u terme de ce travail, c'est avec une profonde gratitude que je tiens à exprimer mes sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.



Encadrement académique : M. Fahmi Kallel

J'adresse mes plus vifs remerciements à mon encadrant pour sa disponibilité constante, ses conseils avisés et l'orientation méthodologique rigoureuse qu'il m'a prodiguée tout au long de ce travail.



Encadrement professionnel — EDI-Solutions

Je remercie chaleureusement l'équipe d'EDI-Solutions pour l'accueil bienveillant, le cadre de travail stimulant et la confiance accordée, qui ont rendu cette expérience professionnelle à la fois formatrice et enrichissante.



Membres du jury

Je remercie les membres du jury, le président **M. Koubaa Mohamed** et le rapporteur **M. Karoui Achraf**, d'avoir accepté d'évaluer ce travail. Leurs remarques et suggestions contribueront sans nul doute à l'enrichissement de cette recherche.



Chère Famille & Amis

J'exprime ma profonde reconnaissance à mes collègues, surtout M. Ghabri Moataz, pour leur soutien, leur patience, leurs conseils avisés et la confiance qu'ils m'ont accordée, lesquels ont grandement contribué à l'aboutissement de ce projet. Je tiens également à remercier chaleureusement mes frères, Chames Eddine, Mohamed et Zied, pour leur présence et leur encouragement constant tout au long de ce travail.

Mabrouk Zoubair

Projet de Fin d'Études — Génie Informatique · IIT Sfax · 2024–2025

Table des matières

Introduction générale	1
1 CADRE GÉNÉRAL DU PROJET	4
1.1 Introduction	5
1.2 Présentation de l'organisme d'accueil : EDI Solutions	5
1.3 Cadre du projet	6
1.3.1 Étude critique de l'existant	7
1.3.2 Problématique	8
1.3.3 Solution proposée	9
1.3.4 Étude de solution	9
1.4 Méthodologie	11
1.4.1 Principes agiles adoptés	12
1.4.2 Planification des sprints	13
1.4.3 Application de Scrum	14
1.4.4 Product Backlog	15
1.5 Conclusion	16
2 SPÉCIFICATION ET ANALYSE DES BESOINS	17
2.1 Introduction	18
2.2 Identification des acteurs	18
2.3 Besoins fonctionnels	19
2.4 Besoins non fonctionnels	21
2.5 Architecture de la solution	22
2.6 Fondements Théoriques	23
2.6.1 Concepts de base de l'informatique décisionnelle	23
2.6.2 Architecture d'un système décisionnel	24
2.6.3 Les modèles de conception	24
2.6.3.1 Le modèle en étoile	24
2.6.3.2 Le modèle en flocon de neige	25
2.6.3.3 Le modèle en constellation de faits	25
2.6.4 Concept de l'Architecture RAG	26
2.6.4.1 Phase de récupération (<i>Retrieval</i>)	26

2.6.4.2	Phase d'augmentation (<i>Augmentation</i>)	27
2.6.4.3	Phase de génération (<i>Generation</i>)	27
2.6.5	Visualisation	28
2.6.5.1	Modélisation par Atomic Design	28
2.6.5.2	Gestion des styles	29
2.6.6	Concept DevOps	29
2.7	Technologies utilisées	30
2.8	Conclusion	32
3	FONDATION FONCTIONNELLE – RELEASE 1	34
3.1	Introduction	35
3.2	Sprint 1 : Mise en place de la couche Data	35
3.2.1	Analyse des sources de données	37
3.2.2	Identification du sujet analytique	38
3.2.3	Conception logique du Data Warehouse	38
3.2.3.1	Table de faits	39
3.2.3.2	Tables de dimensions	39
3.2.4	Implémentation du processus ETL	40
3.2.5	Nettoyage et préparation des données	43
3.2.6	Construction du Cube OLAP	45
3.2.7	Mise en place du Mdx Service	46
3.2.7.1	Diagramme de cas d'utilisation	47
3.2.7.2	Diagramme de séquence	47
3.2.8	Tests unitaires	49
3.2.8.1	Documentation API	50
3.3	Sprint 2 : Mise en place de la couche Intelligente	52
3.3.1	Conception du service RAG	53
3.3.2	Étude comparative	55
3.3.2.1	Comparaison des modèles d'embeddings	56
3.3.2.2	Comparaison des modèles LLM	56
3.3.3	Exploitation des métadonnées OLAP	57
3.3.4	Vectorisation et indexation sémantique	59
3.3.5	Enrichissement du contexte et Prompt Engineering	61
3.3.5.1	Récupération sémantique du contexte	61
3.3.5.2	Construction du prompt enrichi	61
3.3.5.3	Génération de requêtes MDX via LLM	62

3.3.6	Évaluation des performances du système intelligent	63
3.3.6.1	Constitution et annotation du corpus d'évaluation	63
3.3.6.2	Métriques d'évaluation des réponses	63
3.3.6.3	Influence du paramètre k sur la qualité du retrieval	64
3.3.6.4	Évaluation comparative des modèles LLM	65
3.3.6.5	Analyse des résultats et synthèse	66
3.4	Conclusion	66
4	AMÉLIORATION ET DÉPLOIEMENT – RELEASE 2	68
4.1	Introduction	69
4.2	Sprint 3 : Implémentation des services backend complémentaire	69
4.2.1	Mise en Place de Conversation Service	70
4.2.1.1	Diagramme de séquence	70
4.2.1.2	Communication Asynchrone avec RabbitMQ	71
4.2.1.3	Tests Unitaires	72
4.2.1.4	Documentation Swagger	73
4.2.2	Mise en place de l'Authentication Service	74
4.2.2.1	Diagramme de classes	74
4.2.2.2	Cas d'utilisation	75
4.2.2.3	Diagramme de Séquence	76
4.2.2.4	Tests Unitaires	79
4.2.2.5	Documentation Swagger	79
4.2.3	Mise en place de l'API Gateway	80
4.2.3.1	Diagramme de Séquence	80
4.2.3.2	Configuration YARP : routes et clusters	81
4.2.3.3	Interconnexion des services	83
4.2.4	Implémentation du Frontend	84
4.2.4.1	Architecture et organisation du projet	84
4.2.4.2	Communication avec les micro services	85
4.2.4.3	Authentification et sécurité	86
4.2.4.4	Tableau de bord décisionnel	87
4.2.4.5	Interface Intelligente	88
4.3	Sprint 4 : Industrialisation et DevOps	89
4.3.1	Architecture de Déploiement	90
4.3.1.1	Vue globale de l'infrastructure	90
4.3.1.2	Séparation des responsabilités : deux repositories distincts . .	92

4.3.1.3	Architecture CI/CD globale	93
4.3.2	Intégration Continue (CI)	94
4.3.2.1	Définition d'un pipeline CI/CD réutilisable pour les micro services .NET	94
4.3.2.2	Validation de la qualité du code	95
4.3.2.3	Construction et publication des images Docker	96
4.3.2.4	Stratégie de versioning	97
4.3.3	Gestion Centralisée des Versions	98
4.3.4	Déploiement Continu (CD)	100
4.3.4.1	Déclenchement automatique	100
4.3.4.2	Déploiement sur le VPS	100
4.3.4.3	Healthchecks et validation de disponibilité	102
4.3.4.4	Notification de mis à jour	103
4.3.5	Sécurité DevOps	104
4.3.6	Monitoring et Supervision avec Portainer	106
4.3.6.1	Supervision des conteneurs en temps réel	107
4.3.6.2	Consultation des logs et diagnostics	108
4.3.6.3	Surveillance des ressources système	108
4.3.6.4	Gestion des volumes et de la persistance	109
4.3.6.5	Gestion et inspection des réseaux Docker	110
4.4	Conclusion	111
5	CONCLUSION GÉNÉRALE	112
	ANNEXE	115
.1	Liste des Code HTTP	115
.2	Catalogue des Endpoints des Microservices	116
.2.1	Auth Service (<i>auth-service</i>)	116
.2.2	Conversation History Service (<i>conversation-history-service</i>)	116
.2.3	MDX Service (<i>mdx-service</i>)	117
.2.4	RAG Service (<i>rag-service</i>)	118
.2.5	API Gateway (<i>api-gateway</i>)	118
.3	Glossaire des acronymes	118

Liste des figures

1	EDI Solutions	5
2	Pipeline intelligent de traitement des requêtes analytiques	6
3	Critique de Système existant	7
4	Processus Scrum	12
5	Architecture des Composants Globale de Système	23
6	La chaine décisionnelle	24
7	Schéma en étoile	25
8	Schéma en Flocon de neige	25
9	Schéma en Constellation de faits	26
10	Architecture globale du système RAG	28
11	Architecture Atomic de Frontend	29
12	Processus DevOps	30
13	Schémas en étoile de Data Warehouse	39
14	Implémentation des flux ETL sous SSIS pour le chargement du Data Warehouse	41
15	Étapes de conception et de configuration du cube OLAP sous SQL Server Analysis Services	46
16	Diagramme de cas d'utilisation du Mdx Service	47
17	Diagramme de séquence du Mdx Service	48
18	Documentation Swagger des endpoints REST dédiés à l'exposition des indicateurs analytiques (KPIs) du MDX Service	51
19	Documentation Swagger de l'endpoint d'interrogation analytique en langage naturel intégré au chat-bot IA	51
20	Tester l'endpoint avec une requete donner sur swagger	52
21	Architecture du pipeline RAG appliqué au contexte OLAP	54
22	Diagramme de Séquence de Service RAG	55
23	Influence du paramètre k sur la pertinence sémantique, le taux de validité MDX et le temps de réponse – courbes obtenues sur le corpus de 30 requêtes tests avec Phi-3 Mini	65
24	Diagramme de séquence du Conversation Service	71

25	Architecture de communication asynchrone	72
26	Documentation Swagger de Conversation Service	74
27	Diagramme de classes du Authentication Service	75
28	Diagramme de cas d'utilisation du Authentication Service	76
29	Diagramme de séquence de Service Authentification	77
30	Diagramme de séquence du service d'authentification – Création de compte . .	78
31	Documentation Swagger des endpoints REST exposés par le Service d'Authentification	80
32	Diagramme de séquence du service API Gateway – Fonctionnement nominal .	81
33	Structure du projet frontend selon l'architecture Atomic	84
34	Configuration des routes publiques et protégées dans <code>App.tsx</code>	85
35	Configuration de communication entre le frontend et l'infrastructure de backend	86
36	Interface de login	87
37	Tableau de bord principal avec KPIs et visualisations analytiques	88
38	Interface initiale du chat-bot analytique	88
39	Réponse du chat-bot sous forme de graphique	89
40	Réponse du chat-bot sous forme de tableau	89
41	Vue globale de l'infrastructure DevOps	91
42	Séparation des deux repositories : micro services (code + CI) et infra-deploy (versions + CD).	93
43	Configuration d'un workflow CI/CD réutilisable des micro services .NET. . . .	95
44	Pipeline de validation qualité de code .NET	96
45	Pipeline de build et push Docker multi-architecture avec cache optimisé via GitHub Actions.	97
46	Mécanisme de résolution de version : tag Git sémantique ou SHA court du commit	98
47	Fichier <code>.env.prod</code> centralisant les versions des micro services dans une approche GitOps	98
48	Mise à jour automatisée du repository d'infrastructure après publication des images Docker	99
49	Exemple de commit automatique suit un changement de version d'une image Docker	99
50	Déclenchement du pipeline de déploiement continu via modification des fichiers d'infrastructure	100
51	Exécution du déploiement sur VPS via SSH	101
52	Configuration de healthcheck Docker et validation de disponibilité de micro service.	102

53	Implémentation du message Telegram dans GitHub Actions.	103
54	Exemple de message Télégram avec les deux cas : succès et échec	104
55	Les secrets sensibles des micro services dans GitHub Secrets	105
56	Les secrets de l'infrastructure de déploiement dans GitHub Secrets	106
57	Vue d'ensemble du tableau de bord Portainer sur l'environnement de production	107
58	Liste des conteneurs en cours d'exécution avec leur statut et version d'image respective	107
59	Console de logs en temps réel d'un micro service depuis l'interface Portainer .	108
60	Métriques de consommation CPU et mémoire par conteneur dans Portainer - PARTIE 1	109
61	Métriques de consommation CPU et mémoire par conteneur dans Portainer PARTIE 2	109
62	Interface de gestion des volumes Docker et supervision de la persistance des données	110
63	Interface de gestion et d'inspection des réseaux Docker dans Portainer	111

Liste des tableaux

1	Comparaison des stratégies d'intégration des modèles de langage	10
2	Planification des sprints	14
3	Ectrait de User Stories du système décisionnel intelligent	15
4	Besoins fonctionnels de la plateforme décisionnelle intelligente	20
5	Besoins non fonctionnels de la plateforme décisionnelle intelligente	21
6	Récapitulatif des technologies utilisées dans la plateforme décisionnelle	30
7	Backlog du Sprint 1 : Mise en place de la couche Data	36
8	Scénarios de tests unitaires pour le Mdx Service	50
9	User Stories du Sprint 2	53
10	Comparaison des familles de modèles d'embeddings selon leur adéquation aux systèmes RAG	56
11	Comparaison des modèles de langage étudiés	57
12	Métriques d'évaluation du système RAG NL2MDX	64
13	Comparaison des modèles LLM sur les critères de temps de réponse, validité syntaxique MDX et pertinence sémantique, pour $k = 5$	66
14	User Stories du Sprint 3	70
15	Tests unitaires du Conversation Service	73
16	Cas de tests unitaires du service d'authentification	79
17	Mapping des routes API Gateway vers micro services	83
18	User Stories du Sprint 4	90
19	Composants techniques de l'architecture de l'infrastructure	92
20	Principaux codes HTTP utilisés dans la plateforme	115
21	Endpoints exposés par le microservice d'authentification et de gestion des utilisateurs.	116
22	Endpoints exposés par le microservice de gestion de l'historique des conversations.	116
23	Endpoints analytiques exposés par le microservice MDX pour l'interrogation du cube OLAP.	117
24	Endpoints exposés par le microservice RAG basé sur FastAPI pour l'interrogation intelligente des données analytiques.	118

25	Routes exposées par l'API Gateway pour le routage centralisé des requêtes vers les microservices internes.	118
26	Glossaire des acronymes utilisés dans le mémoire	118

Liste des codes

1	Génération de la dimension Date (2000-2030)	42
2	Détection des doublons dans la table Tiers	43
3	Suppression des doublons avec ROW_NUMBER	43
4	Vérification des valeurs invalides dans Dim_Date	44
5	Ajout de contraintes d'intégrité	44
6	Extraction des meta données : mesures	57
7	Représentation des métadonnées du cube OLAP	58
8	Initialisation du service RAG avec embeddings	59
9	Indexation des embeddings dans FAISS	59
10	Persistance et récupération des embeddings dans ChromaDB	60
11	Recherche sémantique dans FAISS	61
12	Construction du prompt enrichi pour RAG	61
13	Appel du LLM pour génération MDX	62
14	Configuration YARP - routes et clusters	81

INTRODUCTION GÉNÉRALE

À l'ère de la transformation numérique, les organisations produisent quotidiennement des volumes massifs de données provenant de multiples systèmes opérationnels tels que les ERP, CRM, applications de gestion commerciale, plateformes e-commerce ou systèmes de facturation. Ces données représentent une ressource stratégique majeure permettant d'orienter les décisions métier, d'anticiper les tendances et d'améliorer la compétitivité des entreprises. Toutefois, la simple accumulation de données ne garantit pas une exploitation efficace de l'information. La véritable problématique réside dans la capacité à transformer ces données hétérogènes en connaissances exploitables et accessibles aux décideurs dans des délais réduits.

Dans ce contexte, les systèmes décisionnels ou plateformes de Business Intelligence (BI) jouent un rôle fondamental. Ils permettent de centraliser les données issues des différentes sources opérationnelles au sein d'un entrepôt de données (Data Warehouse), puis de les organiser sous une forme multidimensionnel adaptée à l'analyse stratégique. Les cubes OLAP (Online Analytical Processing) offrent ainsi des capacités avancées d'exploration, d'agrégation et d'analyse des indicateurs métier selon plusieurs axes décisionnels.

Malgré leur importance, les solutions décisionnelles traditionnelles présentent encore plusieurs limitations. D'une part, l'accès aux données analytiques reste souvent réservé aux profils techniques maîtrisant les langages SQL ou MDX, ce qui constitue une barrière importante pour les utilisateurs métiers. D'autre part, les traitements analytiques complexes peuvent engendrer des problèmes de performance lorsque les volumes de données deviennent importants. De plus, les tableaux de bord classiques demeurent généralement statiques et nécessitent des développements supplémentaires pour répondre à de nouveaux besoins d'analyse. Enfin, l'intégration de l'intelligence artificielle dans les systèmes décisionnels reste encore limitée dans de nombreuses architectures d'entreprise.

Parallèlement, les avancées récentes dans le domaine de l'intelligence artificielle générative, notamment les Large Language Models (LLM) et les architectures RAG (Retrieval-Augmented Generation), ouvrent de nouvelles perspectives pour l'interaction homme-machine. Ces technologies permettent désormais d'interroger des systèmes complexes en langage naturel et de générer automatiquement des requêtes analytiques à partir d'expressions métiers simples. Elles contribuent ainsi à démocratiser l'accès à l'analyse décisionnelle en réduisant considérablement la dépendance aux équipes techniques.

C'est dans cette optique que s'inscrit ce projet de fin d'études, réalisé au sein de la société

EDI-Solutions. Le projet a pour objectif la conception et l'implémentation d'une plateforme décisionnelle intelligente intégrant un Data Warehouse multidimensionnel, des cubes OLAP, une architecture micro services et un chat-bot analytique basé sur un modèle de langage local. La plateforme proposée vise à offrir un environnement analytique moderne, scalable et sécurisé permettant aux utilisateurs métiers d'interroger les données en langage naturel, de générer dynamiquement des requêtes MDX et de visualiser les résultats sous forme de tableaux de bord interactifs.

L'architecture retenue repose sur une approche micro services afin d'assurer la modularité, la maintenabilité et la scalabilité du système. Cette architecture est renforcée par l'utilisation de technologies modernes telles que ASP.NET Core, React, Docker, RabbitMQ, SQL Server Analysis Services (SSAS), ainsi qu'une chaîne CI/CD permettant l'automatisation des phases de build, test et déploiement. En complément, l'intégration d'une couche intelligente basée sur les techniques de RAG et de prompt engineering permet d'améliorer la pertinence des réponses générées par le chat-bot analytique tout en garantissant un contrôle des requêtes produites.

Le développement du projet a été conduit selon la méthodologie agile Scrum afin de favoriser une évolution incrémentale et continue de la plateforme. Cette approche a permis de structurer le travail en plusieurs releases fonctionnelles couvrant progressivement la mise en place de l'infrastructure décisionnelle, l'architecture micro services, les tableaux de bord analytiques, l'intégration de l'intelligence artificielle et l'industrialisation DevOps de la solution.

Ce rapport est structuré en cinq chapitres complémentaires. Le premier chapitre présente le cadre général du projet. Il introduit l'organisme d'accueil, expose l'étude critique de l'existant, identifie la problématique rencontrée et décrit la solution proposée ainsi que la méthodologie de gestion de projet adoptée.

Le deuxième chapitre est consacré à la spécification des besoins et à l'analyse de la solution. Il présente les acteurs du système, les besoins fonctionnels et non fonctionnels, l'architecture globale de la plateforme ainsi que les technologies retenues pour sa réalisation et les fondements théoriques nécessaires à la compréhension du projet.

Le troisième chapitre décrit la première release fonctionnelle de la plateforme. Il détaille la conception et l'implémentation de la couche décisionnelle, incluant la construction du Data Warehouse, les processus ETL, les cubes OLAP, le service d'exécution des requêtes MDX ainsi que la mise en place de la couche intelligente basée sur une architecture RAG pour la génération de requêtes analytiques en langage naturel.

Le quatrième chapitre présente la seconde release consacrée à l'industrialisation et à l'amélioration de la plateforme. Il couvre le développement des micro services backend, de l'API Gateway, du frontend analytique, ainsi que la mise en œuvre des pratiques DevOps, de la

conteneurisation, de l'intégration continue et du déploiement continu.

Enfin, le dernier chapitre clôture ce travail en présentant une synthèse des réalisations effectuées, les résultats obtenus ainsi que les perspectives d'évolution envisageables pour enrichir davantage la plateforme.

Chapitre 1

CADRE GÉNÉRAL DU PROJET

1.1. Introduction

Au départ de ce rapport, ce premier chapitre est consacré à la présentation de l'organisme d'accueil. Il introduit ensuite une citation mettant en évidence l'idée directrice du projet, avant d'exposer la problématique étudiée ainsi que les limites et insuffisances du système existant. La solution proposée est ensuite présentée, puis le chapitre se termine par la description de la méthodologie adoptée pour la réalisation du projet.

Dans un contexte économique toujours plus concurrentiel, l'information est devenue un élément stratégique crucial pour les sociétés. Les systèmes d'exploitation comme les ERP, Les systèmes transactionnels ou CRM produisent chaque jour un volume considérable de données. Cependant, en dépit de cette richesse, l'utilisation efficace de ces informations demeure restreinte. En vérité, les outils d'analyse classiques nécessitent fréquemment des aptitudes techniques. progrès, en particulier en SQL ou en MDX, ce qui constitue un obstacle majeur pour les utilisateurs professionnels.

Par conséquent, ces derniers se retrouvent tributaires des équipes techniques. pour obtenir des informations, ce qui retarde considérablement le processus de décision. Dans ce cadre, ce projet a pour objectif de développer une plateforme analytique intelligente. permettant de rendre l'accès aux données plus accessible, en intégrant les technologies de Data Warehouse, OLAP et Intelligence Artificielle.

1.2. Présentation de l'organisme d'accueil : EDI Solutions



FIGURE 1. EDI Solutions

EDI-Solutions est une société de services et d'ingénierie informatique implantée à la route Gremda, km 3, Sfax, Tunisie. EDI-Solutions est une société d'ingénierie et de services informatiques fondée en 2016 et basée à Sfax en Tunisie, spécialisée dans l'intégration des

ERPs et le développement de solutions sur mesure web et mobile. Son équipe possède une vaste expérience couvrant différents domaines tels que le développement d'applications web et mobiles, le commerce électronique, le business intelligence. Elle met ses compétences et ses ressources à votre disposition pour collaborer à vos projets.

EDI-Solutions offre des services couvrant deux grands volets à savoir :

- Audit et Conseil (Business Consulting, Conseil réseaux et SI)
- L'analyse multidimensionnel à travers des cubes OLAP
- Audit et analyse des processus de l'activité des entreprises à savoir (stock / Achat / vente / trésorerie / etc.)
- Diagnostic SI
- Audit Réseau et sécurité
- Cadrage de projet
- Rédaction des spécifications techniques (cahier des charges, etc.)
- La mise en place des infrastructures
- Le développement et l'intégration des solutions informatiques
- La mise en œuvre de solutions innovantes, notamment des plateformes Cloud adaptées aux besoins des clients, ou l'intégration de solutions proposées par le partenaire EDI, telles que SMART ERP.

1.3. Cadre du projet

Pour atteindre ces objectifs, la solution repose sur un pipeline intelligent assurant la transformation des requêtes utilisateurs en analyses décisionnelles exploitables. Ce processus, illustré dans la Figure 2, intègre des composants d'intelligence artificielle pour interpréter les requêtes, enrichir le contexte via des techniques de type RAG, générer dynamiquement des requêtes MDX, interroger les cubes OLAP et restituer les résultats sous forme de visualisations.

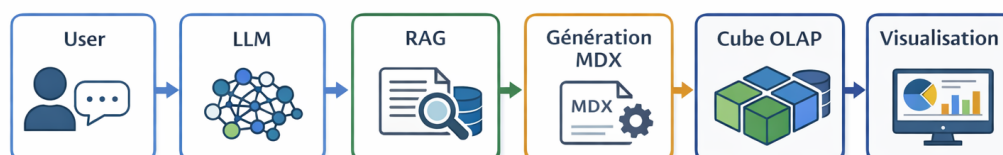


FIGURE 2. Pipeline intelligent de traitement des requêtes analytiques

1.3.1. Étude critique de l'existant

La Figure 3 présente l'architecture du système décisionnel actuellement en place. Celui-ci s'appuie principalement sur une base de données relationnelle alimentée directement par les systèmes opérationnels, notamment l'ERP et les outils de facturation. Les analyses sont effectuées à travers des requêtes SQL exécutées sur ces bases, avec l'appui de certaines optimisations techniques telles que les vues matérialisées et les mécanismes d'indexation.

Dans ces circonstances, les données utilisées pour l'analyse proviennent directement des systèmes transactionnels, ce qui implique une interaction étroite entre les traitements opérationnels et analytiques. Les requêtes analytiques, parfois complexes, sont exécutées sur les mêmes structures de données que celles utilisées pour les opérations quotidiennes.

Par ailleurs, les données exploitées reflètent l'état des systèmes sources, incluant différentes entités métiers et leurs relations. L'organisation actuelle des données repose sur un modèle relationnel adapté aux besoins transactionnels, et les traitements analytiques sont réalisés en fonction de cette structuration.

Enfin, le système intègre diverses techniques d'optimisation afin d'améliorer les performances des requêtes et de faciliter l'accès aux informations nécessaires à l'analyse. Cette approche constitue le socle sur lequel reposent les activités décisionnelles actuelles.

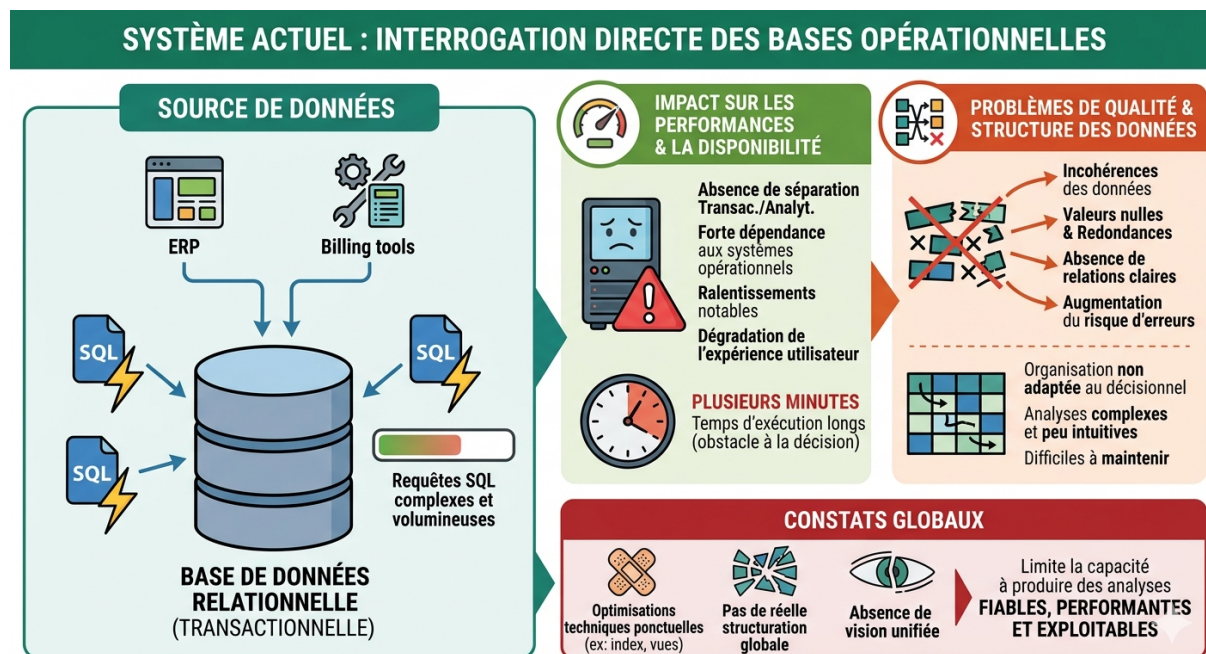


FIGURE 3. Critique de Système existant

1.3.2. Problématique

Malgré les avancées technologiques en matière de **BI**, le système actuel a plusieurs limitations majeures liées à son architecture et à ses capacités analytiques. En effet, l'exploitation des données repose directement sur la base relationnelle transactionnelle, sans recours à un Data Warehouse. Cette approche entraîne une forte complexité dans l'écriture des requêtes SQL et une dégradation significative des performances, notamment pour les analyses multidimensionnel. Certaines requêtes analytiques complexes peuvent ainsi nécessiter plusieurs minutes d'exécution[1].

De plus, les données disponibles présentent des insuffisances importantes, telles que la présence de valeurs nulles, l'absence de relations explicites entre certaines entités, et un manque de structuration globale, ce qui complique davantage leur exploitation.

Les principales limitations peuvent être résumées comme suit :

- Difficulté d'accès aux données pour les utilisateurs métiers
- Complexité des requêtes analytiques (SQL avancé requis)
- Temps de réponse élevé pour les analyses complexes
- Absence de modélisation multidimensionnel
- Qualité de données insuffisante (incohérences, valeurs nulles)
- Absence d'outils intelligents d'assistance à la décision

Ces contraintes engendrent plusieurs impacts négatifs :

- Une forte dépendance vis-à-vis des équipes techniques
- Une lenteur dans le processus de prise de décision
- Une sous-exploitation des données disponibles

À cet égard, la problématique centrale du projet est la suivante :

Comment concevoir une plateforme décisionnelle intelligente basée sur un Data Warehouse et des technologies d'intelligence artificielle, permettant une interaction naturelle avec les données, tout en garantissant performance, qualité des données et simplicité d'utilisation ?

Afin de répondre à cette problématique, les sous-questions suivantes ont été définies :

- **Comment concevoir et modéliser un Data Warehouse adapté aux besoins décisionnels de l'organisation tout en garantissant la qualité, la cohérence et la performance des données ?**
- **Comment exploiter les technologies d'intelligence artificielle et les modèles de langage pour générer automatiquement des requêtes MDX valides, optimisées et conformes à la structure du cube multidimensionnel ?**
- **Comment automatiser le déploiement et l'exploitation de la plateforme décisionnelle afin d'assurer sa scalabilité, sa maintenabilité et sa disponibilité dans un environnement de production ?**

1.3.3. Solution proposée

Après l'analyse des limites du système décisionnel existant ainsi que l'étude des différentes approches d'intégration de l'intelligence artificielle dans un environnement analytique, une nouvelle architecture décisionnelle intelligente a été proposée. Cette solution vise à répondre aux problématiques liées à la complexité des requêtes analytiques, aux performances limitées des systèmes OLTP traditionnels ainsi qu'à la difficulté d'accès aux données décisionnelles pour les utilisateurs métiers.

La plateforme proposée repose sur une combinaison de technologies modernes incluant les Data Warehouses multidimensionnels, les cubes OLAP, les architectures micro services ainsi que les modèles de langage (*LLM*) enrichis par un système RAG (*Retrieval-Augmented Generation*). Cette approche permet de transformer les requêtes formulées en langage naturel en analyses décisionnelles exploitables tout en améliorant la pertinence contextuelle des réponses générées [2].

L'objectif principal de cette solution est de fournir une plateforme analytique intelligente capable de :

- simplifier l'interrogation des données décisionnelles ;
- améliorer les performances des traitements analytiques ;
- automatiser la génération des requêtes MDX ;
- enrichir les réponses grâce à la recherche sémantique ;
- offrir une visualisation interactive des résultats ;
- garantir une architecture scalable, maintenable et extensible.

Les sections suivantes présentent successivement l'étude comparative des différentes stratégies d'intégration des modèles de langage, l'architecture globale proposée ainsi que les principales fonctionnalités offertes par la plateforme.

1.3.4. Étude de solution

Dans le cadre de la conception d'une plateforme décisionnelle intelligente intégrant un chat-bot analytique, plusieurs approches peuvent être envisagées pour intégrer des capacités d'intelligence artificielle basées sur les modèles de langage (*LLM*) dans des outils de Business Intelligence. Le choix de la stratégie d'intégration constitue une étape déterminante, elle appelle *Make or Buy*, puisqu'il impacte directement les aspects liés à la sécurité, aux coûts d'exploitation, aux performances, à la confidentialité des données ainsi qu'à la maintenabilité de la solution [3, 4].

Dans cette optique, une première étude comparative a été menée afin d'évaluer les différentes

stratégies possibles d'intégration des modèles de langage dans un environnement décisionnel d'entreprise.

TABLE 1. Comparaison des stratégies d'intégration des modèles de langage

Critère	Solution SaaS existante	API LLM Cloud	LLM Open Source local	Développement d'un LLM propriétaire
Principe	Utilisation d'une plateforme BI intelligente prête à l'emploi	Communication avec un LLM distant via API	Déploiement local d'un modèle open source	Conception et entraînement complet d'un modèle interne
Exemple	Basedash, Power BI Copilot	OpenAI API, Azure OpenAI	Phi-3, Llama, Mistral	Modèle entraîné en interne
Coût initial	Faible	Faible	Moyen	Très élevé
Coût d'exploitation	Abonnement mensuel	Paieement par requête/token	Infrastructure locale uniquement	Infrastructure et entraînement coûteux
Confidentialité des données	Faible	Moyenne	Élevée	Très élevée
Dépendance externe	Très élevée	Élevée	Faible	Aucune
Complexité de mise en œuvre	Faible	Faible	Moyenne	Très élevée
Personnalisation	Limitée	Moyenne	Élevée	Très élevée
Performance locale	Dépend du réseau	Dépend du réseau	Élevée	Variable
Latence	Variable	Variable	Faible	Faible
Maintenance	Faible	Faible	Moyenne	Très élevée
Adaptation au contexte métier	Limitée	Moyenne	Élevée	Très élevée
Adaptation au budget de la société	Moyenne	Faible à moyenne	Élevée	Faible

Le tableau 1 met en évidence les différentes approches envisageables pour intégrer un modèle de langage dans une plateforme décisionnelle intelligente.

La première approche consiste à utiliser une solution SaaS existante, comme *Power BI Copilot* [5], intégrant déjà des fonctionnalités conversationnelles et analytiques [6]. Bien que cette approche permette un déploiement rapide, elle présente plusieurs limitations importantes, notamment une faible maîtrise des données, une dépendance forte envers le fournisseur ainsi qu'une personnalisation limitée du comportement du modèle.

La seconde approche repose sur l'utilisation d'API cloud permettant de communiquer avec des modèles propriétaires tels que GPT via OpenAI ou Azure OpenAI [3, 4]. Cette solution offre une excellente qualité de génération et une intégration relativement simple. Cependant, elle introduit plusieurs contraintes majeures pour le contexte du projet, notamment :

- des coûts récurrents liés à la consommation des tokens ;
- une dépendance permanente à un fournisseur externe ;
- des problématiques de confidentialité des données analytiques.

Une troisième approche consiste à installer localement un modèle de langage open source [7, 8, 9]. Cette stratégie permet d'exécuter le modèle directement sur l'infrastructure de l'entreprise, sans dépendance à un service externe. Elle offre ainsi plusieurs avantages importants :

- meilleure confidentialité des données ;
- maîtrise complète de l'infrastructure ;
- réduction des coûts d'exploitation ;
- possibilité de personnalisation ;
- faible latence des traitements locaux.

Enfin, la dernière approche consiste à développer un modèle de langage propriétaire. Bien que cette solution fournisse un contrôle maximal et une personnalisation avancée, elle nécessite des ressources matérielles considérables, des compétences spécialisées en intelligence artificielle ainsi qu'un budget très important pour l'entraînement et la maintenance du modèle [10].

Au regard des contraintes réelles du projet, notamment les limitations budgétaires, les ressources matérielles disponibles ainsi que les exigences de sécurité des données décisionnelles, la troisième approche qu'il est basée sur l'installation du serveur Ollama et d'un modèle open source a été retenue. Le modèle LLM retenu dans cette solution a été sélectionné selon plusieurs critères présentés dans la section 3.3.2.2.

1.4. Méthodologie

L'implémentation de ce projet s'est appuyée sur une démarche itérative inspirée des principes agiles, organisée en sprints successifs afin de garantir une progression structurée et maîtrisée du développement. Cette approche a permis de livrer régulièrement des incréments fonctionnels validables, de prioriser les tâches selon leur valeur métier et d'adapter le plan de développement aux contraintes techniques rencontrées.

1.4.1. Principes agiles adoptés

La démarche adoptée s'inspire du cadre **Scrum**, en particulier de son organisation en sprints à durée fixe (4 semaines) et de la notion de Product Backlog priorisé. Dans le contexte d'un projet individuel de fin d'études, certaines cérémonies collectives (daily Scrum, rétrospective d'équipe) ont été adaptées sous la forme de points de suivi hebdomadaires avec l'encadrant professionnel, jouant le rôle de Product Owner, et de bilans réguliers avec l'encadrant académique.

L'organisation du projet repose sur les trois rôles fondamentaux définis par la méthodologie Scrum :

- **Product Owner** : l'encadrant professionnel d'EDI-Solutions, responsable de la définition des besoins fonctionnels et de la priorisation des fonctionnalités.
- **Scrum Master** : l'encadrant académique, assurant la validation des livrables à la fin de chaque sprint et le suivi de la cohérence architecturale.
- **Équipe de développement** : l'auteur de ce mémoire, assurant l'intégralité des tâches de conception, développement, intégration et test des composants de la plateforme.

Cette organisation a permis d'assurer une meilleure répartition des responsabilités et une coordination efficace entre les différentes composantes techniques du projet.



FIGURE 4. *Processus Scrum*

1.4.2. Planification des sprints

Le projet a été structuré en quatre sprints de quatre semaines chacun, couvrant progressivement toutes les couches du système :

- **Sprint 1 (Data Warehouse & OLAP)** : ce sprint a été consacré à l'analyse des données sources, à la conception du schéma multidimensionnel ainsi qu'à la mise en place du Data Warehouse et du cube OLAP, et l'exposition des KPIs dans un API Rest. Les processus ETL permettant l'alimentation des données analytiques ont également été développés.
- **Sprint 2 (IA & chat-bot Analytique)** : ce sprint a permis d'implémenter la couche d'intelligence artificielle basée sur une architecture RAG. La vectorisation des métadonnées du cube, l'intégration de FAISS et la génération automatique des requêtes MDX via un modèle de langage ont été réalisées.
- **Sprint 3 (Backend & Architecture)** : cette phase a porté sur l'intégration des différents services au sein d'une architecture micro services, incluant la mise en place d'une API Gateway, d'un service d'authentification et le développement d'une interface utilisateur avancée avec des tableaux de bord dynamiques.
- **Sprint 4 (Déploiement)** : cette dernière phase a été consacrée à l'optimisation globale du système et préparation au déploiement, incluant l'amélioration des performances, l'automatisation des pipelines CI/CD, la conteneurisation via Docker et la mise en place des outils de monitoring.

Cette organisation a permis de séparer clairement les différentes couches du système tout en assurant une intégration progressive des composants analytiques, backend et intelligents.

TABLE 2. *Planification des sprints*

Sprint	Objectif	Livrables	Durée
Sprint 1	Conception et mise en place de la couche data	Data Warehouse, processus ETL, cube OLAP, indicateurs (KPI) statiques, Exposition d'API	4 semaines
Sprint 2	Développement du service conversationnel intelligent	micro service conversationnel (LLM + RAG), API d'interprétation des requêtes, interface utilisateur initiale	4 semaines
Sprint 3	Intégration des services et développement de l'interface finale	API Gateway, MDX Executor Service, orchestration des micro services, interface utilisateur avancée, dashboards dynamiques	4 semaines
Sprint 4	Optimisation, industrialisation et déploiement	Optimisation des performances, automatisation CI/CD, conteneurisation (Docker), monitoring	4 semaines

1.4.3. Application de Scrum

La méthode Scrum a été appliquée de manière pragmatique dans les phase d'implémentation de la plateforme intelligente dans le but de :

- Structurer le travail par phases itératives,
- Faciliter la priorisation des tâches techniques,
- Assurer un suivi rigoureux des objectifs par sprint

Les rôles Scrum ont été répartis de manière à garantir une gestion efficace du projet :

- Le rôle de **Product Owner** a été assuré par l'encadrant professionnel du projet, chargé de définir les besoins fonctionnels, les objectifs du système ainsi que les priorités des fonctionnalités à développer.
- Le rôle de **Scrum Master** a consisté à assurer le respect des pratiques Agile Scrum, à organiser les réunions de suivi et à faciliter la résolution des problèmes techniques rencontrés durant les différents sprints.
- Le rôle de **l'Équipe de développement** a été assuré par un seul ingénieur logiciel, responsable de la réalisation des tâches définies dans le *Product Backlog*, incluant la conception, le développement, l'intégration et les tests des différentes composantes de la plateforme.

L'adoption de cette approche Agile a permis d'assurer une amélioration progressive de la plateforme, une meilleure gestion des contraintes techniques ainsi qu'une évolution continue du système tout au long du projet.

1.4.4. Product Backlog

Le Product Backlog, présentée dans la table 3, regroupe l'ensemble des besoins fonctionnels et techniques du projet sous forme des *User Stories*. Ces besoins ont été priorisés selon leur importance fonctionnelle et leur impact sur le système global.

TABLE 3. Ectrait de User Stories du système décisionnel intelligent

ID	En tant que	Je souhaite	Afin de	Priorité
US1	Utilisateur métier	Analyser les indicateurs de performance de l'entreprise via des tableaux de bord interactifs	Faciliter et accélérer la prise de décision sans dépendre des équipes techniques	Haute
US2	Système (ETL)	Centraliser, nettoyer et transformer les données opérationnelles provenant des systèmes ERP dans un Data Warehouse modélisé en schéma en étoile	Disposer d'une source analytique unique, cohérente et performante, indépendante des bases transactionnelles	Haute
US3	Utilisateur métier	Interroger la plateforme analytique en langage naturel à travers un chat-bot intelligent	Obtenir automatiquement des analyses et des indicateurs décisionnels sans maîtriser le langage MDX	Haute
US4	Système (RAG)	Générer dynamiquement des requêtes MDX syntaxiquement valides à partir des intentions analytiques exprimées par l'utilisateur	Permettre l'interrogation intelligente des cubes OLAP sans intervention technique manuelle	Haute

ID	En tant que	Je souhaite	Afin de	Priorité
US5	Administrateur système	Gérer les comptes utilisateurs, les rôles et les droits d'accès via une interface d'administration sécurisée	Garantir la sécurité de la plateforme et contrôler l'accès aux données analytiques sensibles	Moyenne
US6	Utilisateur métier	Visualiser les résultats analytiques sous forme de graphiques et de tableaux interactifs adaptés aux données retournées	Interpréter rapidement les tendances, les indicateurs et les agrégats décisionnels sans recourir à un outil externe	Moyenne
US7	Administrateur de bases de données	Exécuter, superviser et planifier les processus ETL d'alimentation du Data Warehouse	Maintenir la fraîcheur, la qualité et la cohérence des données analytiques exploitées par les cubes OLAP	Moyenne
US8	Administrateur système	Superviser en temps réel l'état des conteneurs, des micro services et des ressources système via une interface de monitoring	Détecter et résoudre rapidement les incidents de production sans accès direct au serveur	Faible

Le backlog a été continuellement mis à jour tout au long du projet afin de prendre en compte les nouvelles contraintes techniques, les ajustements fonctionnels et les améliorations identifiées lors des différentes phases de développement et de validation. Cette approche agile a permis de maintenir une forte flexibilité tout en garantissant une progression cohérente du projet.

1.5. Conclusion

Ce chapitre a permis de poser le cadre général du projet en mettant en évidence les limites des systèmes décisionnels traditionnels et la nécessité d'une approche intelligente. La solution proposée repose sur une architecture moderne intégrant Data Warehouse, OLAP et Intelligence Artificielle, permettant une interaction naturelle avec les données. Le chapitre suivant sera consacré à l'analyse détaillée des besoins du système.

Chapitre 2

SPÉCIFICATION ET ANALYSE DES BESOINS

2.1. Introduction

Dans la continuité du chapitre précédent, cette partie a pour objectif de définir de manière précise les besoins du système proposé, ainsi que d'analyser les exigences fonctionnelles et non fonctionnelles nécessaires à sa conception.

Cette étape constitue un pilier fondamental dans le cycle de développement, puisqu'elle permet de traduire la problématique identifiée en un ensemble structuré de spécifications. Elle garantit également une meilleure compréhension entre les parties prenantes et sert de base à la conception technique de la solution.

2.2. Identification des acteurs

L'identification des acteurs permet de modéliser les interactions entre les utilisateurs et le système. Chaque acteur correspond à un profil ayant des responsabilités spécifiques et des droits d'accès bien définis. Dans le cadre de plateforme décisionnelle intelligente basée sur un Data Warehouse, des cubes OLAP et un chat-bot analytique, plusieurs acteurs interagissent avec le système selon des cas d'utilisation précis.

- **Utilisateur métier** : acteur principal du système. Il interagit avec la plateforme via une interface conversationnelle afin d'exprimer ses besoins analytiques en langage naturel.

Interactions principales :

- Formuler une requête analytique en langage naturel
- Consulter les résultats (indicateurs, agrégations OLAP)
- Visualiser des tableaux de bord dynamiques
- Raffiner une requête (itération conversationnelle)

Use Cases associés :

- UC1 : Interroger le système via chat-bot
- UC2 : Visualiser les résultats analytiques
- UC3 : Explorer les données multidimensionnel
- **Administrateur Base de Données** : responsable de la gestion et de l'évolution de la couche décisionnelle.

Interactions principales :

- Exécuter et planifier les processus ETL
- Maintenir et faire évoluer le schéma du Data Warehouse
- Définir et optimiser les cubes OLAP (dimensions, mesures, hiérarchies)
- Garantir la qualité et la cohérence des données

Use Cases associés :

- UC4 : Alimenter le Data Warehouse (ETL)
- UC5 : Gérer les dimensions et mesures OLAP
- UC6 : Optimiser les performances analytiques
- **Administrateur système** : responsable de la gestion technique et opérationnelle de la plateforme.

Interactions principales :

- Configurer les services (API, micro services, chat-bot)
- Gérer la sécurité (authentification, autorisation)
- Superviser les performances et la disponibilité du système
- Déployer les mises à jour et gérer les incidents

Use Cases associés :

- UC7 : Configurer l'infrastructure système
- UC8 : Gérer la utilisateurs et les accès
- UC9 : Superviser et monitorer les services

2.3. Besoins fonctionnels

Les besoins fonctionnels définissent précisément les services que le système doit fournir pour répondre aux attentes des utilisateurs métiers. Dans le cadre de ce projet, ces besoins, mentionné dans la Table 4, sont directement liés à l'objectif de concevoir une plateforme décisionnelle intelligente capable de combiner un Data Warehouse multidimensionnel, des cubes OLAP et un agent conversationnel analytique pour l'interrogation en langage naturel.

Ces besoins servent à garantir que le système réponde à la problématique initiale : les utilisateurs métiers n'ont pas de maîtrise technique suffisante pour interroger directement les bases de données opérationnelles, et les outils analytiques existants sont souvent statiques et peu interactifs.

TABLE 4. *Besoins fonctionnels de la plateforme décisionnelle intelligente*

Fonctionnalité	Description	Importance
Interrogation en langage naturel	Permettre à l'utilisateur de poser des questions analytiques en langage naturel. Supporter différentes formes de requêtes : agrégation, comparaison et évolution temporelle.	Supprime la barrière technique liée aux requêtes SQL/MDX et rend l'analyse accessible aux utilisateurs métiers.
Interprétation intelligente des requêtes	Identifier l'intention analytique de la requête utilisateur. Extraire automatiquement les mesures, dimensions et filtres pertinents.	Garantit des analyses précises et adaptées au contexte métier.
Enrichissement contextuel	Exploiter les métadonnées du Data Warehouse. Utiliser l'historique des requêtes pour améliorer la compréhension.	Améliore la pertinence des réponses et permet une interaction plus intelligente et personnalisée.
Génération automatique de requêtes MDX	Traduire les intentions analytiques en requêtes MDX exécutables. Optimiser les requêtes afin d'assurer de bonnes performances.	Automatise l'accès aux analyses multidimensionnel sans intervention technique.
Analyse multidimensionnel	Permettre l'exploration des données selon plusieurs dimensions. Supporter les opérations OLAP (<i>drill-down, roll-up, slice, dice</i>).	Offre une analyse flexible et approfondie des données décisionnelles.
Visualisation des résultats	Générer automatiquement des visualisations adaptées. Fournir des tableaux de bord interactifs.	Facilite la compréhension des données et accélère la prise de décision.
Gestion des données	Intégrer les données via des processus ETL. Nettoyer et transformer les données. Stocker les données dans un Data Warehouse.	Garantit la qualité, la cohérence et la fiabilité des analyses.

2.4. Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité et de performance du système. Ils sont essentiels pour garantir que la plateforme décisionnelle soit fiable, performante et adaptée aux utilisateurs métiers. Dans le cadre de ce projet, ces besoins, démontrés dans la Tableau 5, assurent que les utilisateurs puissent interagir de manière fluide avec les cubes OLAP, générer des tableaux de bord dynamiques et interroger le système via le agent conversationnel analytique sans rencontrer de limitations techniques.

TABLE 5. *Besoins non fonctionnels de la plateforme décisionnelle intelligente*

Catégorie	Description et justification
Performance	Le système doit offrir des temps de réponse rapides, proches du temps réel, afin de permettre des analyses multidimensionnelles interactives et de maintenir l'engagement des utilisateurs. L'optimisation des requêtes analytiques est essentielle pour traiter efficacement de grands volumes de données issus du Data Warehouse.
Scalabilité	La plateforme doit pouvoir gérer un volume croissant de données et d'utilisateurs. Une architecture basée sur les micro services permet d'ajouter ou de mettre à jour des composants sans impacter l'ensemble du système, garantissant ainsi son extensibilité future.
Disponibilité	Le système doit assurer un accès continu et minimiser les interruptions de service, afin de garantir que les utilisateurs puissent consulter les données et interroger les cubes OLAP à tout moment.
Sécurité	La protection des données sensibles est cruciale, notamment pour les informations clients et financières. La gestion des accès utilisateurs permet de contrôler les autorisations selon les rôles et d'assurer la conformité avec les réglementations.
Maintenabilité	Un code modulaire basé sur les micro services facilite la maintenance, l'évolution et la mise à jour du système, tout en réduisant les risques de dysfonctionnement lors des améliorations ou des corrections.
Utilisabilité	L'interface utilisateur doit être simple et intuitive, accessible même aux utilisateurs non techniques. Cela favorise l'adoption du système et réduit le besoin de formation approfondie.

2.5. Architecture de la solution

L'architecture représentée dans la figure 5 démontre la plateforme intelligente de prise de décision conçue sur une architecture microservices. Le but principal de cette plateforme est de permettre aux utilisateurs métier d'utiliser les données de l'entreprise de façon simple, rapide et intelligente, sans nécessiter la maîtrise des technologies complexes de Business Intelligence ou des langages de requêtes analytiques.

Le concept principal du système consiste à connecter divers composants spécialisés qui coopèrent : une interface web contemporaine, des microservices autonomes, une couche analytique fondée sur un Data Warehouse OLAP, ainsi qu'un moteur d'intelligence artificielle en mesure d'interpréter les interrogations formulées en langage naturel.

L'architecture a été conçue pour satisfaire diverses contraintes essentielles. Tout d'abord, le système doit être évolutif pour gérer l'accroissement du nombre d'utilisateurs et du volume des données. Deuxièmement, les éléments doivent être dissociés afin d'améliorer la maintenance et le déploiement autonome des services.

Enfin, l'ensemble doit fournir de bonnes performances analytiques tout en demeurant sécurisé et aisément surveillable. La communication entre les éléments s'appuie sur deux méthodes complémentaires. Les interactions rapides et critiques se servent de flux synchrones via des APIs REST, alors que les traitements asynchrones et les événements systèmes font appel à RabbitMQ en tant que bus de messages.

Cette dissociation contribue à renforcer la résilience du système et à prévenir le lien direct entre les microservices. L'architecture globale est structurée en différentes couches fonctionnelles. Chaque niveau a une fonction spécifique dans le fonctionnement global de la plateforme.

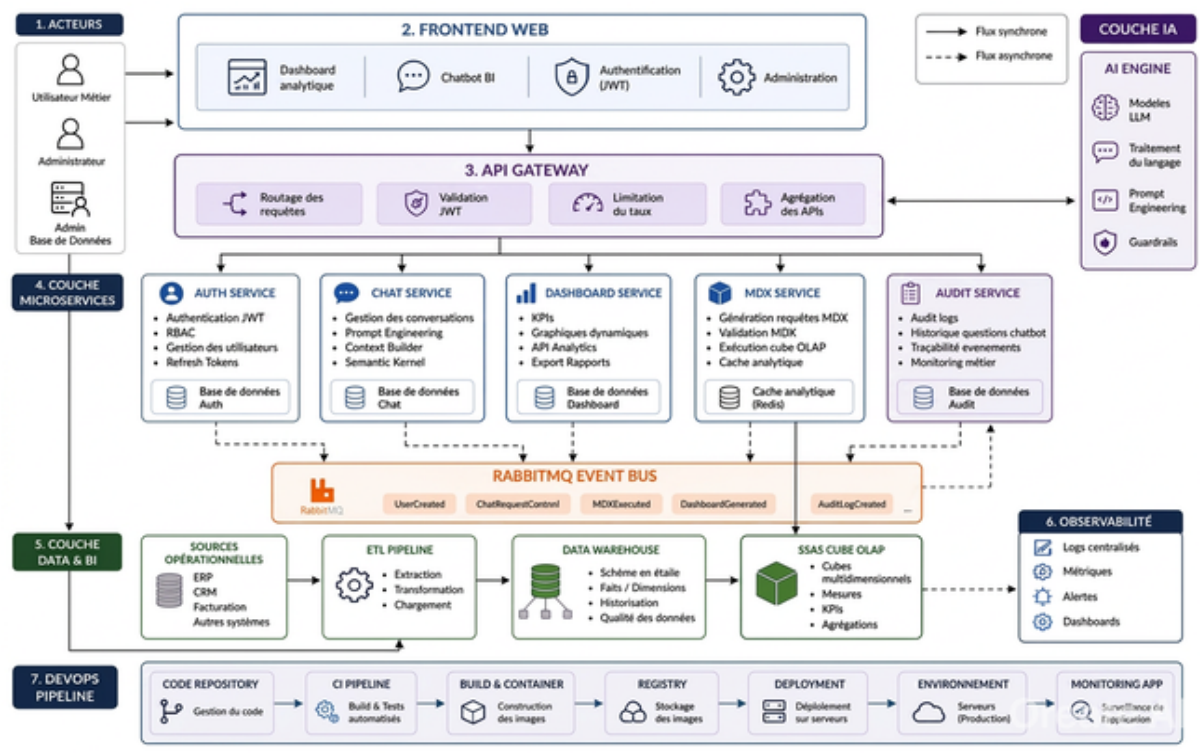


FIGURE 5. Architecture des Composants Globale de Système

2.6. Fondements Théoriques

Ce projet s'appuie sur divers principes techniques et méthodologiques garantissant l'efficacité, l'adaptabilité et l'utilisation maximale des données d'analyse. Ces principes établissent la structure générale du système ainsi que les diverses technologies employées dans le contexte du projet.

2.6.1. Concepts de base de l'informatique décisionnelle

L'informatique décisionnelle (*Business Intelligence - BI*) désigne un ensemble de méthodes, d'outils et de technologies permettant de collecter, organiser, transformer et analyser les données d'une organisation dans le but de faciliter la prise de décision [11, 12, 10].

Dans le contexte de ce projet, la BI joue un rôle central puisqu'elle permet d'exploiter les données issues des systèmes opérationnels afin de produire des analyses multidimensionnelles, des indicateurs métiers et des tableaux de bord interactifs.

L'objectif principal consiste à transformer les données brutes en informations exploitables grâce à un environnement analytique basé sur un *Data Warehouse*, des cubes OLAP et des outils de visualisation.

2.6.2. Architecture d'un système décisionnel

Il est important de rappeler que l'informatique décisionnelle, désigne un ensemble de techniques, d'outils et de méthodes permettant de collecter, consolider, modéliser et présenter les données de l'entreprise dans le but de faciliter la prise de décision et de donner une vue globale de l'activité [13]. Pour rendre ces données facilement exploitables à des fins décisionnelles. Le système de Business Intelligence suit un processus en quatre étapes principales [11] :

- La collecte des données,
- L'intégration et la transformation des données (ETL),
- La modélisation multidimensionnelle,
- L'analyse OLAP,
- La restitution des données via dashboards et chatbot analytique.

La figure 6 présente les différentes phases d'un système décisionnel.

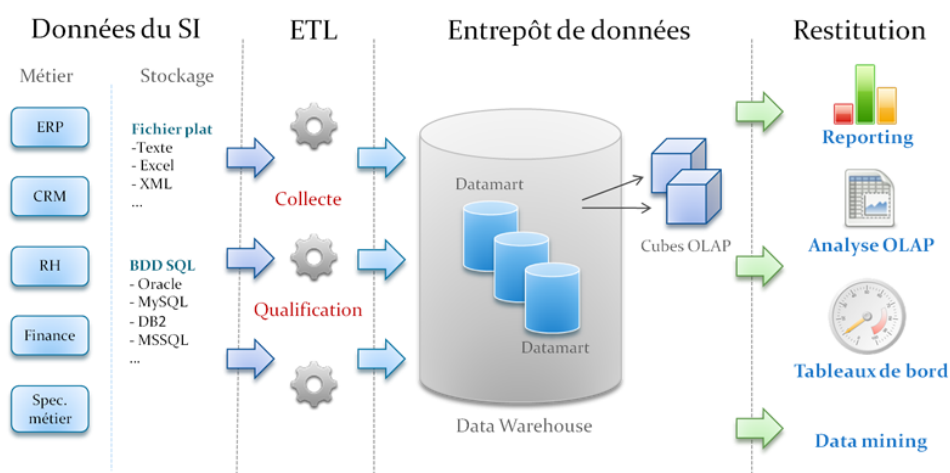


FIGURE 6. La chaîne décisionnelle

2.6.3. Les modèles de conception

La conception d'un Data Warehouse peut reposer sur plusieurs modèles multidimensionnels. Il existe trois types de modèles couramment utilisés : le modèle en étoile, le modèle en flocon de neige et le modèle en constellation de faits

2.6.3.1. Le modèle en étoile

Dans le modèle en étoile, indiqué à la Figure 7, la table de faits est placée au centre du schéma et les dimensions gravitent autour. Chaque dimension est directement connectée à la table de faits. Cela facilite l'analyse des données en permettant de visualiser rapidement les relations entre les différentes dimensions et les faits.

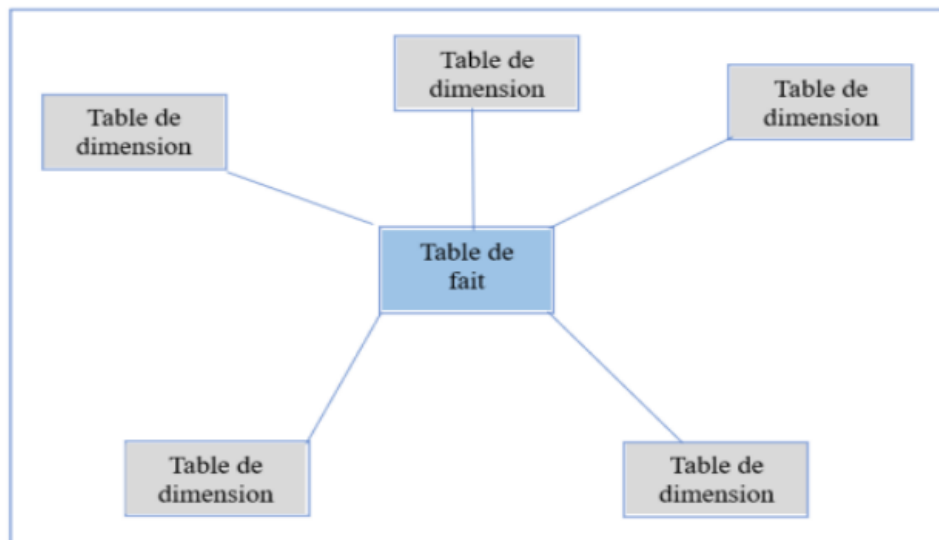


FIGURE 7. Schéma en étoile

2.6.3.2. Le modèle en flocon de neige

Le modèle en flocon, présenté à la Figure 8, utilise des hiérarchies de dimensions pour réduire le nombre de lignes par dimension. Cela permet de réduire la complexité de l'entrepôt de données et de faciliter l'analyse des données en permettant une vue plus claire et plus hiérarchique .

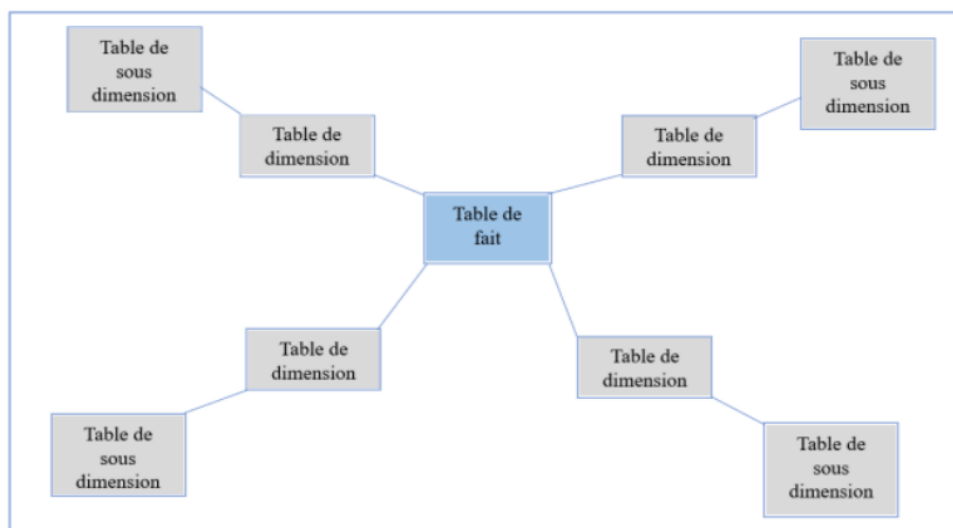


FIGURE 8. Schéma en Flocon de neige

2.6.3.3. Le modèle en constellation de faits

Le modèle en constellation de faits, dans la Figure 9, est une combinaison de schémas en étoiles et/ou en flocon, dans lequel les tables de fait partagent certaines tables de dimension. Cela permet une analyse plus détaillée des données en mettant en évidence les relations entre les différentes dimensions et les faits associés. Ce modèle est particulièrement utile pour les données complexes et multidimensionnelles.

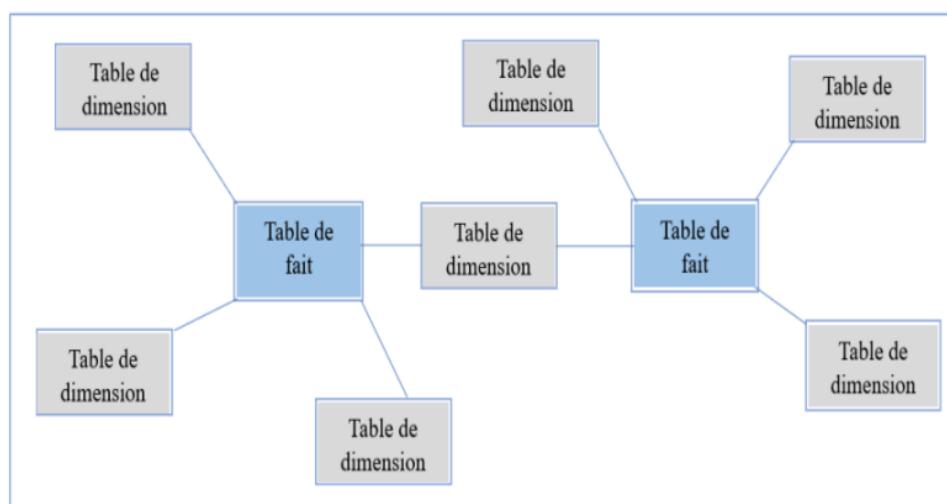


FIGURE 9. Schéma en Constellation de faits

Dans le cadre de ce projet, une modélisation multidimensionnelle adaptée aux besoins analytiques de la plateforme a été utilisée afin d'optimiser les performances des analyses OLAP.

2.6.4. Concept de l'Architecture RAG

Dans le cadre de notre plateforme décisionnelle intelligente, une architecture a été adoptée qu'il se base sur le paradigme *Retrieval-Augmented Generation* afin de permettre l'interrogation du système en langage naturel et la génération automatique de requêtes analytiques en MDX [14].

Cette approche combine les capacités des modèles de langage (*Large Language Models - LLM*) avec une source de connaissances externe, garantissant ainsi des réponses plus précises, contextualisées et adaptées au domaine décisionnel [15].

L'architecture RAG repose principalement sur deux phases complémentaires : la récupération des connaissances (*Retrieval*) et la génération contextuelle (*Generation*) [14].

2.6.4.1. Phase de récupération (*Retrieval*)

Cette étape consiste à rechercher des informations pertinentes à partir d'une base de connaissances externe. Dans notre cas, cette base est constituée d'un **fichier de métadonnées du cube OLAP**, contenant :

- les dimensions (temps, clients, produits, régions) ;
- les hiérarchies multidimensionnelles ;
- les mesures analytiques (chiffre d'affaires, profit, quantité) ;
- les relations entre les différents axes du cube OLAP.

Ces métadonnées sont transformées en représentations vectorielles (*embeddings*) à l'aide d'un modèle *SentenceTransformer*, puis indexées dans un moteur de recherche vectorielle FAISS

afin de permettre une recherche sémantique rapide et efficace [16, 17].

2.6.4.2. Phase d'augmentation (*Augmentation*)

La phase d'augmentation représente l'étape intermédiaire entre la récupération des connaissances et la génération de la réponse finale. Son objectif principal est d'enrichir dynamiquement le contexte envoyé au modèle de langage (*LLM*) en intégrant les informations récupérées lors de la phase de *Retrieval*.

Dans notre architecture, les métadonnées pertinentes extraites depuis l'index vectoriel FAISS sont utilisées pour construire un *prompt contextuel enrichi*. Ce contexte contient notamment :

- les dimensions pertinentes du cube OLAP ;
- les hiérarchies multidimensionnelles associées ;
- les mesures analytiques disponibles ;
- les relations entre faits et dimensions ;
- des exemples de structures MDX ;
- des contraintes métier spécifiques au domaine analytique.

Le système combine ensuite ces informations avec la requête utilisateur exprimée en langage naturel afin de produire un prompt structuré et contextualisé destiné au modèle génératif.

2.6.4.3. Phase de génération (*Generation*)

Une fois les informations pertinentes récupérées, celles-ci sont injectées dans le contexte du LLM. Le modèle exploite alors les métadonnées récupérées afin de :

- interpréter la requête utilisateur exprimée en langage naturel ;
- générer une requête MDX valide et cohérente ;
- contextualiser la réponse analytique ;
- réduire les erreurs liées aux hallucinations du modèle.

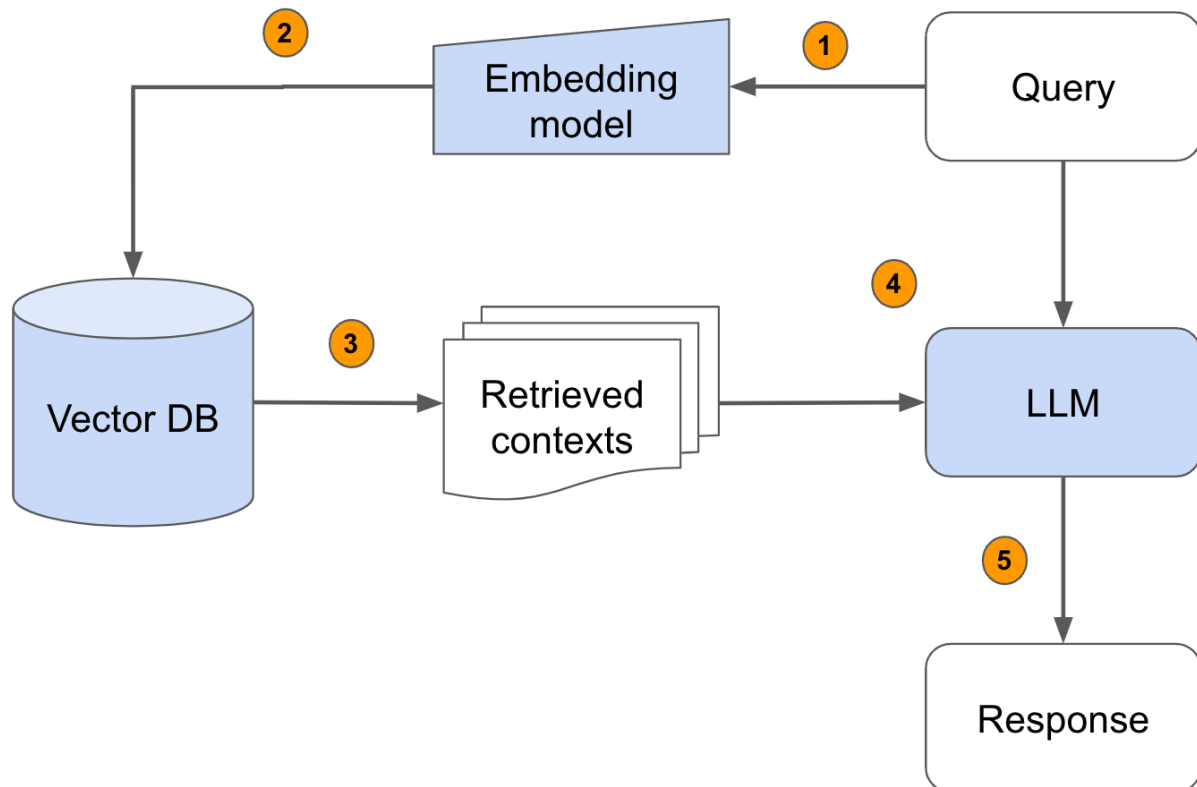


FIGURE 10. Architecture globale du système RAG

2.6.5. Visualisation

L'interface utilisateur de l'application repose sur une architecture modulaire, cohérente et évolutive, implémentée à l'aide de la bibliothèque **React avec TypeScript (React TS)**. Ce choix technologique permet de bénéficier d'un typage statique, d'une meilleure maintenabilité du code ainsi que d'une détection précoce des erreurs [18, 19].

L'architecture adoptée s'inspire des principes de l'*Atomic Design*, combinée à l'utilisation d'une librairie UI spécialisée. Cette approche garantit à la fois une expérience utilisateur fluide et une forte réutilisabilité des composants [20].

2.6.5.1. Modélisation par Atomic Design

L'architecture frontend s'appuie sur le modèle *Atomic Design*, qui structure les interfaces en cinq niveaux hiérarchiques :

Atomes : Les atomes représentent les éléments de base de l'interface utilisateur, tels que les boutons, les champs de saisie et les icônes.

Molécules : Les molécules sont des combinaisons d'atomes formant des composants simples mais fonctionnels, comme un champ de formulaire accompagné de son label ou un groupe de boutons.



FIGURE 11. *Architecture Atomic de Frontend*

Organismes : Les organismes correspondent à des blocs fonctionnels plus complexes, tels que les formulaires complets, les en-têtes (*headers*) ou les barres latérales (*sidebars*).

Templates : Les templates définissent la structure globale des pages sans contenu spécifique, en organisant les organismes et les composants.

Pages : Les pages représentent les vues complètes de l'application, instanciées avec des données réelles.

L'utilisation de ce modèle, combinée à une librairie UI préconstruite compatible avec React TS, a permis :

- d'accélérer le développement de l'interface utilisateur,
- d'assurer une homogénéité visuelle et fonctionnelle,
- de faciliter les tests unitaires grâce à la modularité des composants.

2.6.5.2. Gestion des styles

La gestion de l'interface graphique repose principalement sur l'utilisation de **Tailwind CSS**, permettant de construire une interface moderne, responsive et facilement maintenable au sein de l'application React [21].

L'approche adoptée combine plusieurs technologies complémentaires :

- **Tailwind CSS** pour la gestion principale du style et du layout de l'application ;
- **Material UI Icons** pour l'intégration d'icônes vectorielles modernes et cohérentes avec les composants interactifs de l'interface [22] ;
- des **classes CSS personnalisées** utilisées principalement pour les composants graphiques et analytiques nécessitant des ajustements spécifiques liés aux dashboards et aux visualisations de données.

2.6.6. Concept DevOps

Le concept Dev+Ops est une approche méthodologique conçue pour faciliter la collaboration et la communication entre les équipes de développement logiciel (Dev) et d'exploitation informatique

(Ops). L'objectif principal de DevOps est de réduire les frictions entre ces deux équipes en facilitant un processus de développement logiciel plus efficace, rapide et fiable.

Le processus DevOps est une approche collaborative qui aligne les équipes de développement logiciel et d'exploitation pour optimiser l'efficacité, la qualité et la rapidité de livraison des logiciels. Dans la figure 12, la section bleue illustre les processus de développement, tandis que la section orange correspond aux processus opérationnels.

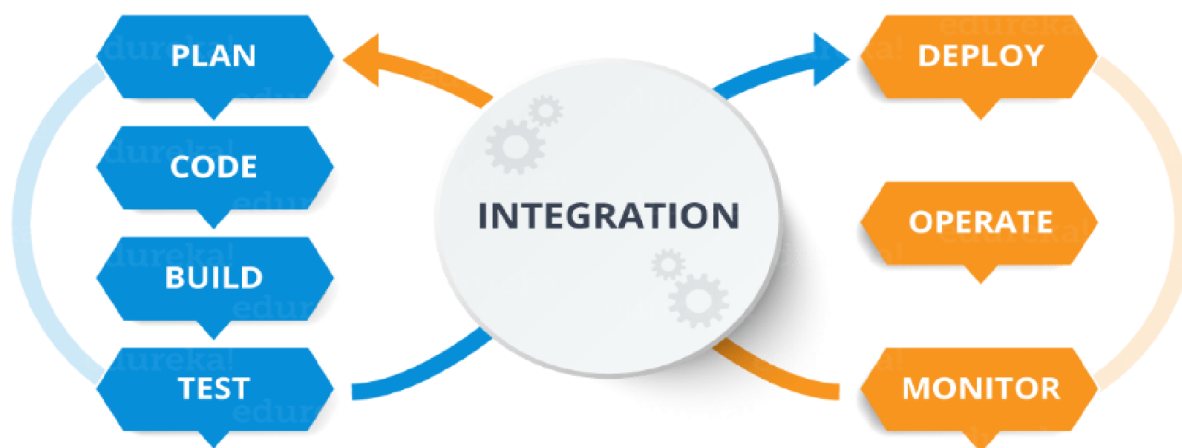


FIGURE 12. Processus DevOps

2.7. Technologies utilisées

Cette section est dédiée à la présentation des technologies et outils logiciels complets qu'ils sont utilisés pour mener à bien ce projet. Le tableau 6 synthétise l'ensemble des technologies retenues pour la réalisation de la plateforme décisionnelle, organisées par couche fonctionnelle.

TABLE 6. Récapitulatif des technologies utilisées dans la plateforme décisionnelle

Couche	Technologie	Rôle et justification
Données & Intégration	SQL Server	SGBDR principal du Data Warehouse ; robustesse, intégration native avec l'écosystème Microsoft et performances analytiques élevées [23].
	SSIS	Outil ETL Microsoft pour l'automatisation des flux de données entre les systèmes sources et le Data Warehouse [24].

Couche	Technologie	Rôle et justification
	SSAS	Création des cubes OLAP et exécution de requêtes multidimensionnelles via MDX; pré-calcul des agrégations[25].
Backend	ASP.NET Core	Framework principal pour les microservices; performances élevées, compatibilité multiplateforme et support natif Docker [26].
	Python FastAPI	Services IA et traitement analytique avancé; documentation OpenAPI automatique et intégration native avec les bibliothèques ML [27].
	API Gateway (YARP)	Centralisation du routage, de la sécurité et du contrôle d'accès entre le frontend et les microservices [28].
	Entity Framework Core	ORM pour l'accès aux données relationnelles via une approche orientée objet[26].
	Dapper	Accès SQL optimisé pour les requêtes critiques nécessitant de hautes performances [29].
	ADOMD.NET	Exécution des requêtes MDX sur les cubes OLAP SSAS [30].
Frontend	React.js	Construction d'interfaces utilisateur dynamiques et modulaires; gestion de l'état et optimisation via le Virtual DOM [31].
	Chart.js	Génération de graphiques interactifs adaptés aux tableaux de bord métier [32].
	D3.js	Visualisation avancée des données complexes et des indicateurs analytiques [33].
Intelligence Artificielle	LLM	Interprétation des requêtes en langage naturel et transformation en requêtes analytiques exploitables [34].

Couche	Technologie	Rôle et justification
	RAG	Amélioration de la pertinence des réponses du chatbot par récupération de contexte depuis les sources internes ; réduction des hallucinations [14].
	Ollama Server	Déploiement et orchestration locale des modèles LLM ; contrôle des données sensibles et indépendance vis-à-vis du cloud [35].
DevOps & Infrastructure	Docker	Conteneurisation de l'ensemble des services ; portabilité et isolation des environnements d'exécution [36].
	Docker Compose	Orchestration coordonnée de tous les conteneurs via un fichier de configuration unique [37].
	Git	Contrôle de version distribué ; suivi de l'historique des modifications et gestion des branches [38].
	GitHub Actions	Pipelines CI/CD automatisés pour la compilation, les tests et le déploiement des microservices [39].
	Portainer	Supervision et administration graphique de l'infrastructure Docker ; visibilité centralisée sur les conteneurs, images et réseaux [40].
Environnements de développement	Visual Studio 2022	IDE principal pour les microservices ASP.NET Core ; débogage, tests unitaires, profiling et intégration SQL Server/SSIS/SSAS [41].
	Visual Studio Code	Éditeur polyvalent pour le frontend React.js, les services Python et les fichiers de configuration DevOps [42].

2.8. Conclusion

Ce chapitre a permis de formaliser les besoins du système en identifiant les exigences fonctionnelles et non fonctionnelles, ainsi que les contraintes techniques et organisationnelles.

L'analyse réalisée met en évidence la nécessité d'une architecture hybride combinant Business Intelligence et Intelligence Artificielle pour répondre efficacement aux problématiques identifiées. Le chapitre suivant sera consacré à la conception et à la mise en œuvre de la première version fonctionnelle de la plateforme

Chapitre 3

FONDATION FONCTIONNELLE – RELEASE 1

3.1. Introduction

Ce chapitre présente la première release fonctionnelle de la plateforme décisionnelle intelligente, structurée en deux sprints complémentaires dont les livrables constituent le socle technique sur lequel reposent les releases ultérieures.

Le Sprint 1 établit la couche *Data-Centric* du système : analyse des sources opérationnelles issues d'un ERP à plus de 30 tables, conception du Data Warehouse selon un schéma en étoile, implémentation des processus ETL sous SSIS, construction du cube OLAP multidimensionnel sous SSAS et exposition des indicateurs via un micro service MDX dédié. Cette infrastructure garantit la disponibilité de données cohérentes, historisées et optimisées pour l'analyse décisionnelle.

Le Sprint 2 introduit la couche *AI-Centric* : intégration d'un modèle de langage via le paradigme *Retrieval-Augmented Generation*, vectorisation des métadonnées OLAP avec FAISS, et génération automatique de requêtes MDX à partir du langage naturel. Cette couche transforme la plateforme décisionnelle classique en un système interactif capable de répondre à des questions analytiques sans connaissance technique préalable de la syntaxe MDX.

L'ensemble s'inscrit dans une démarche d'ingénierie logicielle incrémentale : chaque composant est conçu de manière modulaire afin de faciliter son intégration dans l'architecture micro services de la Release 2.

3.2. Sprint 1 : Mise en place de la couche Data

Le premier sprint constitue la fondation décisionnelle du projet. Son objectif principal est de mettre en place une infrastructure analytique robuste permettant de centraliser, nettoyer et structurer les données issues des systèmes opérationnels.

Ce sprint introduit principalement la couche *Data-Centric* du système, en mettant l'accent sur :

- la conception du Data Warehouse ;
- l'implémentation des processus ETL ;
- la préparation et le nettoyage des données ;
- la construction du cube OLAP multidimensionnel.

Cette étape est essentielle puisqu'elle garantit la disponibilité de données cohérentes, historisées et optimisées pour l'analyse décisionnelle. Le Tableau 7 décrit les fonctionnalités majeures attendues pendant ce sprint.

TABLE 7. Backlog du Sprint 1 : Mise en place de la couche Data

ID	Rôle	Besoin	Bénéfice	Critère d'acceptation	Priorité	Semaine
US1.1	Analyste métier	Centraliser les données des systèmes ERP (30+ tables) dans un Data Warehouse structuré en schéma en étoile	Disposer d'une source analytique unique, cohérente et historisée	Les données ERP sont intégrées dans le DW ; schéma en étoile validé	Haute	S1-
US1.2	Administrateur BI	Nettoyer les données sources via des pipelines ETL SSIS automatisés	Assurer la qualité et la fiabilité des analyses décisionnelles	Taux de doublons = 0 % ; complétude $\geq$ 99 %	Haute	S2
US1.3	Utilisateur décisionnel	Analyser les ventes selon plusieurs dimensions avec drill-down, roll-up et slice & dice	Obtenir des KPI pertinents sans requêtes SQL manuelles	Cube OLAP SSAS déployé ; opérations multidimensionnelles fonctionnelles	Haute	S3
US1.4	Utilisateur métier	Consulter des agrégations rapides via un micro service MDX REST	Réduire le temps d'analyse et accélérer la prise de décision	Requêtes MDX exécutées en moins de 2 secondes ; Swagger disponible	Moyenne	S4

ID	Rôle	Besoin	Bénéfice	Critère d'acceptation	Priorité	Semaine
US1.5	Administrateur BI	Tracer l'historique des exécutions ETL et les lignes rejetées	Garantir la traçabilité du pipeline	Tables ETL_Log et ETL_Quarantaine alimentées automatiquement	Basse	S4

3.2.1. Analyse des sources de données

La première étape du projet consiste à analyser la source de données issue du système opérationnel existant, à savoir l'ERP (*Enterprise Resource Planning*) et sa base de données relationnelle transactionnelle. Cette phase est essentielle dans tout projet décisionnel : elle permet de comprendre la structure des données, leur qualité, leur cohérence et leur pertinence par rapport aux besoins analytiques de l'entreprise.

Le système source contient un volume important de données réparties sur plus de 35 tables relationnelles couvrant plusieurs domaines métiers, notamment la gestion des ventes, les commandes clients, les transactions financières, la gestion des articles et des fournisseurs, ainsi que la logistique et les mouvements d'inventaire.

Cette base de données a été conçue pour répondre à des besoins transactionnels (*OLTP – Online Transaction Processing*), dont l'objectif est d'assurer la rapidité des opérations quotidiennes. Ce type d'architecture n'est cependant pas adapté aux traitements analytiques complexes (*OLAP – Online Analytical Processing*), notamment lorsqu'il s'agit d'effectuer des agrégations massives ou des analyses multidimensionnels.

L'analyse des données sources a permis d'identifier plusieurs limitations majeures :

- présence de données incohérentes ou dupliquées ;
- valeurs manquantes (NULL) dans plusieurs attributs importants ;
- redondance d'informations entre différentes tables ;
- conventions de nommage hétérogènes et absence de standardisation ;
- performances dégradées lors de l'exécution de requêtes analytiques volumineuses ;
- forte normalisation rendant les jointures SQL analytiques complexes à écrire et à maintenir.

Face à ces limitations, la mise en place d'un *Data Warehouse* s'impose comme solution adaptée, permettant de séparer les traitements transactionnels des traitements analytiques tout en améliorant la qualité, la cohérence et l'accessibilité des données.

3.2.2. Identification du sujet analytique

Le domaine d'analyse retenu est celui des ventes et des performances commerciales, en réponse à un besoin métier explicitement exprimé par la société d'accueil. Les responsables souhaitent un système capable de fournir des analyses rapides et interactives sur l'activité commerciale, considérée comme l'axe stratégique principal de l'entreprise.

Dans le système opérationnel existant, les données de vente sont distribuées sur plusieurs tables transactionnelles et nécessitent des requêtes SQL complexes pour produire le moindre rapport analytique. Cette situation génère des temps de réponse élevés, une dépendance forte aux équipes techniques et une absence totale d'interrogation en langage naturel.

La table de faits construite résulte d'une jointure gauche ('*LEFT JOIN*') entre les tables `BonCommand` et `BonCommandDetails` via la clé `UniqueIDDetails`. Elle centralise les transactions commerciales et constitue le noyau analytique du Data Warehouse, associé aux dimensions **Client (Tiers)**, **Produit (Article)**, **Temps** et **Localisation**.

Les indicateurs clés de performance (KPI) exposés par ce modèle couvrent le chiffre d'affaires, les quantités vendues, le nombre de commandes, les marges commerciales et les performances par client, par produit et par période. Ces mesures alimentent à la fois les tableaux de bord décisionnels du frontend et le système de génération automatique de requêtes MDX décrit au Sprint 3.

3.2.3. Conception logique du Data Warehouse

Le modèle multidimensionnel retenu dans ce projet repose sur une architecture de type **schéma en étoile (Star Schema)**. Ce modèle est largement adopté dans les systèmes décisionnels modernes en raison de sa simplicité conceptuelle et de ses performances élevées pour les traitements analytiques [11].

Dans cette architecture, une table de faits centrale est entourée de plusieurs tables de dimensions, formant une structure en étoile comme illustré dans la Figure 13. Cette organisation permet de séparer clairement les données analytiques (faits) des axes d'analyse (dimensions), facilitant ainsi les requêtes OLAP et la compréhension métier.

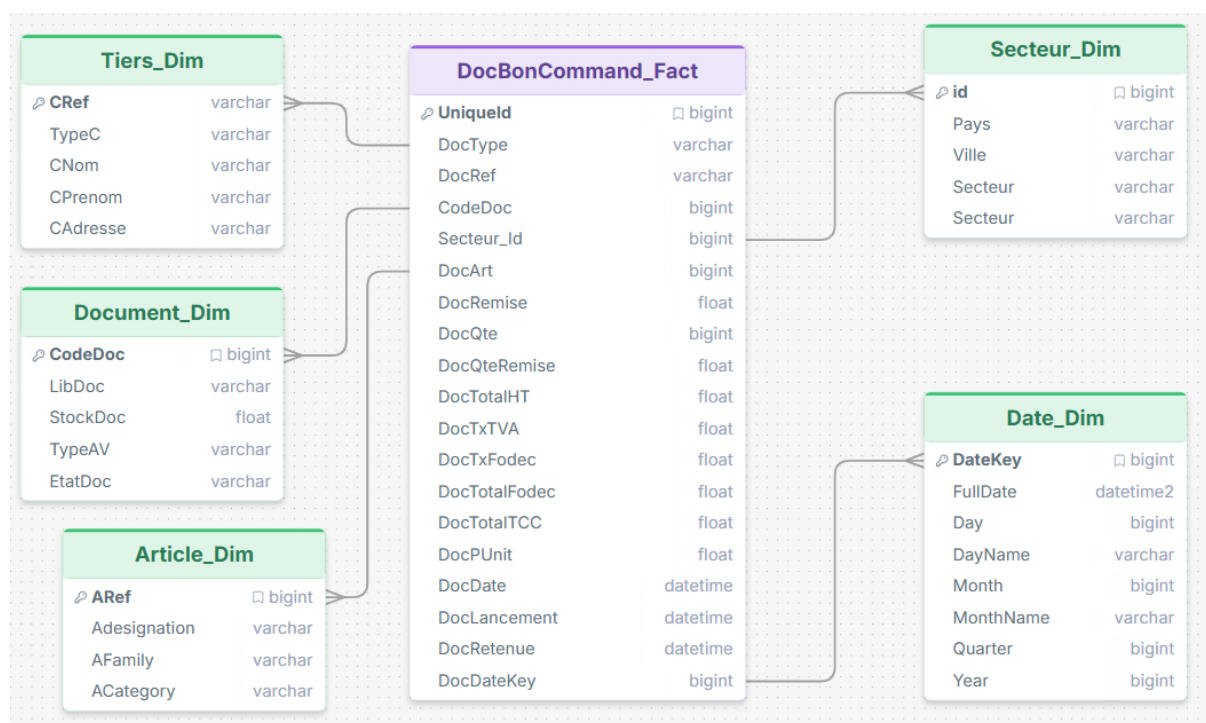


FIGURE 13. Schémas en étoile de Data Warehouse

Le schéma en étoile constitue le choix optimal pour ce projet. Il combine simplicité, performance et compatibilité avec les outils BI modernes. Il constitue la base du Data Warehouse et permet ensuite l'exploitation avancée via les cubes OLAP et les services d'intelligence artificielle intégrés.

3.2.3.1. Table de faits

La **table de faits** constitue le cœur du Data Warehouse. Elle est conçue à un niveau de granularité fin, correspondant ici aux lignes de commande (OrderLines).

Elle contient principalement :

- des **mesures quantitatives** : montant de vente, quantité, prix unitaire, taxes, total ;
- des **clés étrangères** vers les dimensions (client, produit, temps, etc.) ;
- un niveau de granularité atomique permettant des agrégations flexibles.

Cette table est optimisée pour les opérations analytiques telles que SUM, COUNT, AVG, essentielles dans les systèmes OLAP.

3.2.3.2. Tables de dimensions

Les **tables de dimensions** permettent de contextualiser les faits et d'analyser les données selon différents axes métier :

- **Dimension Temps** : analyse temporelle (jour, mois, année, trimestre) ;
- **Dimension Client** : segmentation des clients (catégorie, localisation, profil) ;

- **Dimension Produit** : caractéristiques produit (couleur, catégorie, stock) ;
- **Dimension Géographie** : analyse par ville, région et pays ;
- **Dimension Employé / Vendeur** : suivi de la performance commerciale.

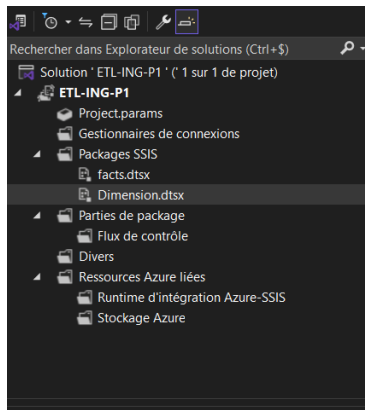
Ces dimensions permettent de réaliser des opérations analytiques avancées telles que : *drill-down, roll-up, slice & dice* et *pivot*.

3.2.4. Implémentation du processus ETL

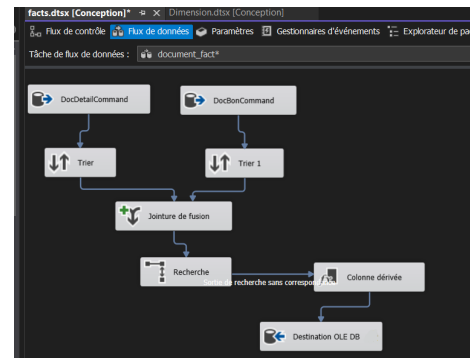
Le processus ETL a été implémenté à l'aide du framework SQL Server Integration Services (SSIS)[24]. Il permet d'extraire les données depuis la base opérationnelle, de les transformer selon les règles métier définies, puis de les charger dans le Data Warehouse.

L'architecture ETL est organisée en plusieurs flux de données (Data Flows), chacun étant dédié à un composant spécifique du modèle multidimensionnel, notamment la table de faits et les différentes dimensions.

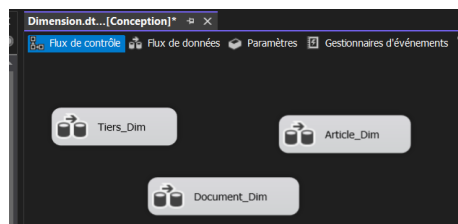
Les sous figures dans la Figure 14 illustrent les principaux flux ETL développés, incluant le chargement de la table de faits, des dimensions, ainsi que les transformations utilisées telles que *Lookup, Sort, Merge* et *Derived Column*.



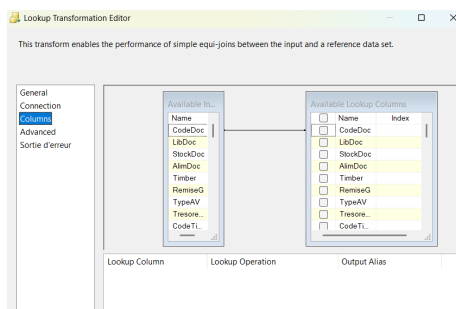
(a) Structure de Solution SSIS



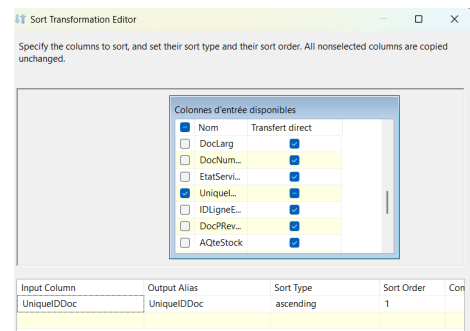
(b) Flux de Fait



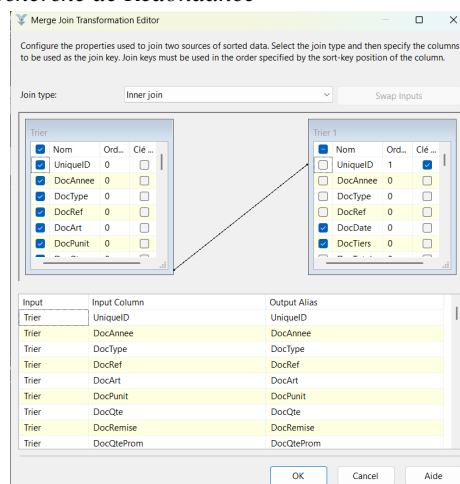
(c) Flux de dimension



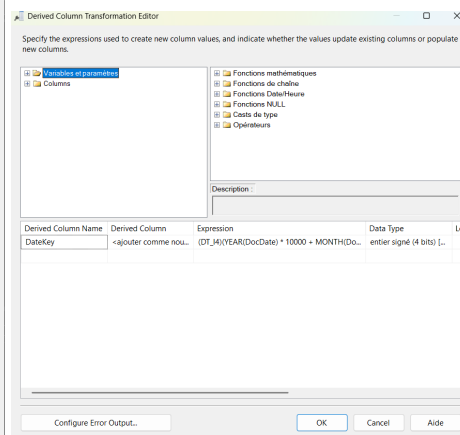
(d) Recherche de Redondance



(e) Trie sur la clé de jointure



(f) Jointure de Fusion



(g) Extension de colonne

FIGURE 14. Implémentation des flux ETL sous SSIS pour le chargement du Data Warehouse

Par ailleurs, la dimension temporelle (Date) a été générée directement à l'aide de requête SQL, couvrant une période allant de 2000 à 2030, afin de garantir une analyse temporelle complète et indépendante des sources opérationnelles. La requête utiliser pour cet dimension est démontré

dans l'extrait de Code 1.

Des mécanismes de gestion des erreurs et de journalisation ont également été intégrés afin d'assurer la robustesse et la traçabilité du processus ETL. Enfin, l'optimisation des performances a été prise en compte à travers l'utilisation de transformations adaptées et une gestion efficace des volumes de données.

```
1
2 -- Creation de la table DimDate
3 CREATE TABLE DimDate (
4     DateKey INT PRIMARY KEY,
5     FullDate DATE,
6     Day INT,
7     Month INT,
8     Year INT,
9     Quarter INT,
10    MonthName NVARCHAR(20),
11    DayOfWeek INT,
12    DayName NVARCHAR(20),
13    IsWeekend BIT
14 );
15
16 -- Variables de boucle
17 DECLARE @StartDate DATE = '2000-01-01';
18 DECLARE @EndDate DATE = '2030-12-31';
19
20 WHILE @StartDate <= @EndDate
21 BEGIN
22     INSERT INTO DimDate (
23         DateKey,
24         FullDate,
25         Day,
26         Month,
27         Year,
28         Quarter,
29         MonthName,
30         DayOfWeek,
31         DayName,
32         IsWeekend
33     )
34     VALUES (
35         CONVERT(INT, FORMAT(@StartDate, 'yyyyMMdd')),
36         @StartDate,
37         DAY(@StartDate),
38         MONTH(@StartDate),
39         YEAR(@StartDate),
40         DATEPART(QUARTER, @StartDate),
41         DATENAME(MONTH, @StartDate),
42         DATEPART(WEEKDAY, @StartDate),
43         DATENAME(WEEKDAY, @StartDate),
44         CASE
```

```

45         WHEN DATEPART(WEEKDAY, @StartDate) IN (1,7) THEN 1
46         ELSE 0
47     END
48 );
49
50     SET @StartDate = DATEADD(DAY, 1, @StartDate);
51 END;

```

Code 1 – Génération de la dimension Date (2000-2030)

3.2.5. Nettoyage et préparation des données

Afin de garantir la qualité et la fiabilité des données intégrées dans le Data Warehouse, plusieurs opérations de nettoyage et de préparation ont été mises en place en amont du processus ETL.

Ces opérations portent principalement sur :

- la gestion des valeurs nulles et incohérentes,
- la détection et la suppression des doublons,
- la normalisation des formats de données,
- la définition de contraintes d'intégrité (clés primaires).

Dans un premier temps, une analyse des données sources a été réalisée afin d'identifier les enregistrements dupliqués. Par exemple, la table *Tiers* a été inspectée pour détecter les occurrences multiples d'une même clé métier (*CRef*) par le Code 2.

```

1 SELECT *
2 FROM [pfe-ing-ds].[dbo].[Tiers]
3 WHERE CRef IN (
4     SELECT CRef
5     FROM [pfe-ing-ds].[dbo].[Tiers]
6     GROUP BY CRef
7     HAVING COUNT(*) > 1
8 )
9 ORDER BY CRef;

```

Code 2 – Détection des doublons dans la table Tiers

Par la suite et à la Code 3, les doublons ont été supprimés à l'aide de la fonction (*ROW_NUMBER*) qu'il permette de conserver uniquement une occurrence par clé.

```

1 WITH Duplicates AS (
2     SELECT *,
3     ROW\_NUMBER() OVER(PARTITION BY CRef ORDER BY CRef) AS RowNum
4     FROM Tiers\_Dim
5 )
6 DELETE FROM Duplicates
7 WHERE RowNum > 1;

```

Code 3 – Suppression des doublons avec ROW_NUMBER

Concernant la qualité des attributs, des vérifications ont été effectuées pour identifier les valeurs nulles ou invalides, notamment sur la dimension temporelle.

```

1 SELECT FullDate
2 FROM [ING-PFE-DW].[dbo].[Dim\_Date]
3 WHERE TRY\_CONVERT(datetime, FullDate) IS NULL
4 AND FullDate IS NOT NULL;
5
6 SELECT *
7 FROM [ING-PFE-DW].[dbo].[Dim\_Date]
8 WHERE LTRIM(RTRIM(FullDate)) = '';
```

Code 4 – Vérification des valeurs invalides dans Dim_Date

Afin d'assurer la cohérence des données, des contraintes d'intégrité ont été définies sur les tables dimensionnelles, notamment via l'ajout de clés primaires et la restriction des valeurs nulles.

```

1 ALTER TABLE Tiers\_Dim
2 ALTER COLUMN CRef NVARCHAR(20) NOT NULL;
3
4 ALTER TABLE Tiers\_Dim
5 ADD CONSTRAINT PK\_Dim\_Tiers PRIMARY KEY (CRef);
6
7 ALTER TABLE Article\_Dim
8 ALTER COLUMN ARef VARCHAR(20) NOT NULL;
9
10 ALTER TABLE Article\_Dim
11 ADD CONSTRAINT PK\_Dim\_Article PRIMARY KEY (ARef);
12
13 ALTER TABLE Document\_Dim
14 ALTER COLUMN CodeDoc VARCHAR(3) NOT NULL;
15
16 ALTER TABLE Document\_Dim
17 ADD CONSTRAINT PK\_Document\_Dim PRIMARY KEY (CodeDoc);
```

Code 5 – Ajout de contraintes d'intégrité

Enfin, ces transformations permettent d'assurer la cohérence globale des données entre les différentes sources et le Data Warehouse, en garantissant l'unicité des clés, la validité des formats et le schémas mentionné dans la Figure 13.

Cette étape constitue un prérequis essentiel pour garantir la fiabilité des analyses multidimensionnels et des requêtes OLAP exploitées par le système décisionnel.

3.2.6. Construction du Cube OLAP

Le cube OLAP a été conçu à l'aide de la framework SQL Server Analysis Services, fournit par Microsoft[25], afin de structurer les données du Data Warehouse sous une forme multidimensionnel, facilitant ainsi leur exploration et leur analyse.

La conception du cube a suivi plusieurs étapes clés, illustrées par les captures d'écran présentées dans la Figure 15, depuis la définition de la source de données jusqu'au déploiement et au traitement du cube.

Dans un premier temps, la source de données et le Data Source View (DSV) ont été configurés afin de représenter le schéma en étoile du Data Warehouse, incluant la table de faits et les différentes tables de dimensions.

Ensuite, les dimensions ont été définies en spécifiant leurs attributs et hiérarchies, notamment la dimension temporelle, permettant des analyses par année, trimestre, mois et jour. Ces hiérarchies facilitent la navigation multidimensionnel et l'exploration des données.

La table de faits a été intégrée en définissant les mesures principales telles que le chiffre d'affaires, les quantités vendues et le nombre de transactions. Des agrégations ont été configurées afin d'optimiser les performances des requêtes et réduire le temps de réponse lors de l'analyse.

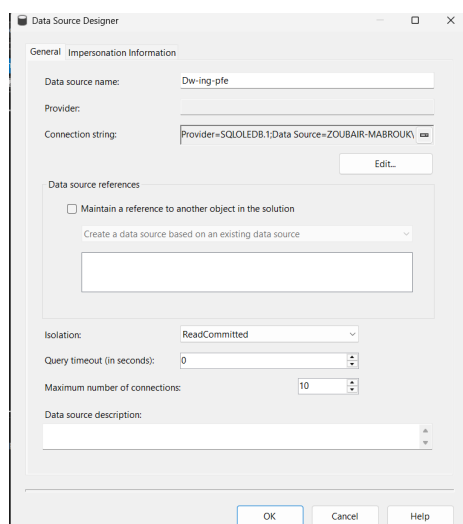
Le raffinement des mesures et des indicateurs clés de performance (KPI) a été réalisé directement au sein du cube à l'aide du langage MDX. Cela inclut la définition de mesures calculées et d'indicateurs permettant des analyses avancées, telles que :

- les comparaisons temporelles (année en cours vs année précédente),
- les taux de croissance,
- les agrégations conditionnelles.

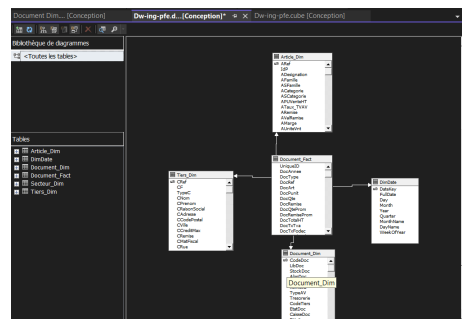
Ces calculs permettent d'enrichir le cube en fournissant une couche analytique avancée, adaptée aux besoins décisionnels.

Enfin, le cube a été déployé et traité (processing) afin de rendre les données disponibles pour l'interrogation via des requêtes MDX et leur exploitation dans les outils de visualisation et le chat-bot analytique.

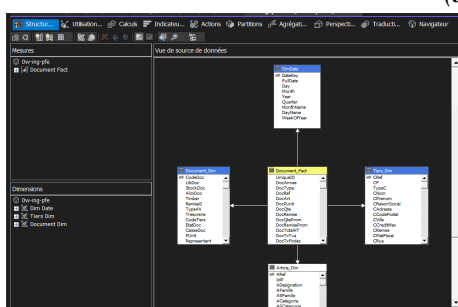
Ainsi, le cube OLAP constitue le cœur du système décisionnel, offrant une structure optimisée pour l'analyse multidimensionnel et une base solide pour la génération dynamique de requêtes analytiques.



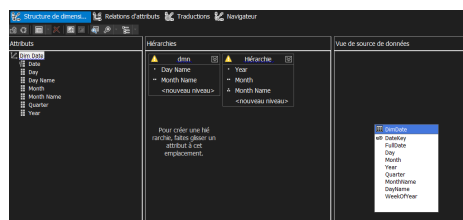
(a) Source de Donnée



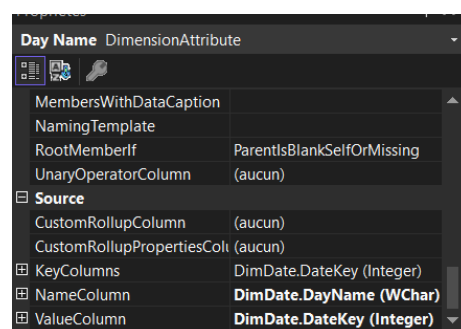
(b) Source de Donnée Virtuelle



(c) Cube OLAP



(d) Hiérarchit



(e) Raffinement

FIGURE 15. Étapes de conception et de configuration du cube OLAP sous SQL Server Analysis Services

3.2.7. Mise en place du Mdx Service

Dans une architecture micro services dédiée à l'analyse décisionnelle, l'exécution de requêtes MDX (Multidimensional Expressions) sur des cubes OLAP (Online Analytical Processing) constitue un besoin central pour l'extraction d'indicateurs stratégiques. Pour répondre à cette exigence, un micro service dédié nommé Mdx Service a été implémenté en ASP.NET Core. Son rôle est d'exposer une API REST permettant d'exécuter des requêtes MDX sur un cube OLAP (typiquement SQL Server Analysis Services – SSAS) et de retourner les résultats sous un format JSON exploitable par les clients métier.

Ce service suit une architecture par couches (Clean Architecture [43]), simplifiée pour répondre à un besoin d'exécution efficace et de maintenance aisée.

3.2.7.1. Diagramme de cas d'utilisation

Le diagramme des cas d'utilisation illustré à la Figure 16 met en évidence les interactions du service MDX.

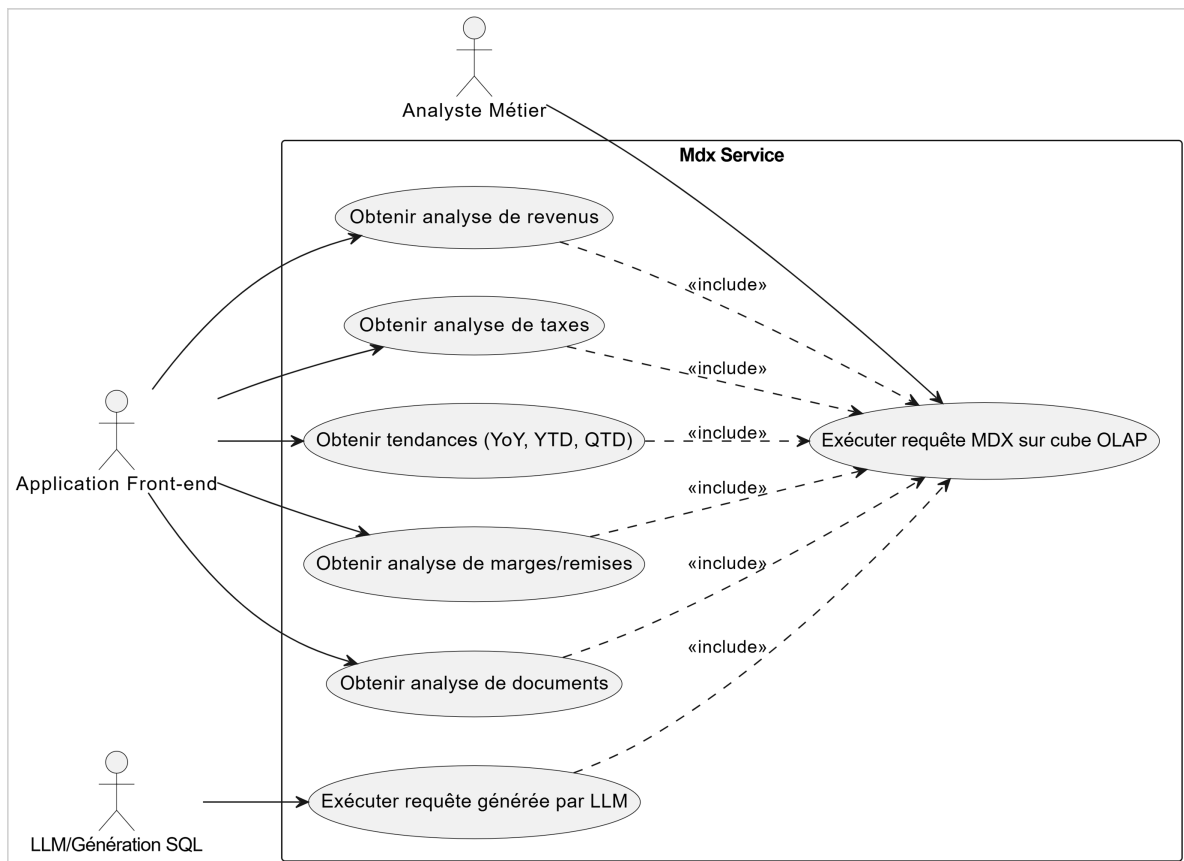


FIGURE 16. Diagramme de cas d'utilisation du Mdx Service

3.2.7.2. Diagramme de séquence

Le diagramme de séquence présenté à la Figure 17 illustre le processus d'exécution d'une requête analytique (MDX).

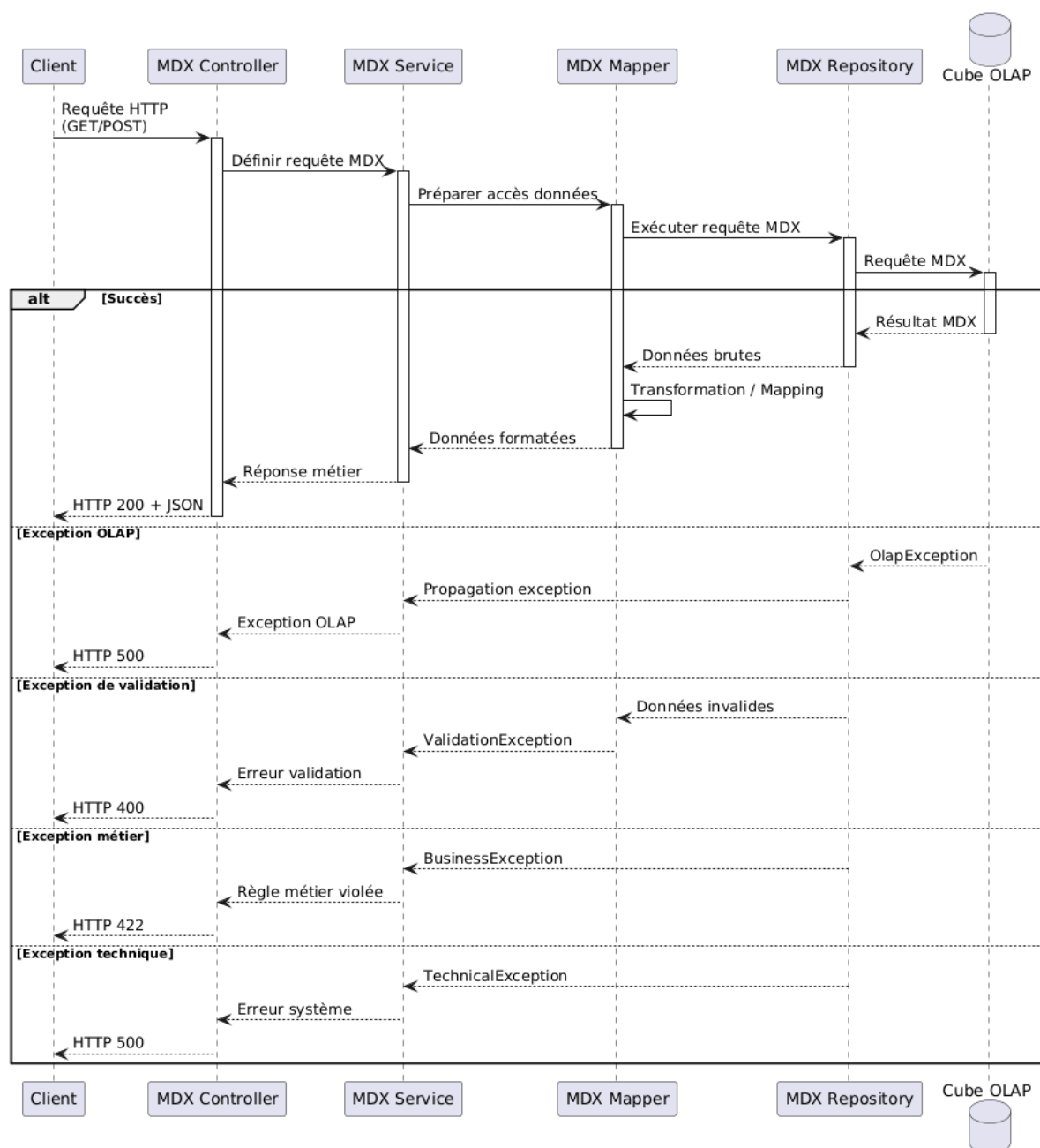


FIGURE 17. Diagramme de séquence du Mdx Service

Le Mdx Service agit comme une passerelle API entre les applications clientes et le cube OLAP. Son fonctionnement se déroule comme suit :

1. Un client (frontend, outil de reporting, module LLM) envoie une requête HTTP à l'API Gateway, qui la redirige vers l'un des contrôleurs spécialisés du Mdx Service.
2. Le contrôleur appelle le MdxService avec la requête MDX (soit directement via une méthode dédiée du DocumentCubeQueries, soit via une requête générée par le LLM).
3. MdxService transmet la requête à IMdxQuery, qui exécute la requête sur le cube SQL Server Analysis Services via ADOMD.NET ou une technologie équivalente.

4. Les résultats bruts sous forme de DataTable sont convertis par MapTableToJson() en une liste de dictionnaires (JSON dynamique).
5. Le contrôleur utilise MdxMapper pour transformer cette structure générique en DTOs fortement typés (ClientDto, PeriodDto, DocumentDto, etc.), adaptés aux besoins métier du frontend.
6. Les DTOs sont sérialisés en JSON et retournés au client pour affichage (tableau, graphique, ou intégration dans un rapport).

Cette architecture permet une séparation claire entre l'accès aux données OLAP (MDX) et la présentation des résultats, tout en offrant la flexibilité d'exposer des requêtes génériques ou adhoc issues de LLM sans refonte majeure.

3.2.8. Tests unitaires

La robustesse et la fiabilité du service MDX est assuré par une suite de tests unitaires. Ces tests couvrent les composants clés de l'architecture, en validant aussi bien les comportements nominaux que les cas d'erreur. Le tableau 8 synthétise les principaux scénarios de test et les résultats attendus.

TABLE 8. Scénarios de tests unitaires pour le Mdx Service

Composant	Scénario testé	Résultat attendu
MdxControllerBase	Exécution d'une requête MDX valide	Retour HTTP 200 avec le JSON contenant les données
MdxControllerBase	Requête MDX invalide (syntaxe erronée)	Retour HTTP 400 avec message d'erreur détaillé
MdxControllerBase	Connexion au cube OLAP impossible	Retour HTTP 503 avec indication de panne du service
ClientController	Appel de l'endpoint top clients pour une année donnée	Liste des clients triés par chiffre d'affaires, DTOs correctement typés
MdxService	Transformation d'un DataTable vide en JSON	Liste JSON vide ([]) sans erreur
IMdxQuery (mock)	Exécution d'une requête avec timeout	Levée d'une exception gérée, retour HTTP 504
MdxMapper	Mapping d'une cellule MDX null vers un DTO	Propriété DTO à null ou valeur par défaut (selon type)
ServiceCollection	Injection des dépendances	Tous les services résolus correctement par le conteneur IoC

L'exécution de ces tests est intégrée au pipeline d'intégration continue (CI) afin de détecter toute régression lors des évolutions du service.

3.2.8.1. Documentation API

Afin de faciliter l'intégration et la validation des fonctionnalités analytiques proposées par le *MDX Service*, une documentation interactive des APIs a été mise en place à l'aide de l'outil Swagger. Cette documentation permet aux développeurs ainsi qu'aux utilisateurs techniques de visualiser les endpoints disponibles, de comprendre les paramètres attendus et de tester directement les requêtes HTTP depuis une interface web centralisée.

La figure 18 présente l'ensemble des endpoints dédiés aux indicateurs de performance (KPIs)

exposés par le *MDX Service*. Ces endpoints permettent d'interroger les cubes OLAP afin de récupérer des mesures analytiques et des agrégations multidimensionnels destinées aux tableaux de bord décisionnels.

Client - GET /api/clients/above-average - GET /api/clients/outstanding-balance - GET /api/clients/high-discount-low-margin - GET /api/clients/by-margin - GET /api/clients/avg-order-value	Revenue - GET /api/revenue/by-year - GET /api/revenue/by-month/{year} - GET /api/revenue/by-quarter-doctype - GET /api/revenue/by-client - GET /api/revenue/top-clients - GET /api/revenue/bottom-clients - GET /api/revenue/concentration	Trend - GET /api/trends/yoy-revenue/{currentYear}/{priorYear} - GET /api/trends/yoy-volume/{currentYear}/{priorYear} - GET /api/trends/ytd/{year} - GET /api/trends/qty/{year} - GET /api/trends/margin/{year}
Discount - GET /api/discounts/ratio-by-year - GET /api/discounts/top-clients - GET /api/discounts/by-document-type	Tax - GET /api/tax/tva-by-month/{year} - GET /api/tax/fodec-by-doctype - GET /api/tax/burden-by-quarter	
Document GET /api/documents/volume-by-type-quarter		

FIGURE 18. Documentation Swagger des endpoints REST dédiés à l'exposition des indicateurs analytiques (KPIs) du MDX Service

Dans le cadre de l'intégration du chat-bot analytique, une API spécifique a été développée afin de recevoir les requêtes en langage naturel et de générer dynamiquement des requêtes MDX adaptées. La figure 19 illustre l'endpoint Swagger dédié à cette fonctionnalité, permettant de tester les interactions entre le chat-bot IA et le moteur OLAP.

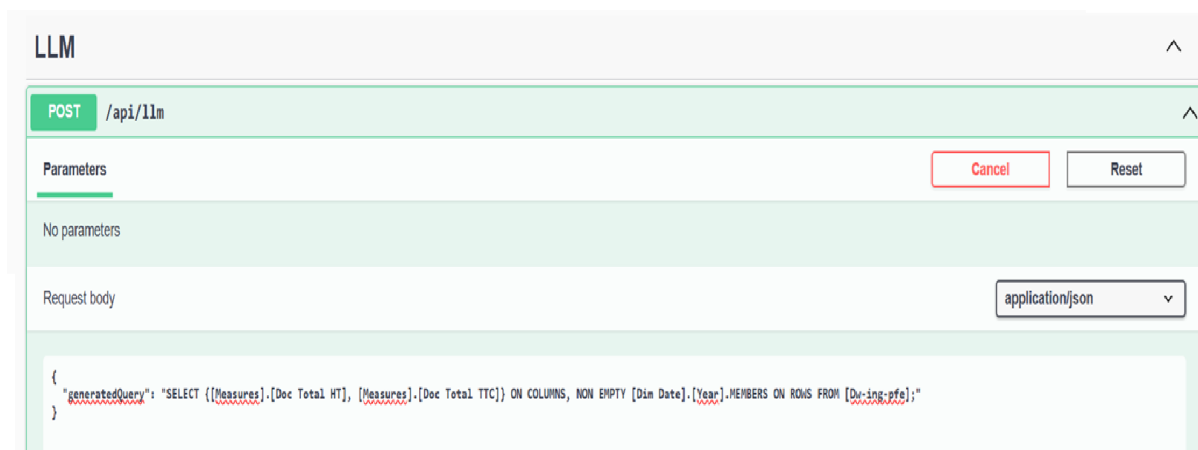


FIGURE 19. Documentation Swagger de l'endpoint d'interrogation analytique en langage naturel intégré au chat-bot IA

Après la soumission d'une requête analytique via l'interface Swagger, le système génère automatiquement une requête MDX puis exécute celle-ci sur le cube OLAP avant de retourner les résultats sous forme structurée. La figure 20 montre un exemple de test réalisé directement

depuis Swagger avec une requête utilisateur, illustrant ainsi le fonctionnement complet de la chaîne de traitement analytique.

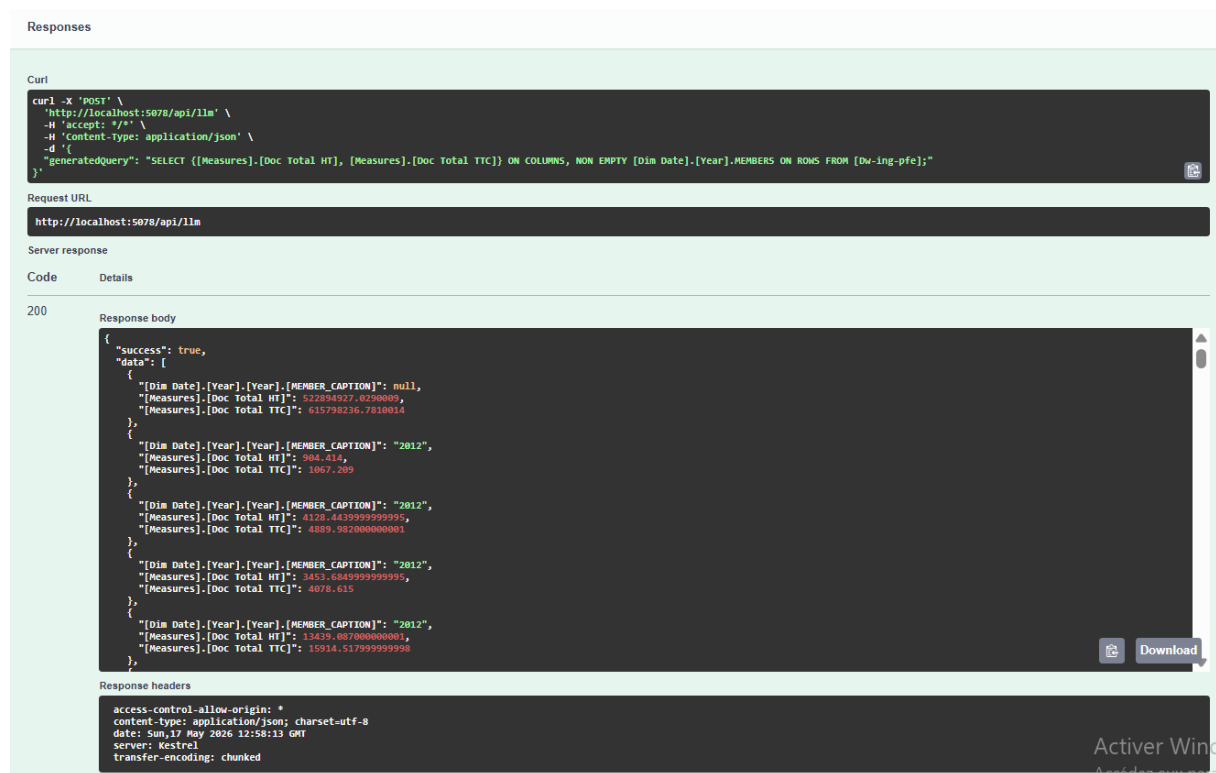


FIGURE 20. Tester l'endpoint avec une requête donner sur swagger

3.3. Sprint 2 : Mise en place de la couche Intelligente

Le deuxième sprint introduit la couche intelligente du système à travers l'intégration des composants d'intelligence artificielle et de recherche sémantique.

L'objectif principal de ce sprint est de permettre l'interrogation du système décisionnel en langage naturel via un chat-bot analytique capable de générer dynamiquement des requêtes MDX.

Ce sprint couvre principalement :

- l'intégration du modèle de langage ;
- la conception du service RAG ;
- l'exploitation des métadonnées OLAP ;
- l'indexation vectorielle avec FAISS ;
- la génération automatique de requêtes analytiques.

Cette étape marque la transition vers une plateforme décisionnelle intelligente capable d'assister les utilisateurs dans l'exploration analytique des données. Les principales fonctionnalités attendues durant ce sprint sont décrites dans le Tableau 9.

TABLE 9. *User Stories du Sprint 2*

ID	Rôle	Besoin	Bénéfice	Critère d'acceptation	Priorité	Semaine
US2.1	Utilisateur métier	Poser des questions analytiques en langage naturel	Accéder aux données sans compétence MDX	Interprétation correcte de ≥ 80 % des requêtes du corpus de test	Haute	S5
US2.2	Analyste	Générer automatiquement des requêtes MDX valides via un pipeline RAG	Simplifier l'accès aux données multidimensionnels	Validité syntaxique MDX 80 % sur 30 requêtes	Haute	S6-
US2.3	Administrateur BI	Extraire automatiquement les métadonnées OLAP	Maintenir un référentiel sémantique synchronisé	Métadonnées exportées en JSON et synchronisées automatiquement	Haute	S5
US2.4	Utilisateur final	Recevoir des réponses enrichies du contexte OLAP	Réduire les hallucinations du LLM	Pertinence sémantique ≥ 0.84 (MRR) avec FAISS k=5	Haute	S7
US2.5	Architecte logiciel	Intégrer le service RAG via FastAPI et ChromaDB	Garantir modularité et scalabilité	Embeddings rechargés automatiquement au démarrage	Moyenne	-S8
US2.6	Architecte IA	Évaluer les performances du système RAG	Valider le choix du modèle et du paramètre k	Rapport comparatif Phi-3, Mistral et LLaMA disponible	Moyenne	S8

3.3.1. Conception du service RAG

Le service RAG constitue un composant central de l'architecture intelligente du système. Il permet d'enrichir les requêtes utilisateur en s'appuyant sur des sources de connaissances externes.

Ce service s'intègre dans l'architecture micro services en interagissant avec le service LLM et le service OLAP. Le pipeline repose sur trois étapes principales : l'interprétation de la requête, la récupération du contexte et la génération de la réponse comme t-il représenté dans la figure 21. Cette approche permet d'améliorer la précision des résultats et de réduire les erreurs.

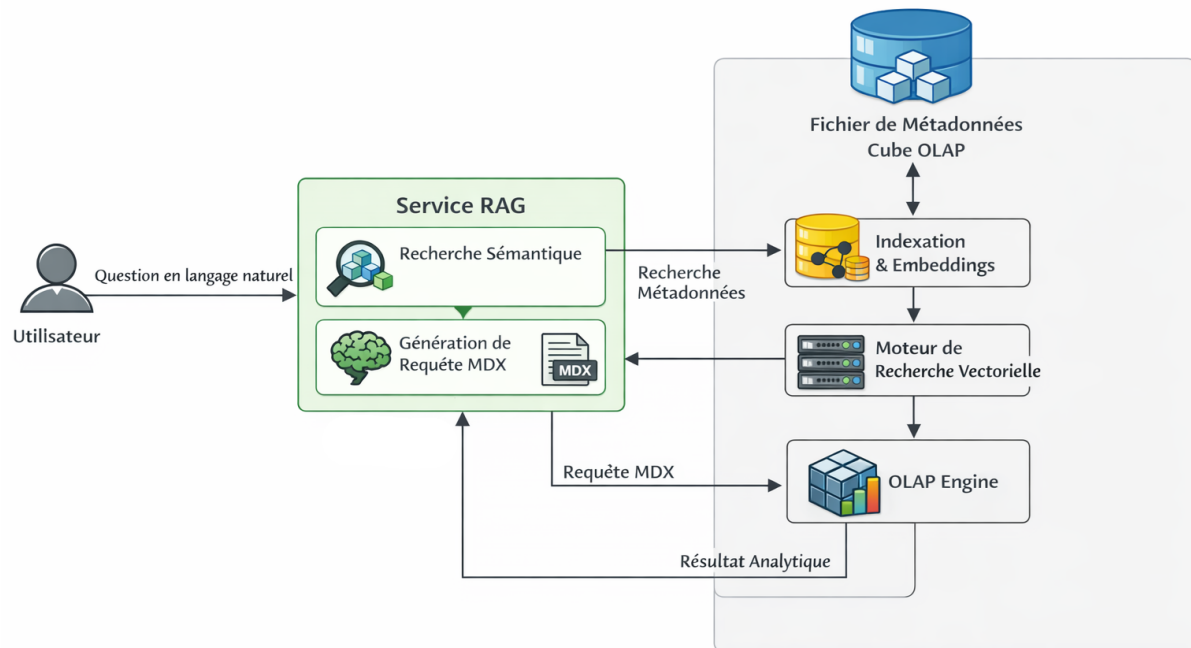


FIGURE 21. Architecture du pipeline RAG appliqué au contexte OLAP

La figure 22 présente le diagramme de séquence du service RAG. Le processus débute lorsqu'un utilisateur soumet une question via l'API REST. Après une phase de validation et de prétraitement, la question est transformée en vecteur d'embedding afin d'effectuer une recherche de similarité dans la base vectorielle. Les documents contextuels récupérés sont ensuite utilisés pour enrichir le prompt envoyé au modèle de langage, qui génère une requête MDX adaptée à la demande analytique. Cette requête est vérifiée avant son exécution par le service MDX sur le cube décisionnel. Les résultats obtenus sont finalement retournés au client sous forme de réponse analytique. Le diagramme met également en évidence la gestion des exceptions à chaque étape du traitement, garantissant la robustesse et la fiabilité du système face aux erreurs de validation, d'accès aux données, de génération ou d'exécution des requêtes.

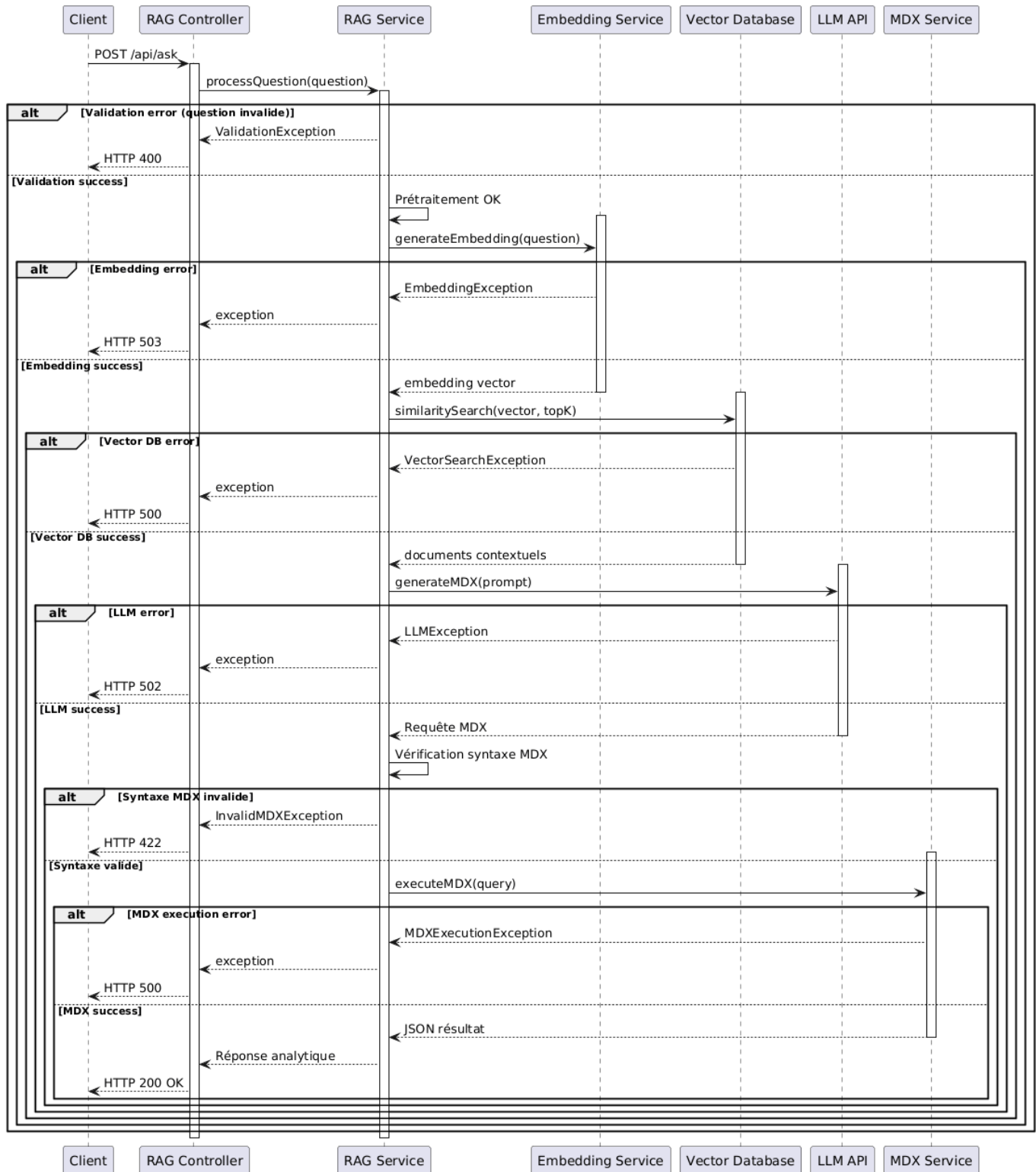


FIGURE 22. Diagramme de Séquence de Service RAG

3.3.2. Étude comparative

Les modèles d'embeddings jouent un rôle central dans les architectures de recherche sémantique et les systèmes de type RAG. Leur objectif principal est de transformer des données

textuelles en représentations vectorielles denses, permettant de mesurer la similarité sémantique entre requêtes et documents.

Dans le cadre de ce projet, plusieurs familles de modèles d'embeddings ont été étudiées afin d'identifier la solution la plus adaptée aux contraintes de performance, de précision et de scalabilité.

3.3.2.1. Comparaison des modèles d'embeddings

TABLE 10. Comparaison des familles de modèles d'embeddings selon leur adéquation aux systèmes RAG

Modèle	Type	Contexte	Performance sémantique	Adéquation RAG
Word2Vec / GloVe	Statique	Absent	Faible	Insuffisante
FastText	Hybride	Sous-mots	Limitée	Partielle
BERT	Encoder-Only	Bidirectionnel	Élevée	Excellente
MiniLM (all-MiniLM-L6-v2)	Encoder-Only	Bidirectionnel	Élevée	Excellente
RoBERTa	Encoder-Only	Bidirectionnel	Très élevée	Excellente

Les modèles statiques présentent une limitation fondamentale : leur représentation est indépendante du contexte, rendant impossible la distinction sémantique entre "*chiffre d'affaires par mois*" et "*évolution mensuelle du CA*". Les modèles basés sur l'architecture Transformer, en particulier la variante all-MiniLM-L6-v2, offrent un embedding contextuel bidirectionnel adapté à l'indexation FAISS et à la recherche sémantique dans un espace de métadonnées OLAP.

3.3.2.2. Comparaison des modèles LLM

Après avoir retenu l'approche basée sur l'intégration d'un modèle open source avec le serveur Ollama, comme il est indiqué à la Section 1.3.4, une seconde étude comparative, illustrée dans le tableau 11, a été réalisée afin d'identifier le modèle le plus adapté aux contraintes techniques du projet.

Les principaux critères d'évaluation retenus sont :

- les performances de raisonnement ;
- la consommation mémoire ;
- la taille du modèle ;
- la capacité d'exécution sur une infrastructure limitée ;
- le temps de réponse ;
- la facilité d'intégration dans une architecture micro services ;

— l'adaptation aux tâches NL2SQL/NL2MDX et RAG [14].

TABLE II. Comparaison des modèles de langage étudiés

Modèle	Taille	MMLU (%)	HumanEval (%)	Empreinte RAM	Licence	Source
Phi-3 Mini	3,8 B	69,9	60,9	~ 2,5 Go (4-bit)	MIT	Microsoft (2024) [7]
Llama 3.2	8 B	66,0	62,5	~ 5 Go (4-bit)	Llama 3 Community	Meta AI (2024) [8]
Mistral 7B v0.3	7 B	64,2	40,2	~ 4,5 Go (4-bit)	Apache 2.0	Mistral AI (2024) [9]
Gemma 7B	7 B	64,3	32,3	~ 5 Go (4-bit)	Gemma ToU	Google (2024) [44]
DeepSeek Coder 6.7B	6,7 B	49,1	73,8	~ 4 Go (4-bit)	DeepSeek License	DeepSeek (2024) [45]

Phi-3 Mini obtient le meilleur score MMLU de sa catégorie sub-4B (69,9 %), surpassant des modèles deux fois plus grands. Sa licence MIT autorise un déploiement commercial sans restriction. Son empreinte mémoire réduite (2,5 Go en quantisation 4-bit) est déterminante dans le contexte d'un VPS aux ressources contraintes (absence de GPU dédié). Pour la tâche NL2MDX, la compréhension du schéma analytique enrichi par le pipeline RAG prime sur la capacité de génération de code pure mesurée par HumanEval, ce qui rend le score MMLU plus représentatif du cas d'usage.

3.3.3. Exploitation des métadonnées OLAP

Les métadonnées du cube OLAP constituent une source de connaissance structurée essentielle au fonctionnement du système de type RAG. Elles décrivent de manière formelle les dimensions analytiques, les mesures métiers ainsi que les hiérarchies permettant la navigation multidimensionnel au sein du cube.

Dans le cadre de ce travail, ces métadonnées sont extraites de manière automatisée et périodique à chaque processus de ré-alimentation du Data Warehouse. Cette extraction repose sur l'exécution de requêtes MDX adressées au schéma système du serveur OLAP, permettant d'interroger directement la structure logique du cube sans dépendre de la couche applicative.

Plus précisément, les requêtes suivantes sont utilisées afin de récupérer les différents éléments constitutifs du modèle multidimensionnel :

```

1
2 SELECT *
3 FROM $SYSTEM.MDSHEMA\_MEASURES
4
5 SELECT *
6 FROM $SYSTEM.MDSHEMA\_DIMENSIONS
7
8 SELECT *
9 FROM $SYSTEM.MDSHEMA\_HIERARCHIES
10

```

```

11 SELECT *
12 FROM $SYSTEM.MDSHEMA\_LEVELS

```

Code 6 – Extraction des meta données : mesures

Cette approche garantit la synchronisation entre la structure physique du Data Warehouse et les composants intelligents du système, en particulier le module RAG chargé de l'interprétation sémantique des requêtes utilisateur.

L'objectif principal de ce mécanisme est de maintenir un référentiel de métadonnées cohérent, actualisé et exploitable par les services analytiques. Ce référentiel permet notamment d'enrichir le contexte sémantique des requêtes formulées en langage naturel et d'améliorer la génération automatique de requêtes analytiques (MDX).

Les métadonnées extraites sont ensuite transformées en une représentation structurée (format JSON), facilitant leur intégration dans un moteur de recherche sémantique et leur exploitation par les modèles d'intelligence artificielle.

```

1 {
2   "cubeName": "CUBE-DOCUMENT",
3   "measures": [
4     { "name": "Dim Date", "mdxExpression": "[Measures].[Dim Date]" },
5     { "..."}
6   ],
7   "dimensions": [{
8     "name": "[Dim Date]",
9     "hierarchies": [
10      { "name": "Year", "mdxExpression": "[Dim Date].[Year]" },
11      { "..."}
12    ]
13  },
14  {
15    "name": "[Document Dim]",
16    "hierarchies": [
17      { "name": "Document", "mdxExpression": "[Document Dim].[Document]" },
18      { "..."}
19    ]
20  },
21  {
22    "name": "[Tiers Dim]",
23    "hierarchies": [
24      { "name": "Tiers", "mdxExpression": "[Tiers Dim].[Tiers]" },
25      { "..."}
26    ]
27  }
28 ],
29 "sampleQueries": [{
30   "question": "chiffre d'affaire par mois en 2024",
31   "mdx": "SELECT [Measures].[Doc Total HT] ON COLUMNS ..."
32 },

```

```

33     {"..."}
34 ]
35 }

```

Code 7 – Représentation des métadonnées du cube OLAP

3.3.4. Vectorisation et indexation sémantique

Dans le cadre de la mise en place du module de recherche intelligente, les métadonnées du cube OLAP sont transformées en représentations vectorielles à l'aide d'un modèle d'embeddings basé sur *SentenceTransformers*. Cette étape permet de projeter les éléments structurés (mesures, dimensions et descriptions métier) dans un espace sémantique continu, facilitant leur comparaison par similarité.

```

1 from sentence_transformers import SentenceTransformer
2 import faiss
3 import numpy as np
4
5 class RAGServiceOLAP:
6     def __init__(self, model_name="all-MiniLM-L6-v2"):
7         self.model = SentenceTransformer(model_name)
8         self.index = None
9         self.documents = []
10        self.metadata_store = []

```

Code 8 – Initialisation du service RAG avec embeddings

L'utilisation de FAISS (*Facebook AI Similarity Search*) permet d'indexer efficacement les embeddings en mémoire et d'assurer une recherche rapide même sur de grands volumes de métadonnées OLAP.

```

1 embeddings = self.model.encode(texts)
2 dim = embeddings.shape[1]
3
4 if self.index is None:
5     self.index = faiss.IndexFlatL2(dim)
6
7 self.index.add(np.array(embeddings).astype("float32"))

```

Code 9 – Indexation des embeddings dans FAISS

Si FAISS assure la recherche vectorielle en mémoire avec une latence minimale, il ne dispose d'aucune capacité de persistance native : l'index est perdu à chaque redémarrage du service. Afin de garantir la durabilité des embeddings entre les sessions et d'éviter une réindexation systématique des métadonnées OLAP à chaque démarrage, une base de données vectorielle **ChromaDB** est utilisée comme couche de stockage persistant.

ChromaDB joue ainsi un rôle complémentaire à FAISS dans l'architecture : les embeddings sont calculés une seule fois, stockés de manière durable dans ChromaDB avec leurs métadonnées associées (nom de la mesure, dimension, description métier, expression MDX), puis rechargés en mémoire dans l'index FAISS au démarrage du service. Cette séparation entre stockage persistant et recherche en mémoire permet de bénéficier à la fois de la fiabilité de ChromaDB et des performances de recherche de FAISS.

Le code suivant illustre la logique de persistance : lors du premier chargement, les documents sont encodés et insérés dans ChromaDB ; lors des démarrages ultérieurs, les embeddings sont directement récupérés depuis la collection existante, sans réencodage.

```

1 import chromadb
2 from chromadb.config import Settings
3 class RAGServiceOLAP:
4     def __init__(self, model_name="all-MiniLM-L6-v2",
5                  chroma_path="./chroma_store"):
6         self.model = SentenceTransformer(model_name)
7         self.index = None
8         self.documents = []
9         self.metadata_store = []
10        self.chroma_client = chromadb.Client(
11            Settings(chroma_db_impl="duckdb+parquet",
12                    persist_directory=chroma_path,
13                    anonymized_telemetry=False))
14        self.collection = self.chroma_client.get_or_create_collection(
15            name="olap_metadata",
16            metadata={"hnsw:space": "12"})
17        def index_documents(self, documents: list[dict]):
18            texts = [doc["text"] for doc in documents]
19            ids = [doc["id"] for doc in documents]
20            metas = [doc.get("metadata", {}) for doc in documents]
21            embeddings = self.model.encode(texts, show_progress_bar=False)
22            embeddings_f32 = np.array(embeddings).astype("float32")
23            self.collection.add(
24                ids = ids,
25                embeddings = embeddings_f32.tolist(),
26                documents = texts,
27                metadatas = metas)
28            self.chroma_client.persist()
29            dim = embeddings_f32.shape[1]
30            if self.index is None:
31                self.index = faiss.IndexFlatL2(dim)
32            self.index.add(embeddings_f32)
33            self.documents.extend(texts)
34            self.metadata_store.extend(metas)
35        def load_from_chroma(self):
36            result = self.collection.get(include=["embeddings",
37                                                "documents",
38                                                "metadatas"])

```

```

39     if not result["ids"]:
40         return
41     embeddings_f32 = np.array(result["embeddings"]).astype("float32")
42     dim = embeddings_f32.shape[1]
43     self.index = faiss.IndexFlatL2(dim)
44     self.index.add(embeddings_f32)
45     self.documents = result["documents"]
46     self.metadata_store = result["metadatas"]

```

Code 10 – Persistance et récupération des embeddings dans ChromaDB

Cette approche permet de supporter des requêtes sémantiques complexes telles que « *Chiffre d'affaires par région et par mois* », sans dépendre d'une correspondance exacte avec les noms techniques des dimensions ou mesures du cube. La combinaison FAISS–ChromaDB assure ainsi à la fois la rapidité de la recherche vectorielle en régime nominal et la résilience du système face aux cycles de vie du micro service dans l'architecture de déploiement Docker.

3.3.5. Enrichissement du contexte et Prompt Engineering

Cette étape constitue la continuité directe du processus d'indexation présenté précédemment. Elle exploite les indexes vectoriels afin d'effectuer une récupération sémantique des éléments les plus pertinents en fonction de la requête utilisateur.

3.3.5.1. Récupération sémantique du contexte

Lorsqu'une requête utilisateur est exprimée en langage naturel, celle-ci est encodée dans le même espace vectoriel que les documents indexés. Une recherche de similarité est ensuite effectuée afin d'extraire les fragments de contexte les plus proches sémantiquement.

```

1 query_embedding = self.model.encode([user_query])
2 D, I = self.index.search(np.array(query_embedding).astype("float32"), k=5)
3
4 context_docs = [self.metadata_store[i] for i in I[0]]

```

Code 11 – Recherche sémantique dans FAISS

Cette étape permet de filtrer dynamiquement le contexte pertinent issu du cube OLAP (mesures, dimensions et métadonnées métier), réduisant ainsi le bruit informationnel.

3.3.5.2. Construction du prompt enrichi

Les documents récupérés sont ensuite agrégés afin de constituer un contexte structuré injecté dans un prompt destiné au modèle de langage. Cette étape de *prompt engineering* vise à contraindre la génération dans un cadre OLAP précis.

```

1 def build_rag_prompt(user_query, context_docs):

```

```

2     context = "\n\n".join([doc["text"] for doc in context_docs])
3
4     return f"""
5 You are an OLAP expert working with a multidimensional cube.
6
7 Here is the relevant cube metadata:
8 {context}
9
10 User question:
11 {user_query}
12
13 Task:
14 Generate a valid MDX query that answers the question from the context.
15
16 Rules:
17 - ONLY return MDX
18 - No explanation
19 - No comments
20 - No markdown
21 - No text before or after
22
23 OUTPUT:
24 <<MDX Query>>
25 """

```

Code 12 – Construction du prompt enrichi pour RAG

Ce mécanisme permet de réduire significativement les hallucinations du modèle en lui fournissant un contexte métier explicite et structuré.

3.3.5.3. Génération de requêtes MDX via LLM

Le prompt enrichi est ensuite transmis à un LLM via une API externe du serveur Ollama. Le modèle est contraint de générer uniquement une requête MDX exploitable par le moteur OLAP.

```

1 response = requests.post(
2     self._url,
3     json={
4         "model": self._model,
5         "messages": [
6             {
7                 "role": "system",
8                 "content": "You are an OLAP expert. Output ONLY valid MDX queries."
9             },
10            {
11                "role": "user",
12                "content": prompt
13            }
14        ]
15    }
16 )

```

Code 13 – Appel du LLM pour génération MDX

3.3.6. Évaluation des performances du système intelligent

L'évaluation d'un système RAG appliqué à la génération de requêtes analytiques (NL2MDX) requiert une approche multidimensionnel, distinguant la qualité sémantique des réponses produites, l'influence des hyperparamètres de recherche sur la pertinence du contexte récupéré, et les performances temporelles des modèles de génération. Ces trois axes sont traités successivement dans cette section.

3.3.6.1. Constitution et annotation du corpus d'évaluation

Afin de disposer d'une base d'évaluation rigoureuse et ancrée dans le schéma réel du cube CUBE-DOCUMENT, un corpus de 30 requêtes tests a été constitué manuellement, couvrant l'ensemble des domaines analytiques exposés par la plateforme. Le corpus est disponible dans GitHub¹

Les 30 requêtes sont réparties en six domaines fonctionnels de cinq requêtes chacun : analyse du chiffre d'affaires, segmentation clients, fiscalité (TVA, FODEC, timbre fiscal), documents commerciaux, remises et marges, et analyses temporelles avancées (YoY, YTD, TOPN). Cette répartition équilibrée garantit une couverture représentative de l'espace analytique de l'entreprise et évite toute surreprésentation d'un domaine particulier dans l'évaluation globale.

Chaque entrée du corpus associe à une requête en langage naturel une requête MDX de référence (*gold standard*), construite à partir des expressions exactes du cube : mesures ([Measures] . [DocTotal], etc.)), dimensions ([Dim Date] . [Year], [Tiers Dim] . [Tiers], etc) et fonctions MDX avancées (TOPN, FILTER, ParallelPeriod, etc). La complexité des requêtes est intentionnellement graduée : les premières entrées de chaque domaine mobilisent une dimension et une mesure uniques, tandis que les dernières requièrent des calculs multi-mesures, des filtres conditionnels ou des analyses inter-périodes.

3.3.6.2. Métriques d'évaluation des réponses

L'évaluation de la qualité des réponses repose sur cinq critères complémentaires, adaptés aux contraintes spécifiques d'un système NL2MDX :

1. <https://github.com/ZoubairMabrouk/microservices/tree/main/rag-service/app/data>

TABLE 12. Métriques d'évaluation du système RAG NL2MDX

Métrique	Définition et méthode de mesure
Exactitude syntaxique	Proportion de requêtes MDX générées syntaxiquement valides, mesurée par leur exécution sur SQL Server Analysis Services et la vérification de l'absence d'erreurs de compilation.
Pertinence sémantique	Similarité entre la requête MDX générée et une requête de référence attendue. Cette métrique est évaluée à l'aide de scores automatiques tels que BLEU, ROUGE-L et BERTScore afin de mesurer la proximité lexicale et sémantique des requêtes.
Cohérence contextuelle	Capacité du système à utiliser correctement les dimensions, mesures, hiérarchies et membres définis dans les métadonnées OLAP indexées dans la base vectorielle FAISS.
Taux d'hallucination	Proportion de requêtes contenant des références à des dimensions, mesures ou membres inexistantes dans le cube multidimensionnel.
Satisfaction fonctionnelle	Pourcentage de requêtes produisant le résultat analytique attendu, validé par comparaison avec les résultats de référence définis par les experts métier.

L'évaluation a été conduite sur un corpus de 30 requêtes couvrant les principaux cas d'usage analytiques : analyse du chiffre d'affaires par période, segmentation client, comparaisons temporelles (YoY, YTD) et analyses multi-dimensionnelles croisées.

3.3.6.3. Influence du paramètre k sur la qualité du retrieval

Le paramètre k désigne le nombre de documents extraits de l'index FAISS lors de la phase de récupération sémantique. Il constitue l'hyperparamètre le plus déterminant du pipeline RAG : une valeur trop faible prive le modèle génératif d'un contexte suffisant, tandis qu'une valeur trop élevée introduit des fragments non pertinents susceptibles de dégrader la précision de la génération.

Une évaluation empirique a été conduite en faisant varier $k \in \{1, 3, 5, 7, 10\}$ sur le corpus de 30 requêtes tests, avec le modèle Phi-3 Mini comme générateur. Pour chaque valeur de k , trois indicateurs ont été mesurés : (1) le score de pertinence sémantique moyen (validation manuelle), (2) le taux de requêtes MDX valides, (3) le temps de réponse de bout en bout.

Les résultats, présentés dans la Figure 23, révèlent un optimum situé entre $k = 3$ et $k = 5$ selon le critère considéré.

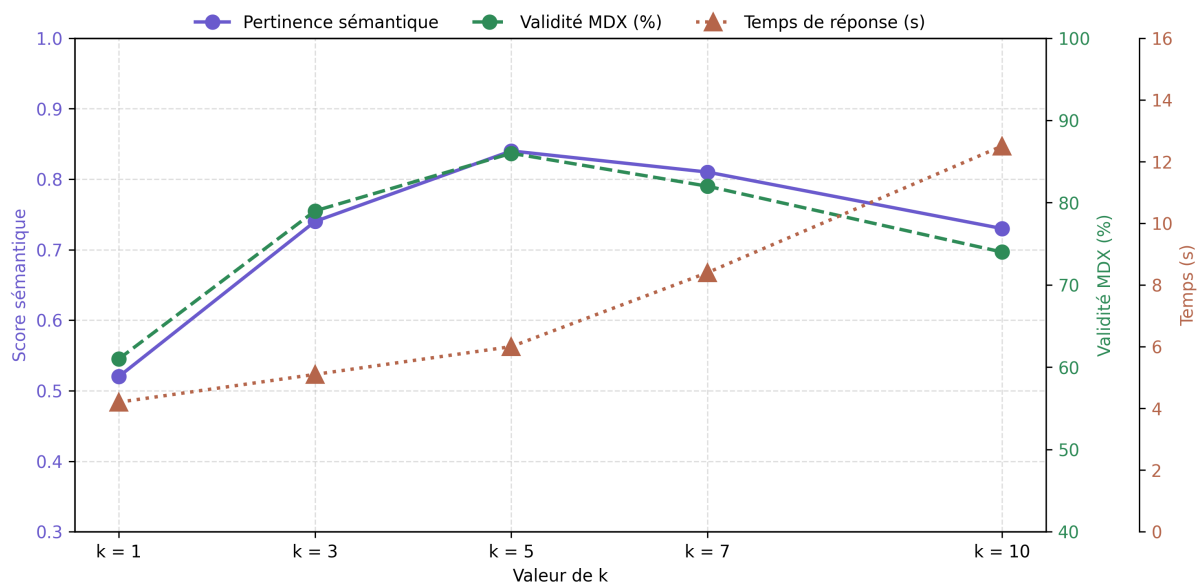


FIGURE 23. Influence du paramètre k sur la pertinence sémantique, le taux de validité MDX et le temps de réponse – courbes obtenues sur le corpus de 30 requêtes tests avec Phi-3 Mini

Pour $k < 3$, le contexte fourni au modèle est insuffisant : les métadonnées OLAP les plus pertinentes ne sont pas systématiquement récupérées, conduisant à un taux d'hallucination élevé. Pour $k > 5$, le contexte devient trop volumineux : des fragments sémantiquement éloignés de la requête sont injectés dans le prompt, augmentant la probabilité de confusion entre dimensions homonymes et allongeant significativement le temps d'inférence. La valeur $k = 5$ a été retenue comme compromis optimal, offrant un score de pertinence sémantique de 0,84 et un taux de validité syntaxique MDX de 82 %, pour un temps de réponse de 6,1 s avec Phi-3 Mini.

3.3.6.4. Évaluation comparative des modèles LLM

Une étude comparative a été effectuée sur trois modèles open-source intégrés dans l'architecture (**Phi-3 Mini**, **Mistral** et **LLaMA 3.2 8B**), avec $k = 5$ fixé pour tous les essais. Le temps de réponse est défini comme le délai entre la soumission du prompt enrichi et la réception de la requête MDX complète, mesuré dans l'environnement de déploiement (VPS à ressources mutualisées, sans GPU dédié).

TABLE 13. Comparaison des modèles LLM sur les critères de temps de réponse, validité syntaxique MDX et pertinence sémantique, pour $k = 5$

Modèle	Temps moyen (s)	Validité MDX (%)	Pertinence sémantique	Adéquation infrastructure
Phi-3 Mini (3.8B)	6.1	82 %	Bonne	Très élevée – CPU seul
Mistral	10.2	91 %	Très bonne	Moyenne – 8 Go RAM min.
LLaMA 3.2 8B	13,5	94 %	Excellente	Faible – GPU recommandé

3.3.6.5. Analyse des résultats et synthèse

Les résultats mettent en évidence un compromis classique entre qualité de génération et efficacité temporelle. LLaMA 3.2 8B produit les requêtes MDX les plus précises (94 % de validité syntaxique, pertinence sémantique qualifiée d'excellente), mais au prix d'une latence de 13,4 s incompatible avec une interaction conversationnelle fluide. Phi-3 Mini offre à l'inverse la meilleure réactivité (6,1 s), avec une qualité satisfaisante pour la majorité des cas d'usage analytiques courants. Mistral se positionne comme un compromis intermédiaire pertinent lorsqu'une infrastructure légèrement plus robuste est disponible.

Dans le contexte de déploiement contraint du projet (VPS à ressources mutualisées, sans GPU dédié), **Phi-3 Mini avec $k = 5$** constitue la configuration retenue, offrant un équilibre viable entre qualité analytique et contraintes matérielles. Une migration vers Mistral est envisageable dans une phase ultérieure si l'infrastructure est renforcée.

3.4. Conclusion

Ce chapitre a présenté les deux premières releases fonctionnelles de la plateforme décisionnelle intelligente, couvrant l'intégralité de la chaîne allant de l'intégration des données brutes à l'interrogation en langage naturel.

Le Sprint 1 a établi la fondation analytique du système : un Data Warehouse en schéma en étoile alimenté par des flux SSIS, un cube OLAP multidimensionnel déployé sous SSAS, et un micro service MDX exposant les indicateurs via API REST. Ce socle garantit la séparation des traitements transactionnels et analytiques, la qualité des données par des opérations de nettoyage formalisées, et des performances d'interrogation compatibles avec les besoins temps réel d'un tableau de bord décisionnel.

Le Sprint 2 a introduit la couche intelligente via un pipeline RAG complet : vectorisation des métadonnées OLAP avec `all-MiniLM-L6-v2`, indexation sémantique dans FAISS, enrichissement du contexte par prompt engineering, et génération de requêtes MDX via le modèle Phi-3 Mini. Ce pipeline permet à un utilisateur non technique de formuler des questions analytiques en langage naturel et d'obtenir une réponse exploitable, sans écrire une seule ligne de MDX.

Deux limites méritent d'être signalées pour les sprints suivants. D'une part, le modèle Phi-3 Mini, choisi pour ses contraintes légères, peut produire des requêtes MDX syntaxiquement valides mais sémantiquement incorrectes dans des cas de jointures multidimensionnels complexes ; l'ajout d'une étape de validation syntaxique et d'un mécanisme de retry constitue une amélioration prioritaire. D'autre part, le système RAG actuel ne conserve aucun historique de session : chaque requête est traitée de manière stateless, sans mémoire conversationnelle. L'intégration d'un *Conversation Service* — décrit au Sprint 3 de la Release 2 — répond directement à cette limitation.

L'ensemble de ces composants est conçu de manière modulaire et stateless, facilitant leur intégration dans l'architecture micro services de la Release 2 et leur évolution indépendante dans les sprints ultérieurs.

Chapitre 4

AMÉLIORATION ET DÉPLOIEMENT – RELEASE 2

4.1. Introduction

La première release a permis de poser les fondations analytiques de la plateforme : entrepôt de données, modélisation multidimensionnel via cubes OLAP, et pipeline RAG (*Retrieval-Augmented Generation*) intégrant un modèle de langage pour l'interrogation en langage naturel. Ces composants, bien que fonctionnels, demeuraient intégrés dans un prototype monolithique — déployé manuellement, difficilement reproductible, et peu adapté à une montée en charge réelle.

Ce deuxième release engage une rupture architecturale délibérée : le prototype est transformé en une plateforme distribuée, structurée autour de quatre micro services indépendants, orchestrés par une API Gateway, conteneurisés via Docker, et déployés automatiquement via une chaîne CI/CD GitOps. L'objectif n'est pas d'ajouter des fonctionnalités, mais de garantir que celles déjà développées soient fiables, maintenables et opérationnelles en environnement de production.

Le Sprint 3 couvre la conception et l'implémentation des services backend — authentification, conversation historisé, et gateway de routage — ainsi que le développement d'un frontend unifiant ces services. Le Sprint 4 industrialise ce socle : pipeline CI/CD avec GitHub Actions, conteneurisation, gestion GitOps des versions, rollback automatique, et supervision opérationnelle.

4.2. Sprint 3 : Implémentation des services backend complémentaire

Le Sprint 3 est consacré à la mise en place de l'architecture micro services ainsi qu'à l'intégration des services métiers. L'objectif principal de ce sprint est de transformer les éléments d'analyse et d'intelligence développés lors des sprints précédents en une plateforme distribuée opérationnelle, accessible via des services REST sécurisés et interconnectés. Le Tableau 14 liste les *User Story* prévues pour ce sprint.

TABLE 14. *User Stories du Sprint 3*

ID	Rôle	Besoin	Bénéfice	Critère d'acceptation	Priorité	Semaine
US3.1	Utilisateur	Accéder aux services via une API Gateway YARP	Simplifier les échanges client-serveur	Routage correct vers les 4 micro services	Haute	S11
US3.2	Administrateur	Sécuriser les APIs avec JWT et gestion des rôles	Protéger les données analytiques	Endpoints protégés et authentification fonctionnelle	Haute	S9
US3.3	Utilisateur métier	Interroger le cube OLAP via chat-bot analytique	Obtenir des analyses sans MDX	Génération et exécution correcte des requêtes MDX	Haute	S10
US3.4	Architecte logiciel	Implémenter un Conversation Service avec RabbitMQ	Garantir résilience et scalabilité	DLQ opérationnelle ; ACK/NACK tracés	Moyenne	S10
US3.5	Utilisateur final	Disposer d'une interface React moderne	Faciliter l'utilisation de la plateforme	Dashboard et chat-bot opérationnels	Haute	S11–S12
US3.6	Administrateur système	Centraliser les logs et métriques	Faciliter la supervision	Serilog et endpoints HealthCheck actifs	Moyenne	S12

4.2.1. Mise en Place de Conversation Service

La traçabilité des opérations ainsi que la supervision des échanges inter-services représentent des éléments essentiels dans cet architecture distribuée pour garantir la fiabilité, la sécurité et la maintenabilité du système. Pour cette situation, la plateforme mise en place un micro service dédié appelé **Conversation Service**, dont le rôle principal est de centraliser la collecte, la validation et la persistance des événements métiers et techniques générés par les différents composants de la plateforme.

L'objectif principal de ce service est de fournir une solution d'audit découplée, asynchrone et hautement scalable permettant d'assurer la journalisation des requêtes utilisateurs, le suivi des communications entre micro services, la surveillance des traitements analytiques et IA ainsi que la centralisation des erreurs techniques et fonctionnelles. Cette approche améliore considérablement l'observabilité globale du système et facilite les opérations de monitoring et de debugging. !

4.2.1.1. Diagramme de séquence

Le traitement de l'événement suit un pipeline en couches successives et indépendantes, illustré par le diagramme de séquence de la figure 24

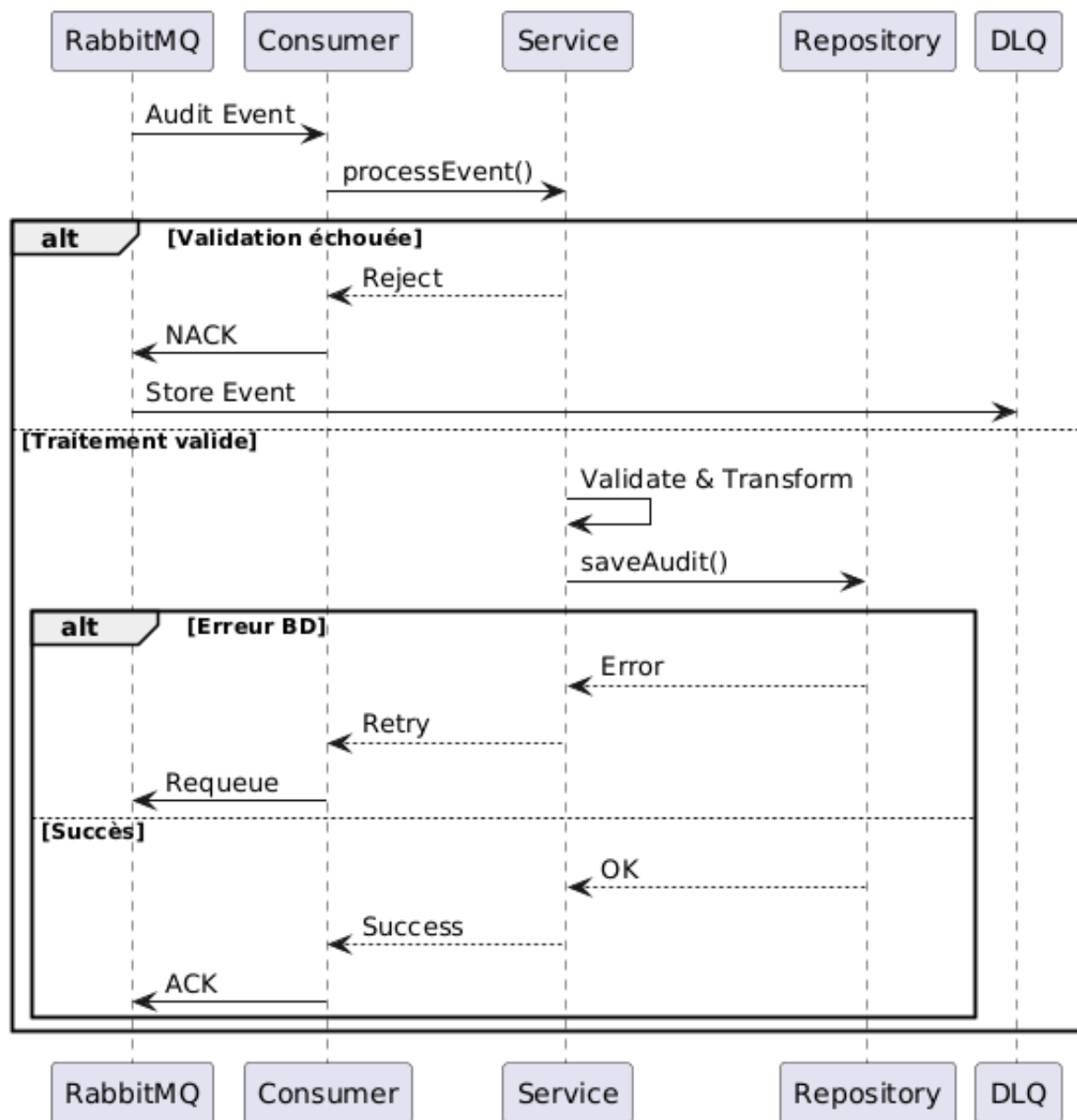


FIGURE 24. Diagramme de séquence du Conversation Service

4.2.1.2. Communication Asynchrone avec RabbitMQ

La figure 25 illustre l'architecture asynchrone du service d'audit reposant sur RabbitMQ, un broker de messages basé sur le protocole AMQP, dont le rôle principal est d'assurer un échange fiable et découplé des événements entre les micro services producteurs et le service consommateur. Tous les services publient leurs événements vers un Topic Exchange, qui se charge de les router vers la Conversation Queue en fonction des clés de routage définies. Le Conversation Service consomme ensuite ces messages, les traite et les persiste dans la base de données dédiée aux conversations. Ce modèle garantit un fort découplage entre producteurs et consommateur, améliorant ainsi la scalabilité et la résilience du système. En cas de succès, un ACK est envoyé pour confirmer le traitement et supprimer le message de la file, tandis qu'en

cas d'échec critique un NACK déclenche son redirection vers la Dead Letter Queue (DLQ), permettant de centraliser les messages non traités pour analyse ou réexécution ultérieure. Ainsi, cette architecture met en évidence un flux de traitement robuste intégrant routage intelligent, gestion des erreurs et persistance fiable des événements.

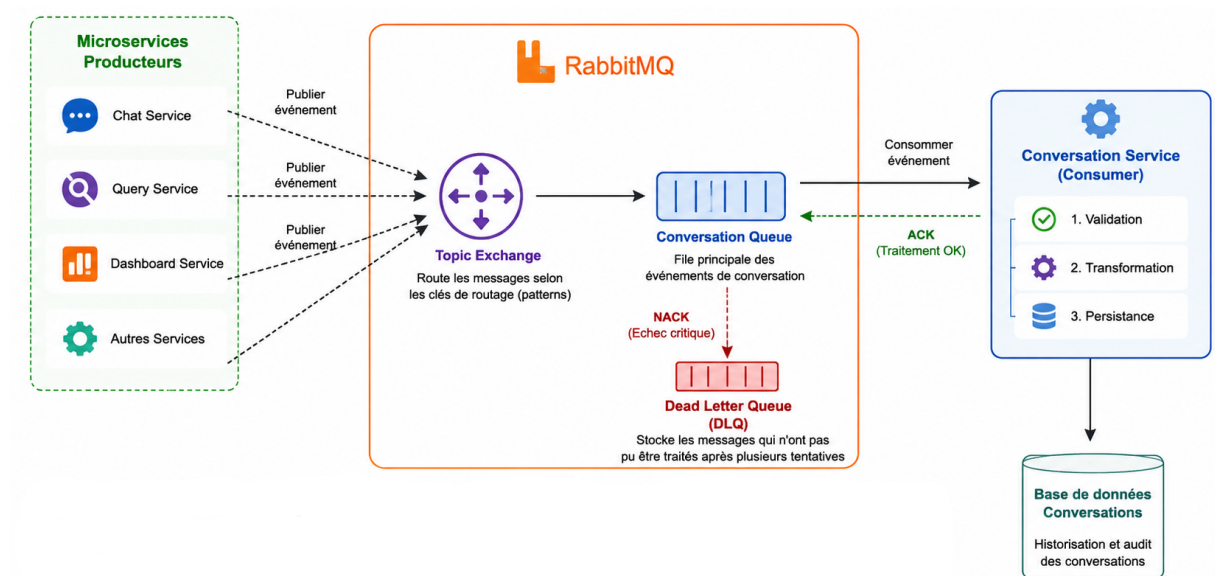


FIGURE 25. Architecture de communication asynchrone

4.2.1.3. Tests Unitaires

Pour Assurer la fiabilité et la qualité du *Conversation Service*, une série de tests unitaires a été réalisée sur les principaux composants métier. L'objectif de ces tests est d'évaluer le fonctionnement. Il est attendu de chaque module de manière individuelle, en particulier les mécanismes de validation, transforme, maintient et gère les messages RabbitMQ.

Le tableau 15 présente les principaux scénarios testés ainsi que les résultats attendus.

TABLE 15. Tests unitaires du Conversation Service

Composant	Scénario testé	Résultat attendu
Validation Layer	Réception d'un message contenant tous les champs obligatoires	Validation réussie et poursuite du traitement.
Transformation Layer	Conversion d'un DTO valide vers l'entité <code>ConversationEntity</code>	Mapping effectué sans perte de données.
Conversation Repository	Insertion d'un événement valide dans la base de données	Création correcte de l'enregistrement et retour de succès.
RabbitMQ Consumer	Consommation d'un message valide depuis la file RabbitMQ	Traitement complet du message et émission d'un ACK.
Dead Letter Queue	Réception d'un message rejeté	Stockage correct du message dans la file DLQ.
Logging Service	Traitement réussi d'un événement	Création d'une trace de journalisation correspondante.
Exception Handler	Survenue d'une exception non contrôlée	Capture de l'erreur et génération d'un log d'alerte.
API Controller	Consultation de l'historique des conversations	Retour d'une réponse HTTP 200 contenant les données demandées.

L'ensemble des tests unitaires a été exécuté à l'aide du framework *xUnit* et de la bibliothèque *Moq* afin de simuler les dépendances externes telles que RabbitMQ et la base de données. Les résultats obtenus ont confirmé le bon fonctionnement des différents composants du *Conversation Service* ainsi que la robustesse des mécanismes de validation, de persistance et de gestion des erreurs.

4.2.1.4. Documentation Swagger

Les fonctionnalités de ce service sont exposées sous format *REST* comme t-il indique la figure 26.

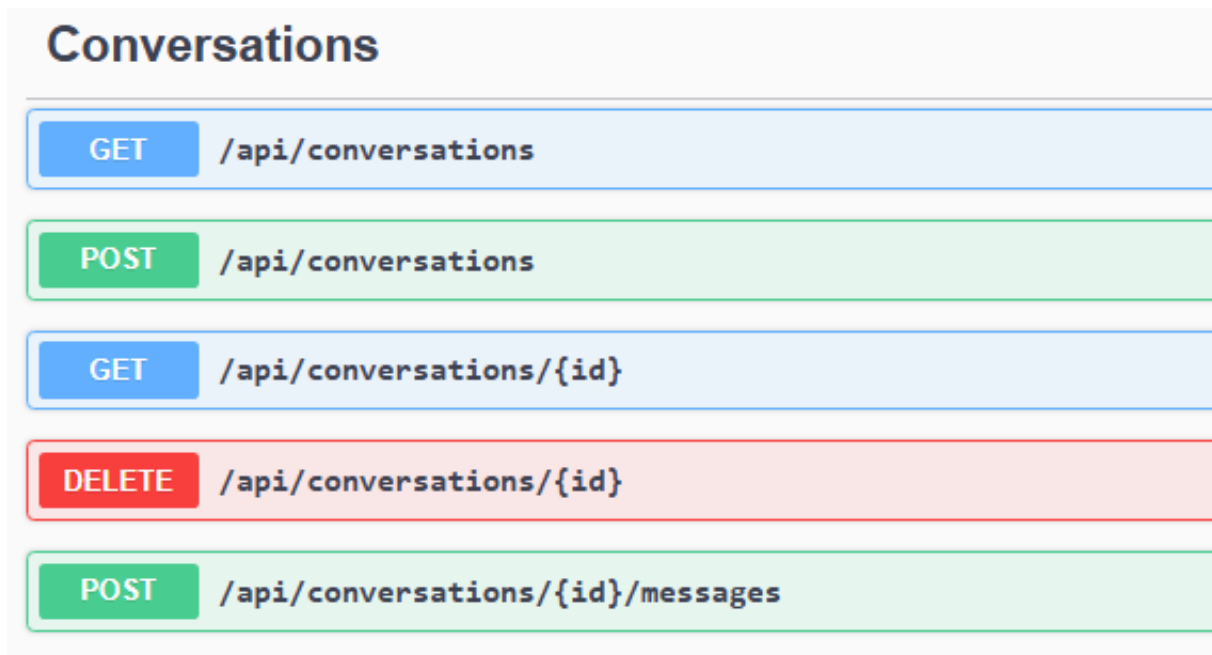


FIGURE 26. Documentation Swagger de Conversation Service

4.2.2. Mise en place de l'Authentication Service

Dans une architecture orientée micro services, l'authentification constitue un socle critique pour la sécurité et la gestion des accès. Pour répondre à ces exigences, un micro service dédié, implémenté avec ASP.NET Core et s'appuyant sur les JSON Web Tokens (JWT), a été développé. Ce service est responsable de l'identification des utilisateurs, de la génération des jetons d'accès et du contrôle des autorisations. Les données utilisateurs sont persistées dans une base SQL Server via Entity Framework Core.

4.2.2.1. Diagramme de classes

Le diagramme de classes dans la Figure 27 illustre les principaux composants du service d'authentification ainsi que leurs relations.

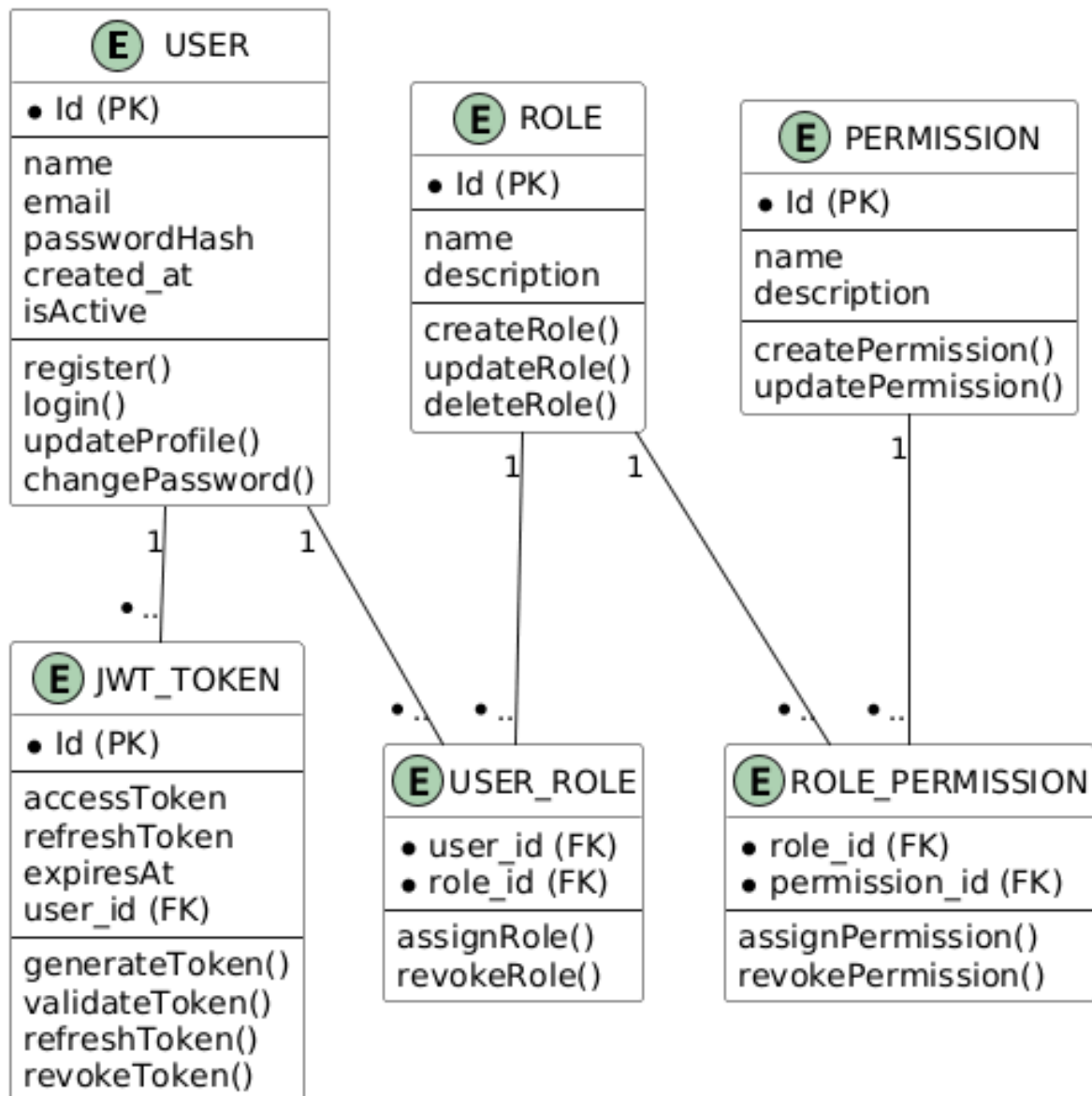


FIGURE 27. Diagramme de classes du Authentication Service

4.2.2.2. Cas d'utilisation

Le service d'authentification expose plusieurs fonctionnalités clés permettant la gestion des utilisateurs et des accès. Ces derniers sont présentés dans un diagramme de cas d'utilisation à la Figure 28.

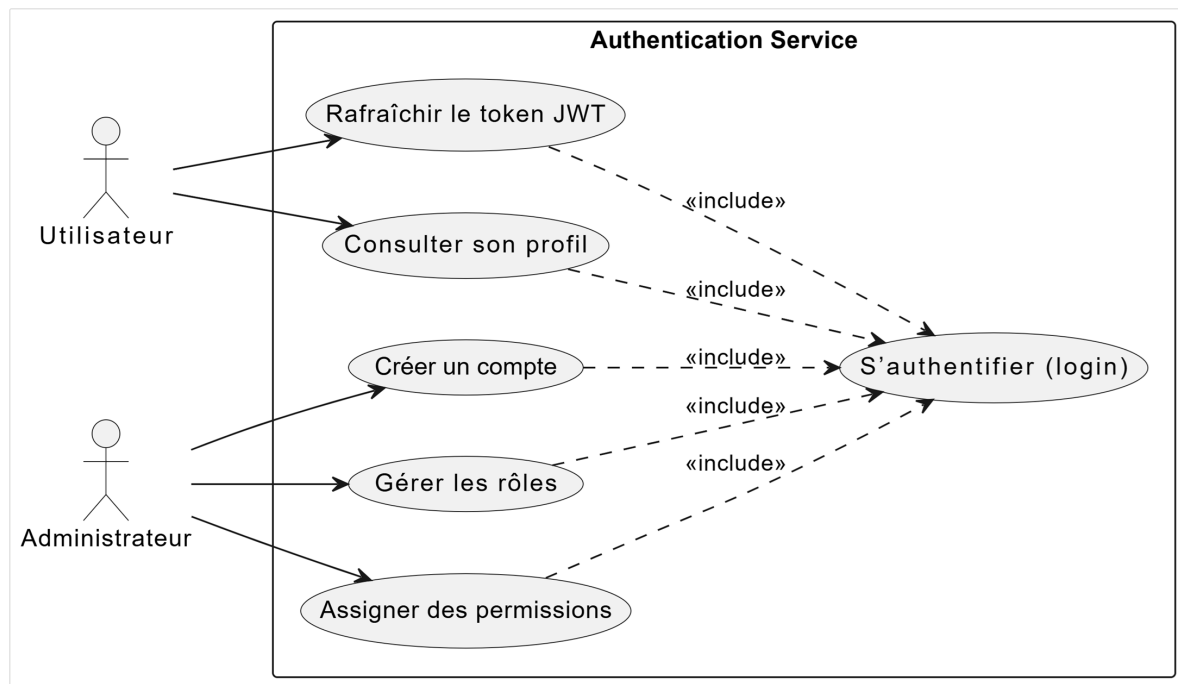


FIGURE 28. Diagramme de cas d'utilisation du Authentication Service

Le service d'authentification assure la gestion complète des identités et des accès au sein de la plateforme. Il permet l'inscription sécurisée des utilisateurs grâce au stockage chiffré des mots de passe, l'authentification via la vérification des identifiants, ainsi que la génération de jetons JWT garantissant un accès sécurisé aux ressources. Afin d'améliorer l'expérience utilisateur, un mécanisme de rafraîchissement des tokens est mis en place pour maintenir les sessions actives sans nécessiter de reconnexion fréquente. Enfin, le service assure la gestion des rôles et des permissions afin de contrôler l'accès aux différentes fonctionnalités selon les privilèges attribués à chaque utilisateur.

4.2.2.3. Diagramme de Séquence

La figure 29 détaille le scénario nominal d'authentification d'un utilisateur métier.

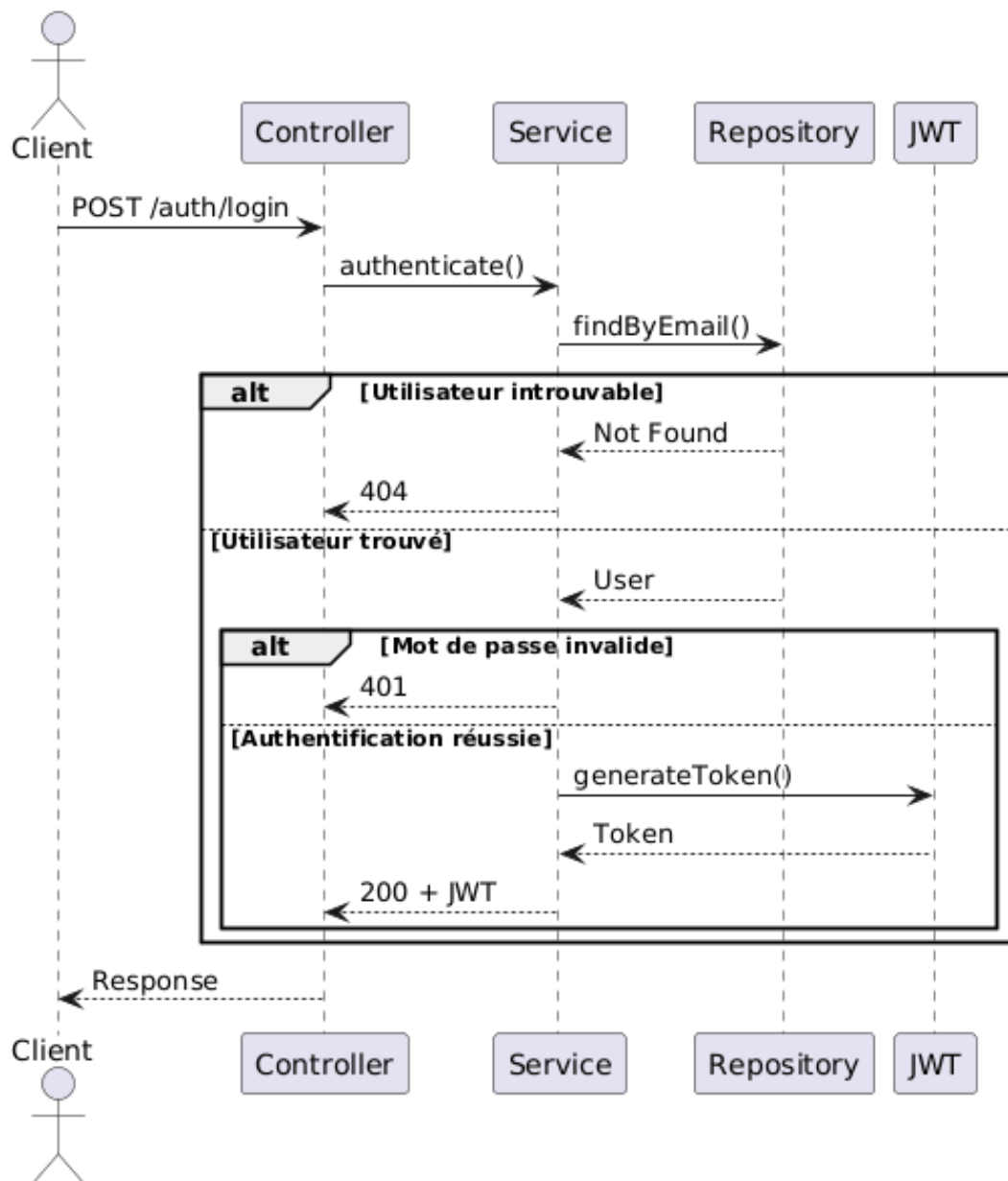


FIGURE 29. Diagramme de séquence de Service Authentication

La création des comptes utilisateurs métier et l'assignation des rôles sont des opérations réservées à l'administrateur système, dont la séquence est représentée à la figure 30. Les exceptions — identifiants invalides, token expiré, compte verrouillé — sont systématiquement journalisées et retournées au client sous forme de codes d'erreur HTTP normalisés.

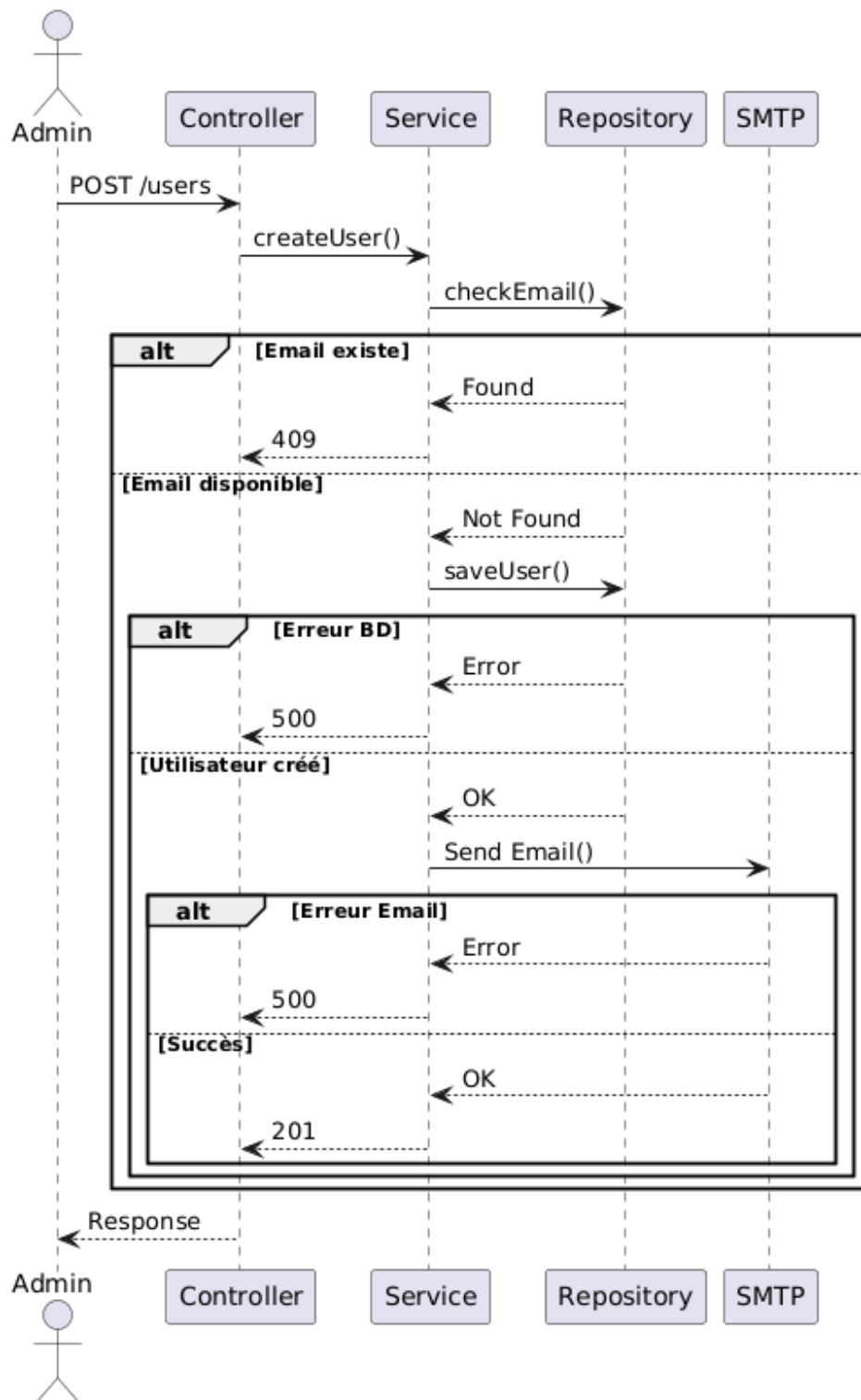


FIGURE 30. Diagramme de séquence du service d'authentification – Création de compte

Cette conception stateless (hors refresh token) est parfaitement adaptée aux architectures micro services distribuées. L'utilisation de JWT évite les appels systématiques au service d'authentification pour chaque requête, tout en permettant une vérification décentralisée des droits via la signature du jeton.

4.2.2.4. Tests Unitaires

Afin de garantir la fiabilité et la robustesse du service d'authentification, une suite de tests unitaires a été mise en place en suivant les principes du *Test-Driven Development* (TDD). Ces tests couvrent l'ensemble des cas nominaux et des cas d'exception susceptibles d'affecter les composants critiques du service.

Les tests unitaires couvrent les trois composants principaux du service : `AuthService`, `TokenService` et `AuthController`. Le tableau 16 synthétise les cas de test définis pour les scénarios les plus critiques.

TABLE 16. Cas de tests unitaires du service d'authentification

Composant	Scénario testé	Résultat attendu
AuthService	Identifiants valides	Retourne l'utilisateur
AuthService	Mot de passe incorrect	Lève <code>UnauthorizedException</code>
AuthService	Utilisateur inexistant	Lève <code>NotFoundException</code>
AuthService	Compte verrouillé (423)	Lève <code>AccountLockedException</code>
TokenService	Génération access token	Token JWT non nul et signé
TokenService	Claims injectés correctement	Contient <code>userId</code> et <code>role</code>
TokenService	Token expiré (validation)	Retourne <code>false</code>
TokenService	Signature invalide	Retourne <code>false</code>
AuthController	Login nominal	Retourne <code>200 OK</code> avec tokens
AuthController	Payload manquant	Retourne <code>400 Bad Request</code>
AuthController	Identifiants erronés	Retourne <code>401 Unauthorized</code>

4.2.2.5. Documentation Swagger

Afin de faciliter l'intégration et les tests des fonctionnalités du *Service d'Authentification*, une documentation interactive des APIs REST a été mise en place à l'aide de Swagger/OpenAPI. Cette interface permet de centraliser l'ensemble des endpoints exposés par le micro service tout en offrant une visualisation claire des routes disponibles, des paramètres attendus ainsi que des formats de réponses retournées par le système.

La documentation Swagger constitue également un outil essentiel pour les phases de développement, de validation et d'intégration entre micro services. Elle permet aux développeurs de tester directement les opérations d'authentification, de gestion des utilisateurs et de sécurisation des accès sans recourir à des outils externes supplémentaires.

La figure 31 présente l'interface Swagger du *Service d'Authentification*, illustrant les principales routes REST exposées par le micro service.

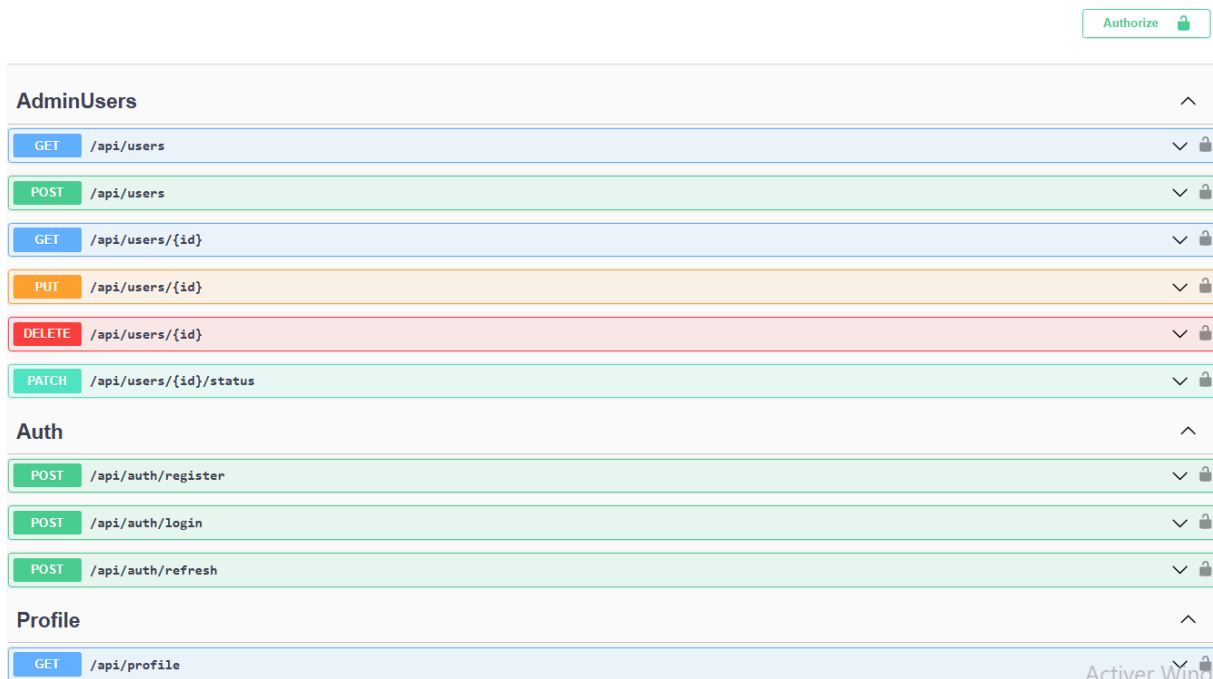


FIGURE 31. Documentation Swagger des endpoints REST exposés par le Service d'Authentification

4.2.3. Mise en place de l'API Gateway

Dans une architecture micro services distribuée, une API Gateway constitue le point d'entrée unique pour tous les clients (frontend web, applications mobiles, services externes). Il agit comme un reverse proxy centralisé, responsable du routage des requêtes vers les micro services appropriés, de l'agrégation des réponses et de l'application des politiques transversales (authentification, autorisation, résilience, observabilité). Sans API Gateway, chaque client devrait connaître l'adresse exacte de chaque micro service, ce qui complexifie la maintenance, réduit la sécurité et rend difficile l'évolution de l'architecture.

Pour cette plateforme, un API Gateway dédié a été implémenté en ASP.NET Core utilisant la bibliothèque YARP (Yet Another Reverse Proxy), un projet open-source maintenu par Microsoft. YARP permet de construire facilement un reverse proxy haute performance, entièrement configurable et intégrable dans l'écosystème .NET.

4.2.3.1. Diagramme de Séquence

Le diagramme de séquence dans la figure 32 illustre le flux standard d'une requête client. Après réception de la requête HTTP accompagnée du jeton JWT, l'API Gateway délègue au filtre JWT la validation du token auprès du service d'authentification. En cas de succès, le

routeur détermine dynamiquement le micro service cible (RAG ou MDX). La requête est alors transmise au service concerné, tandis qu'un journal d'audit enregistre l'opération. La réponse JSON obtenue est enfin renvoyée au client, garantissant ainsi un routage sécurisé et traçable.

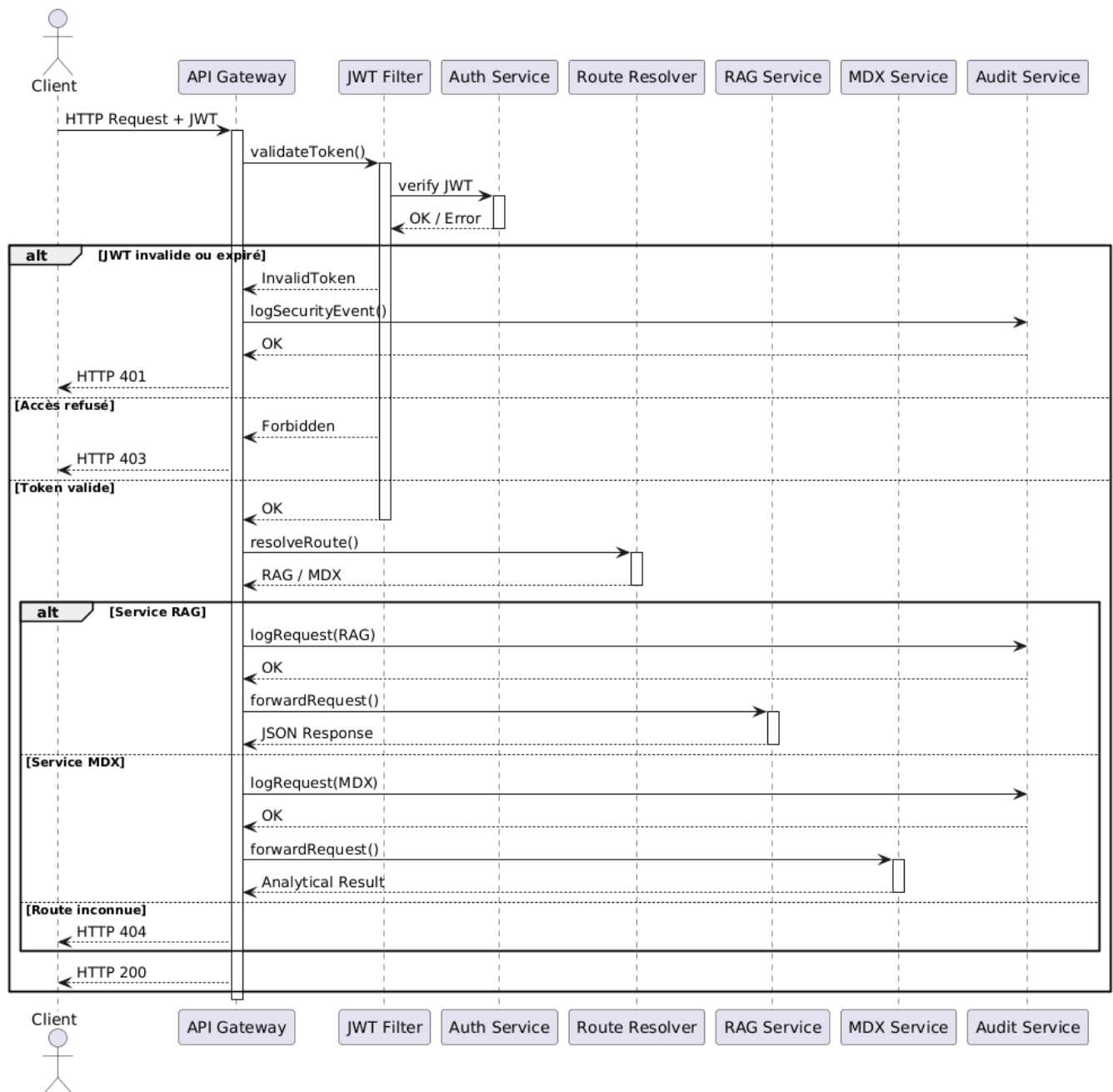


FIGURE 32. Diagramme de séquence du service API Gateway – Fonctionnement nominal

4.2.3.2. Configuration YARP : routes et clusters

Le cœur de l'API Gateway repose sur la configuration YARP, stockée dans le fichier `yarp.json`. Cette configuration définit les règles de routage (routes) et les services backend (clusters) vers lesquels rediriger le trafic.

1 {

```

2  "ReverseProxy": {
3    "Routes": {
4      "auth-public": {
5        "ClusterId": "auth-cluster",
6        "AuthorizationPolicy": "Anonymous",
7        "Match": {
8          "Path": "/api/auth/{**catch-all}"
9        },
10       "Transforms": [
11         { "RequestHeaderOriginalHost": "false" },
12         { "X-Forwarded": "Set" }
13       ]
14     },
15     "mdx-clients": {
16       "ClusterId": "mdx-cluster",
17       "AuthorizationPolicy": "authenticated",
18       "Match": {
19         "Path": "/api/clients/{**catch-all}"
20       },
21       "Transforms": [
22         { "RequestHeaderOriginalHost": "false" },
23         { "X-Forwarded": "Set" }
24       ]
25     },
26     .....
27   "Clusters": {
28     "auth-cluster": {
29       "LoadBalancingPolicy": "RoundRobin",
30       "Destinations": {
31         "primary": { "Address": "http://auth-service:8080/" }
32       },
33       "HealthCheck": {
34         "Active": {
35           "Enabled": true,
36           "Interval": "00:00:10",
37           "Timeout": "00:00:05",
38           "Policy": "ConsecutiveFailures",
39           "Path": "/health"
40         },
41         "ConsecutiveFailuresHealthPolicy": { "DefaultThreshold": 3 }
42       }
43     },
44
45     "mdx-cluster": {
46       "LoadBalancingPolicy": "RoundRobin",
47       "Destinations": {
48         "primary": { "Address": "http://mdx-service:8080/" }
49       },
50       "HealthCheck": {
51         "Active": {
52           "Enabled": true,

```

```

53     "Interval": "00:00:10",
54     "Timeout": "00:00:05",
55     "Policy": "ConsecutiveFailures",
56     "Path": "/health"
57 },
58     "ConsecutiveFailuresHealthPolicy": { "DefaultThreshold": 3 }
59 }
60 },
61 .....
62 }

```

Code 14 – Configuration YARP - routes et clusters

Les routes associent un motif d'URL à un cluster. Les clusters définissent les destinations des micro services grâce au réseau docker expliqué dans la section 4.3.1.1.

4.2.3.3. Interconnexion des services

Le service **API Gateway** agit comme une façade unifiée permettant de masquer la complexité de l'architecture micro services, tout en offrant un point d'accès unique, sécurisé et scalable à l'ensemble de la plateforme. Grâce à ce rôle central, l'API Gateway assure le routage intelligent des requêtes vers les différents services backend, tout en appliquant des politiques transversales telles que l'authentification, la journalisation et la résilience.

La Table 17 illustre de manière synthétique le mécanisme de redirection des requêtes vers les services appropriés. Elle permet de comprendre comment chaque route définie dans la Gateway est associée à un cluster et à un micro service cible.

Par exemple, toute requête HTTP dont l'URI (Uniform Resource Identifier) commence par `/mdx` est automatiquement interceptée par la Gateway, puis redirigée vers le *Mdx Service*. La route est conservée intacte lors de la redirection, ce qui permet au service cible de traiter la requête comme s'il était directement exposé au client. Ce mécanisme est également appliqué aux autres services tels que l'authentification, le chat-bot et l'ETL, garantissant ainsi une cohérence dans la gestion des endpoints et une séparation claire des responsabilités.

TABLE 17. Mapping des routes API Gateway vers micro services

Route	Cluster	Service cible	Adresse
/auth/**	auth-cluster	Authentification	http://auth-service:8080
/chat/**	chat-cluster	chat-bot	http://chat-bot-service:8080
/mdx/**	mdx-cluster	MDX	http://mdx-service:8080
/conv/**	conversation-cluster	Conversation	http://conv-service:8080

4.2.4. Implémentation du Frontend

Le frontend de la plateforme constitue la couche de présentation unifiée permettant d'interagir avec l'ensemble des micro services backend. Développé en React 18 (TypeScript) selon une architecture orientée composants, il repose sur un écosystème technique cohérent : Redux Toolkit pour la gestion d'état, React Router v6 pour la navigation déclarative, Axios pour la communication HTTP avec l'API Gateway, Recharts pour la visualisation décisionnelle, et Socket.IO-client pour les échanges temps réel avec le service LLM2MDX. Le déploiement est conteneurisé via Docker et servi par Nginx.

4.2.4.1. Architecture et organisation du projet

L'organisation du code suit les principes de l'architecture Atomic Design, distinguant les composants atomiques réutilisables (ui/), les composants métier (components/), les pages (pages/), les hooks personnalisés (hooks/) et les services de communication (services/). Cette séparation garantit la maintenabilité et l'évolutivité de l'application. La figure 33 illustre l'arborescence complète du projet.

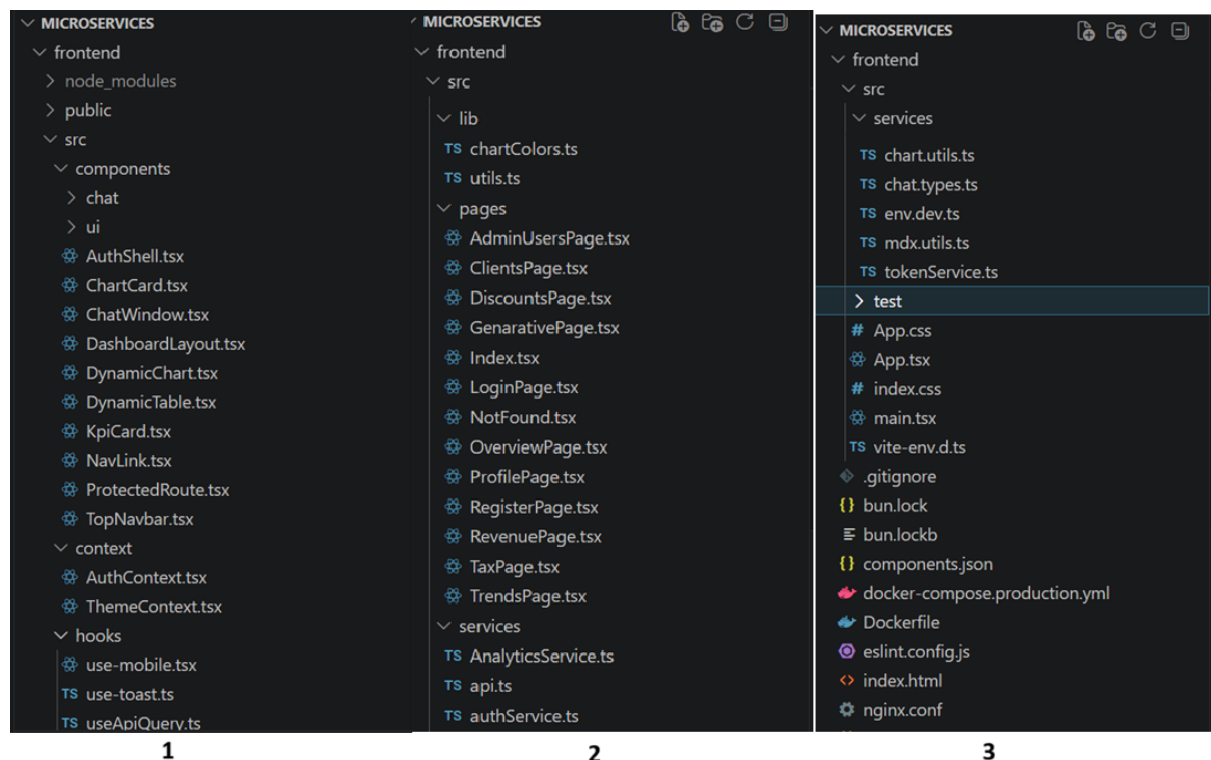


FIGURE 33. Structure du projet frontend selon l'architecture Atomic

La navigation est configurée dans `App.tsx` via React Router, avec une distinction explicite entre routes publiques et routes protégées (figure 34). Le composant `ProtectedRoute` intercepte toute tentative d'accès non authentifié en vérifiant la validité du jeton JWT stocké dans les

Cookies, et redirige vers /login en cas d'absence ou d'expiration. Un second niveau de protection restreint les routes d'administration aux utilisateurs disposant du rôle Admin.

```

24  const queryClient = new QueryClient();
25
26  const App = () => {
27    <QueryClientProvider client={queryClient}>
28      <ThemeProvider>
29        <TooltipProvider>
30          <Toaster />
31          <Sonner />
32          <BrowserRouter>
33            <AuthProvider>
34              <Routes>
35                <Route path="/login" element={<LoginPage />} />
36                <Route element={<ProtectedRoute roles={["User", "Admin"]} />>
37                  <Route element={<DashboardLayout />} />
38                  <Route path="/" element={<OverviewPage />} />
39                  <Route path="/revenue" element={<RevenuePage />} />
40                  <Route path="/trends" element={<TrendsPage />} />
41                  <Route path="/tax" element={<TaxPage />} />
42                  <Route path="/clients" element={<ClientsPage />} />
43                  <Route path="/discounts" element={<DiscountsPage />} />
44                  <Route path="/chat" element={<GenerativePage />} />
45                  <Route path="/me" element={<ProfilePage />} />
46                  <Route element={<ProtectedRoute roles={["Admin"]} />>
47                    <Route path="/admin/users" element={<AdminUsersPage />} />
48                  </Route>
49                </Route>
50              </Route>
51              <Route path="*" element={<NotFound />} />
52            </Routes>
53          </AuthProvider>
54        </BrowserRouter>
55      </TooltipProvider>
56    </ThemeProvider>
57  </QueryClientProvider>
58  };

```

FIGURE 34. Configuration des routes publiques et protégées dans App.tsx

4.2.4.2. Communication avec les micro services

Les communications entre le frontend et l'infrastructure backend s'effectuent principalement via des APIs REST sécurisées exposées par l'API Gateway (figure 35). Cette méthode offre la possibilité d'isoler le frontend de la complexité interne des micro services tout en assurant une meilleure évolutivité de la plateforme. Les services spécialisés (revenueApi, mdxService, etc.) exposent des méthodes typées qui encapsulent les appels REST vers les endpoints métier (chiffre d'affaires par période, concentration client, etc.). En cas de perte de connexion ou d'exception, des données par défaut sont injectées pour éviter le risque de panne avec des variables non définies.

```

1 import { API_GATEWAY_URL } from "../env.dev";
2
3 const API_BASE = (API_GATEWAY_URL as string);
4
5 interface ApiResponse<T> {
6   success: boolean;
7   count?: number;
8   data: T[];
9 }
10
11 async function fetchApi<T>(endpoint: string): Promise<T[]> {
12   try {
13     const res = await fetch(`${API_BASE}${endpoint}`);
14     const json: ApiResponse<T> = await res.json();
15     if (json.success) return json.data;
16     return [];
17   } catch {
18     return getMockData<T>(endpoint);
19   }
20 }
21
22 export const revenueApi = {
23   byYear: () => fetchApi<{ label: string; value: number }>("/revenue/by-year"),
24   byMonth: (year: number) => fetchApi<{ label: string; value: number }>(`/revenue/by-month/${year}`),
25   byQuarterDoctype: () => fetchApi<{ label: string; value: number }>("/revenue/by-quarter-doctype"),
26   byClient: () => fetchApi<{ client: string; totalHT: number; totalTTC: number }>("/revenue/by-client"),
27   topClients: () => fetchApi<{ client: string; totalHT: number; totalTTC: number }>("/revenue/top-clients"),
28   bottomClients: () => fetchApi<{ client: string; totalHT: number; totalTTC: number }>("/revenue/bottom-clients"),
29   concentration: () => fetchApi<{ client: string; revenue: number; sharePercent: number }>("/revenue/concentration"),
30 };
31

```

FIGURE 35. Configuration de communication entre le frontend et l'infrastructure de backend

4.2.4.3. Authentification et sécurité

L'authentification est déléguée au micro service Auth Service via l'API Gateway. Après soumission des identifiants depuis LoginPage (figure 36) — dont la validation est assurée par React Hook Form et Zod — deux jetons JWT (access et refresh token) sont retournés par le backend et persisté dans les Cookies. Le jeton d'accès est ensuite injecté automatiquement dans l'en-tête Authorization de chaque requête sortante par un intercepteur Axios configuré dans `apiClient.ts`. Un mécanisme de rafraîchissement silencieux gère les expirations : toute réponse HTTP 401 déclenche une tentative de renouvellement du token avant de rejouer la requête initiale. L'état d'authentification est centralisé dans `authSlice` (Redux Toolkit), exposant les informations utilisateur, les rôles et le statut de session.

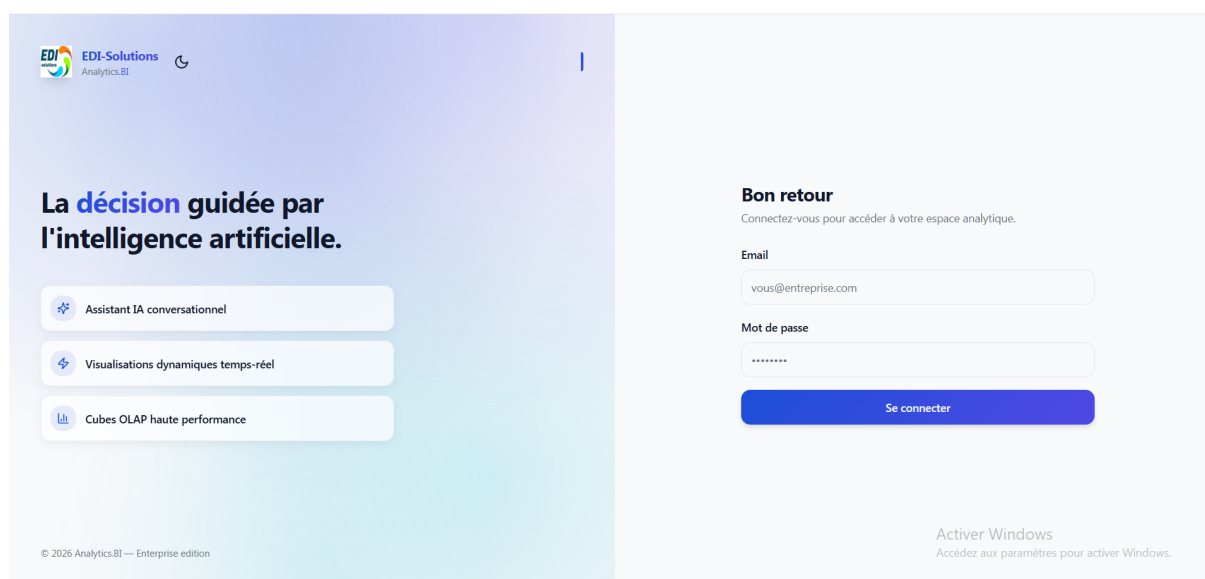


FIGURE 36. Interface de login

L'état applicatif est structuré autour de trois slices Redux : `authSlice` pour les données de session, `mdxSlice` pour les résultats des requêtes analytiques (revenus, marges, tendances), et `uiSlice` pour les préférences d'interface (thème, notifications). Les opérations asynchrones sont encapsulées dans des `createAsyncThunk`, assurant une gestion uniforme des états de chargement et d'erreur. Les réponses MDX sont mises en cache localement dans le slice pendant cinq minutes afin de limiter la charge sur les micro services analytiques.

4.2.4.4. Tableau de bord décisionnel

Le tableau de bord principal (`OverviewPage`) constitue le point d'entrée après authentification. Il agrège en temps quasi-réel les indicateurs clés de performance — chiffre d'affaires, marge totale, taux de marge — récupérés depuis le service MDX via l'API Gateway, et les présente sous forme de KPI cards accompagnées d'une courbe d'évolution mensuelle et d'un graphique de concentration du CA par client (figure 37). Les visualisations sont réalisées avec `Recharts`, en composants réutilisables paramétrables par les données, le titre et la palette de couleurs, avec gestion native du responsive et des animations de chargement.

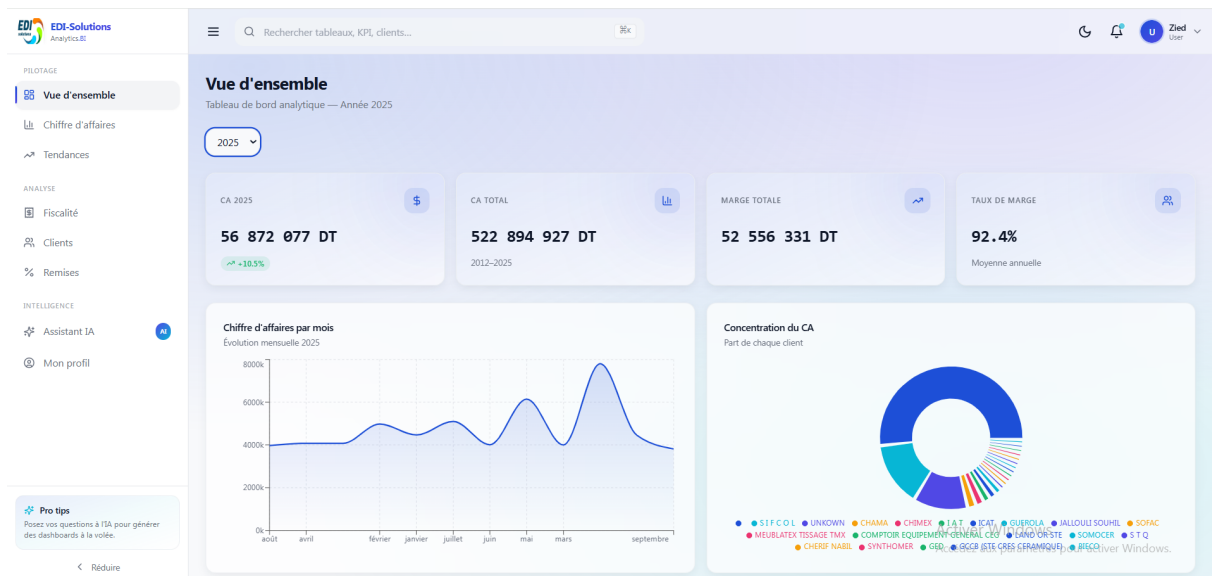


FIGURE 37. Tableau de bord principal avec KPIs et visualisations analytiques

4.2.4.5. Interface Intelligente

Le chat-bot analytique, accessible via la route `/chat`, représente la fonctionnalité d'intelligence augmentée de la plateforme. Il établit une connexion persistante avec le micro service LLM2MDX par Socket.IO, permettant une interaction en langage naturel avec les données décisionnelles. L'utilisateur soumet une question analytique ; le service backend génère et exécute la requête MDX correspondante, puis retourne un résultat structuré. Le frontend interprète dynamiquement ce résultat : les données tabulaires sont rendues dans un composant `DynamicTable`, tandis que les séries temporelles ou agrégats sont automatiquement visualisés par `DynamicChart` selon le type de données détecté (figures 38, 39, 40).

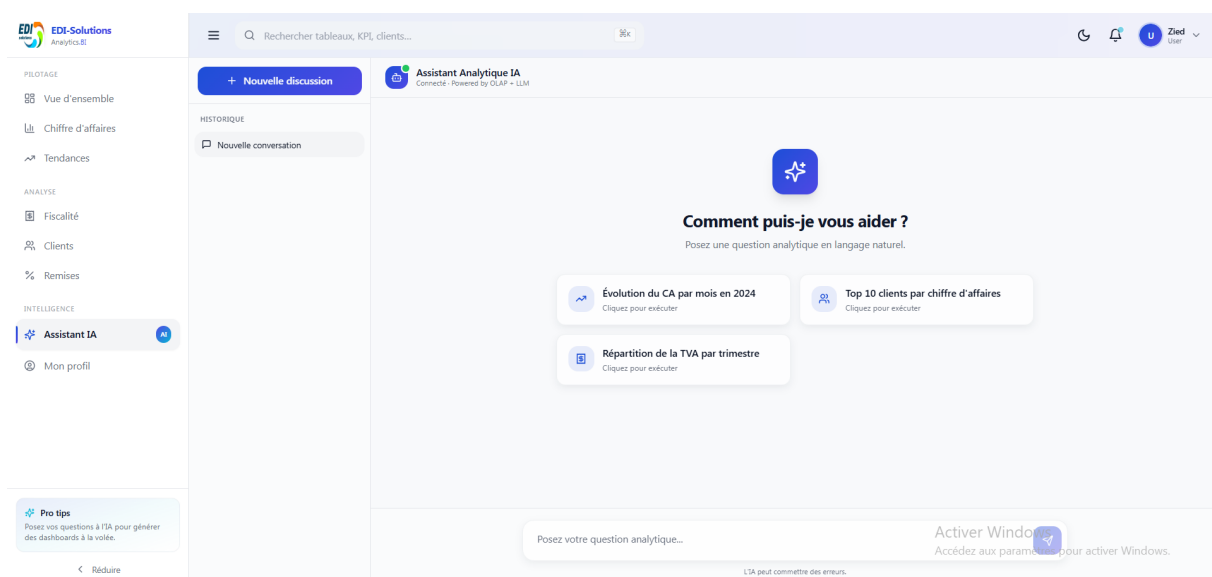


FIGURE 38. Interface initiale du chat-bot analytique

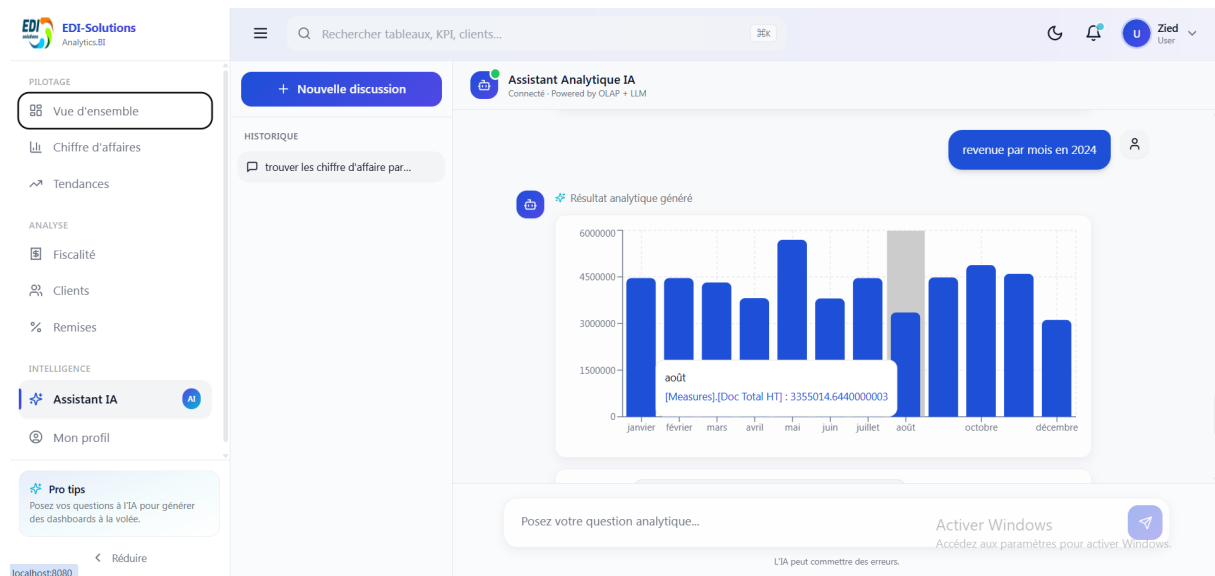


FIGURE 39. Réponse du chat-bot sous forme de graphique

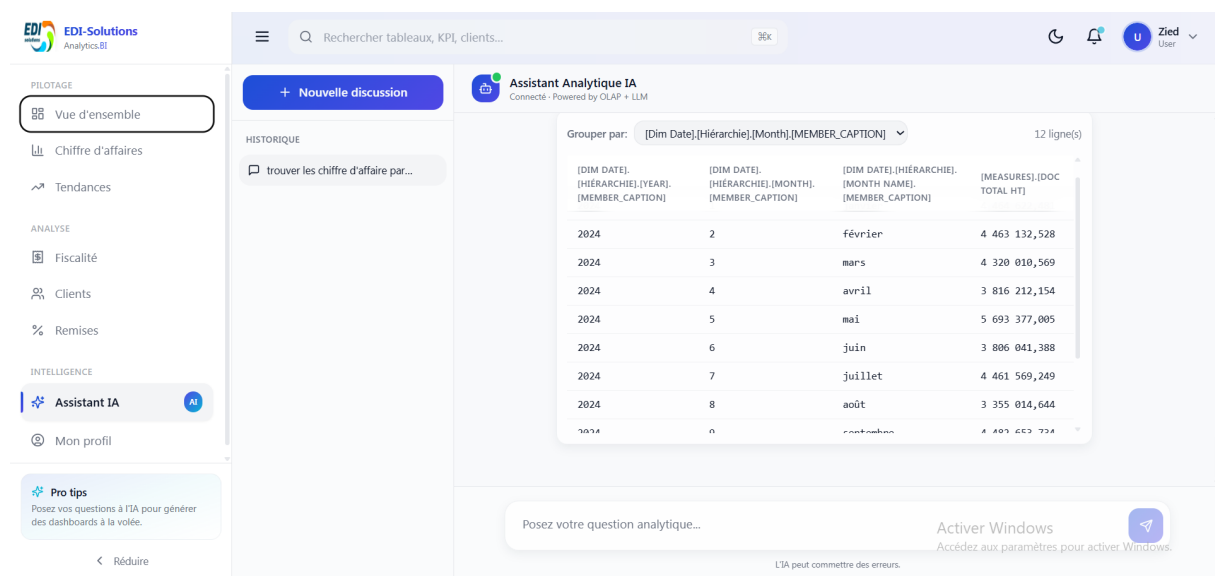


FIGURE 40. Réponse du chat-bot sous forme de tableau

4.3. Sprint 4 : Industrialisation et DevOps

Le Sprint 4 vise à renforcer la qualité globale du système en introduisant des pratiques d'industrialisation et en améliorant l'expérience utilisateur.

Les objectifs principaux incluent :

- La mise en place d'une chaîne DevOps (CI/CD)
- La conteneurisation des services via Docker
- L'amélioration des performances et de la scalabilité
- L'optimisation de l'interface utilisateur

— L'intégration de mécanismes de sécurité avancés

Ce dernier sprint représente la phase d'industrialisation du projet. Il permet de rendre la plateforme plus robuste, déployable et adaptée à un environnement de production réel. Les *User Story* attendue par ceci sont présentées dans la Table 18.

TABLE 18. *User Stories du Sprint 4*

ID	Rôle	Besoin	Bénéfice	Critère d'acceptation	Priorité	Semaine
US4.1	Équipe DevOps	Automatiser le déploiement via GitHub Actions	Réduire les interventions manuelles	Pipeline CI/CD opérationnel sur VPS	Haute	S13–S14
US4.2	Administrateur système	Conteneuriser les micro services avec Docker	Garantir portabilité et isolation	Images publiées sur GHCR et stack opérationnelle	Haute	S13
US4.3	Architecte logiciel	Gérer les versions via GitOps	Assurer la traçabilité des déploiements	Rollback réalisable en moins de 2 minutes	Haute	S14
US4.4	Administrateur sécurité	Renforcer la sécurité CI/CD	Protéger l'infrastructure	Aucun secret exposé ; SSH par clé dédiée	Haute	S14–S15
US4.5	Équipe technique	Superviser les conteneurs avec Portainer	Détecter rapidement les anomalies	Notifications Telegram et monitoring actifs	Moyenne	S15–S16
US4.6	Architecte logiciel	Valider la qualité du code à chaque build	Prévenir les régressions	Rapports de tests et couverture disponibles	Moyenne	S13–S14
US4.7	Utilisateur métier	Obtenir des réponses chat-bot rapides	Améliorer l'expérience utilisateur	Temps moyen < 10 s ; dashboard < 2 s	Basse	S16

4.3.1. Architecture de Déploiement

4.3.1.1. Vue globale de l'infrastructure

L'infrastructure de déploiement repose sur une architecture hybride. Les micro services applicatifs (.NET Core, Python FastAPI, React) sont conteneurisés via Docker et déployés sur un VPS Linux (Ubuntu 22.04 LTS), orchestrés par Docker Compose. En revanche, SQL Server Analysis Service — qui ne dispose pas de support Linux pour son moteur multidimensionnel MDX — est déployé sur un serveur Windows Server 2019 distinct, hébergeant également SQL Server 2022 Enterprise. La communication entre les micro services .NET et SSAS s'effectue via le driver ADOMD.NET sur le protocole TCP/IP interne. Cette séparation préserve l'intégrité de l'infrastructure Docker tout en respectant les contraintes de compatibilité de SSAS.

L'isolation des processus applicatifs est assurée par Docker, moteur de conteneurisation qui encapsule chaque micro service dans un environnement d'exécution étanche, portable et reproductible, indépendamment des spécificités du système hôte. Docker Compose orchestre l'ensemble de ces conteneurs au sein d'un réseau interne dédié, nommé *micro services-net*, qui isole les communications inter-services du réseau public. Seul le *gateway-service* expose un port vers l'extérieur (8080), les autres services communiquant exclusivement via la résolution DNS interne de Docker.

Les images Docker de chaque service sont hébergées dans le GitHub Container Registry (GHCER), registre privé intégré à l'écosystème GitHub qui simplifie considérablement la gestion des credentials et l'intégration avec GitHub Actions. Le pipeline CI/CD est entièrement piloté par GitHub Actions, plateforme d'automatisation native de GitHub, permettant de définir des workflows déclaratifs au format YAML, déclenchés par des événements Git précis. Enfin, la couche de persistance repose sur Microsoft SQL Server 2022, déployé lui aussi sous forme de conteneur, avec des volumes Docker pour la durabilité des données.

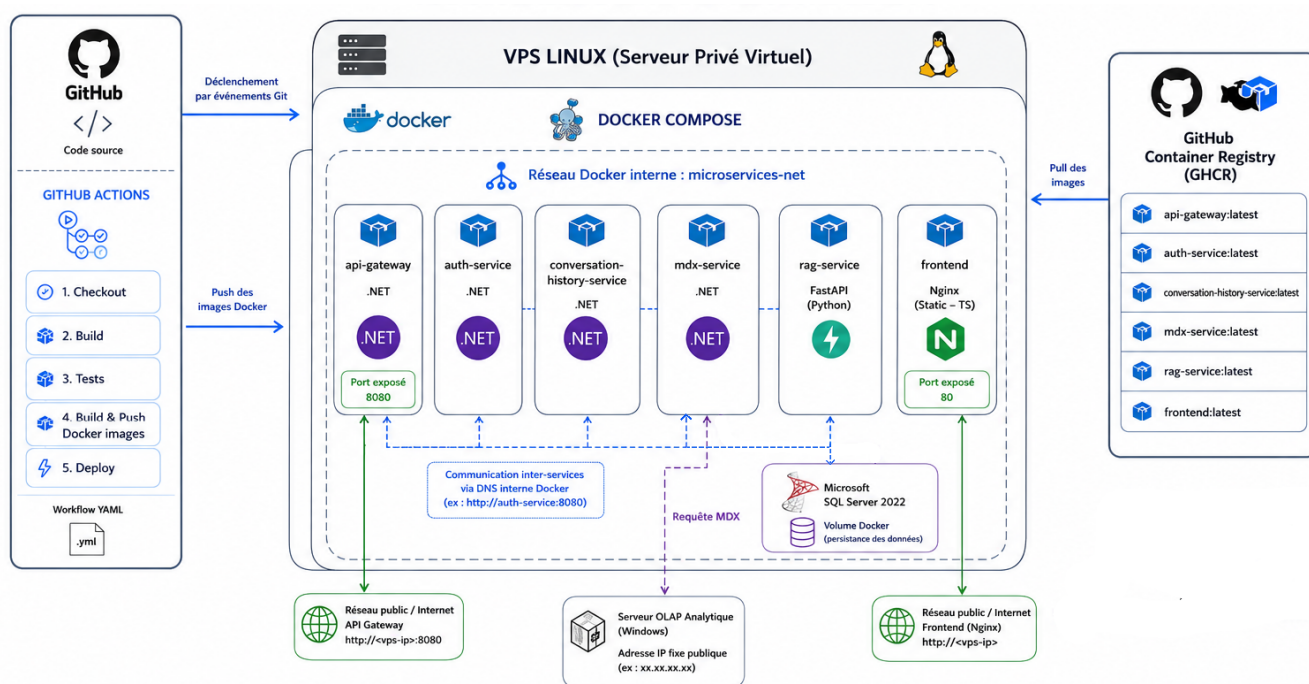


FIGURE 41. Vue globale de l'infrastructure DevOps

TABLE 19. Composants techniques de l'architecture de l'infrastructure

Composant	Technologie	Environnement	Protocole d'accès
Data Warehouse	SQL Server 2022	Windows Server 2019	ODBC
Cube OLAP	SSAS Multidimensionnel	Windows Server 2019	ADOMD.NET / XMLA
ETL	SSIS	Windows Server 2019	Interne
MDX Service	ASP.NET Core	Docker (Linux)	REST / HTTP
RAG Service	Python FastAPI	Docker (Linux)	REST / HTTP
Auth Service	ASP.NET Core	Docker (Linux)	REST / HTTP
Conversation Service	ASP.NET Core	Docker (Linux)	REST / HTTP
API Gateway	YARP (.NET)	Docker (Linux)	HTTP
Frontend	React.js + Nginx	Docker (Linux)	HTTP

4.3.1.2. Séparation des responsabilités : deux repositories distincts

L'une des décisions architecturales les plus structurantes du dispositif DevOps mis en place réside dans la séparation stricte entre le code applicatif et la configuration d'infrastructure. Cette séparation se traduit par l'utilisation de deux repositories GitHub distincts, dont les responsabilités sont clairement délimitées et mutuellement exclusives.

Le premier repository, désigné micro services, contient l'intégralité du code source des quatre services de la plateforme, organisés en répertoires autonomes (auth-service/, mdx-service/, ai-service/, gateway-service/), ainsi que les Dockerfiles associés et les workflows GitHub Actions dédiés à chaque service. Ce repository est le domaine exclusif des équipes de développement : chaque modification de code, chaque correctif, chaque nouvelle fonctionnalité y est soumise, déclenchant automatiquement le pipeline CI du service concerné.

Le second repository, nommé infra-deploy, remplit une fonction radicalement différente : il constitue la source de vérité de l'état de production. Il ne contient pas de code applicatif, mais exclusivement la configuration de déploiement : le fichier docker-compose.prod.yml définissant la topologie des conteneurs en production, le fichier .env.prod référençant les versions précises de chaque image déployée, et le workflow de déploiement automatisé. Ce repository est mis à jour non par les développeurs, mais par les pipelines CI eux-mêmes, de manière entièrement automatisée, après chaque build réussi.

Cette architecture à deux repositories présente plusieurs avantages fondamentaux. D'une part, elle découple totalement les cycles de vie du développement et du déploiement : un développeur peut pousser des modifications dans le repository micro services sans interférer avec les autres services en production. D'autre part, elle confère à infra-deploy un rôle de journal

immuable des versions déployées, facilitant les audits, la traçabilité et les opérations de rollback. Enfin, elle permet de protéger la configuration de production par des contrôles d'accès stricts, indépendamment des droits sur le code source.

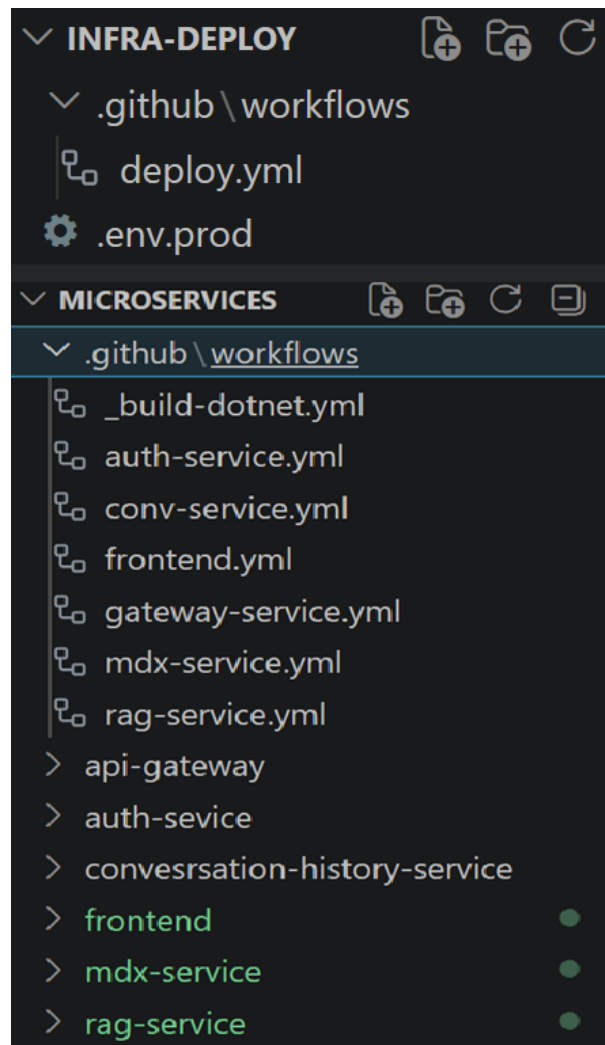


FIGURE 42. Séparation des deux repositories : micro services (code + CI) et infra-deploy (versions + CD).

4.3.1.3. Architecture CI/CD globale

Le schéma d'automatisation de la plateforme repose sur un enchaînement causal précis, dans lequel chaque étape conditionne la suivante de manière déterministe. Lorsqu'un développeur pousse une modification sur la branche principale du repository micro services, GitHub Actions détecte l'événement et n'active que le pipeline du service affecté, grâce aux filtres de chemins (paths) configurés dans chaque workflow. Cette granularité évite les recompilations et re-déploiements inutiles des services non modifiés, préservant ainsi les ressources de calcul et réduisant les délais de livraison [46].

4.3.2. Intégration Continue (CI)

4.3.2.1. Définition d'un pipeline CI/CD réutilisable pour les micro services .NET

La duplication des pipelines CI/CD pour chaque service entraîne une complexité opérationnelle élevée, une incohérence des pratiques de build, ainsi qu'une augmentation progressive de la dette DevOps. Afin de remédier à cette problématique, la plateforme adopte une approche basée sur les *Reusable Workflows* de GitHub Actions.

Cette approche consiste à centraliser la logique de build et de déploiement dans un workflow générique, réutilisable par l'ensemble des micro services .NET (auth-service, mdx-service, conversation-service et gateway-service), tout en conservant un niveau de flexibilité suffisant pour s'adapter aux spécificités de chaque service.

Le pipeline est défini comme un workflow réutilisable, montré dans la figure 43, déclenché via l'événement `workflow_call`, permettant son invocation depuis d'autres workflows sans duplication de logique. Ce choix architectural transforme le pipeline en une brique CI/CD standardisée, pouvant être intégrée dans n'importe quel micro service de la plateforme.

Afin de garantir la généricité du pipeline, plusieurs paramètres sont injectés dynamiquement lors de son appel. Cette conception permet un découplage complet entre :

- le code du pipeline CI/CD,
- la structure interne des micro services,
- la logique de déploiement.

Ainsi, un unique workflow peut orchestrer plusieurs services hétérogènes sans aucune modification interne.

Le pipeline intègre également la gestion des secrets nécessaires aux opérations CI/CD. Ces secrets sont utilisés respectivement pour :

- l'authentification au registre de conteneurs GHCR (GitHub Container Registry),
- la mise à jour du repository d'infrastructure *infra-deploy*.

Cette séparation respecte les principes du *Security by Design*, en évitant toute exposition des informations sensibles dans le code source.

```

.github > workflows > _build-dotnet.yml
1  name: Reusable .NET build
2  on:
3    workflow_call:
4      inputs:
5        service-name:
6          required: true
7          type: string
8        service-path:
9          required: true
10         type: string
11       solution-file:
12         required: true
13         type: string
14       env-var-name:
15         required: true
16         type: string
17     secrets:
18       GHCR_TOKEN:
19         required: true
20       DEPLOY_REPO_TOKEN:
21         required: true
22   env:
23     DOTNET_VERSION: "8.0.x"
24

```

FIGURE 43. Configuration d'un workflow CI/CD réutilisable des micro services .NET.

4.3.2.2. Validation de la qualité du code

La première étape du pipeline CI sécurise le code source avant toute construction d'artefact. Dans une architecture microservices, ce contrôle précoce est indispensable pour éviter la dérive des standards de code et l'accumulation de dette technique. Comme l'illustre la figure 44, la validation s'articule selon la technologie du service :

- **Services .NET** : L'outil *dotnet format --verify-no-changes* effectue une analyse statique non destructive ; le pipeline échoue en cas de non-conformité. Les tests unitaires et d'intégration sont ensuite exécutés via *dotnet test*, générant des rapports de couverture (XPlat Code Coverage) et des fichiers de résultats *TRX* archivés.
- **Service intelligent (FastAPI)** : La validation repose sur *ruff*, un linter et formateur de dernière génération, plus rapide que *flake8* ou *pylint*. Les tests sont orchestrés par *pytest* avec une couverture exportée en *XML*.

Cette rigueur initiale constitue la première ligne de défense de la plateforme. Dans un écosystème distribué où les déploiements sont indépendants, elle garantit le respect des contrats d'interface et empêche qu'un microservice défaillant ne compromette l'ensemble du système.

```

25 jobs:
26   code-quality:
27     name: Code Quality & Security Scan
28     runs-on: ubuntu-latest
29     steps:
30       - name: Checkout source code
31         uses: actions/checkout@v4
32         with:
33           fetch-depth: 0
34       - name: Setup .NET
35         uses: actions/setup-dotnet@v4
36         with:
37           dotnet-version: ${{ env.DOTNET_VERSION }}
38       - name: Restore dependencies
39         working-directory: ${{ inputs.service-path }}
40         run: dotnet restore ${{ inputs.solution-file }}
41       - name: Build verification
42         working-directory: ${{ inputs.service-path }}
43         run: |
44           dotnet build \
45             ${{ inputs.solution-file }} \
46             --configuration Release \
47             --no-restore
48       - name: Run unit tests
49         working-directory: ${{ inputs.service-path }}
50         run: |
51           dotnet test \
52             ${{ inputs.solution-file }} \
53             --configuration Release \
54             --no-build \
55             --verbosity normal \
56             --logger "trx;LogFileName=results.trx"
57       - name: Upload test results
58         if: always()
59         uses: actions/upload-artifact@v4
60         with:
61           name: test-results-{{ inputs.service-name }}
62           path: |
63             ${{ inputs.service-path }}/**/*.trx
64           retention-days: 30
65

```

FIGURE 44. Pipeline de validation qualité de code .NET

4.3.2.3. Construction et publication des images Docker

La construction des images Docker constitue le coeur technique du pipeline CI. Elle est réalisée via `docker/build-push-action`, action officielle de la communauté GitHub, qui s'appuie sur Docker Buildx pour produire des images multi-architectures ciblant simultanément les plateformes linux/amd64 et linux/arm64. Cette capacité multi-architecture est déterminante pour la portabilité des déploiements, permettant notamment d'exécuter les services sur des serveurs équipés de processeurs ARM sans modification du Dockerfile. Le code *YAML* de cette étape est

bien présent dans la figure 45.

Le mécanisme de cache GitHub Actions (cache-from/cache-to de type gha) est activé pour chaque build, réduisant significativement les durées de construction en réutilisant les couches Docker inchangées entre les exécutions successives du pipeline. Dans un contexte où les images .NET ou Python peuvent atteindre plusieurs centaines de mégaoctets, cette optimisation peut réduire les temps de build de 60 à 80%.

Chaque image produite reçoit deux tags distincts : un tag de version précis, permettant l'identification univoque de l'artefact, et le tag conventionnel latest, maintenu à jour pour les usages ponctuels. La stratégie de versioning est détaillée dans la section suivante.

```

107 - name: Login to GHCR
108   uses: docker/login-action@v3
109   with:
110     registry: ghcr.io
111     username: ${ github.actor }
112     password: ${ secrets.GHCR_TOKEN }
113 - name: Build & push Docker image
114   uses: docker/build-push-action@v6
115   with:
116     context: ${ inputs.service-path }
117     file: ${ inputs.service-path }/Dockerfile
118     platforms: linux/amd64,linux/arm64
119     push: true
120     tags: |
121       ghcr.io/zoubairmabrouk/${ inputs.service-name }:${ steps.version.outputs.version }
122       ghcr.io/zoubairmabrouk/${ inputs.service-name }:latest
123     cache-from: type=gha
124     cache-to: type=gha,mode=max

```

FIGURE 45. Pipeline de build et push Docker multi-architecture avec cache optimisé via GitHub Actions.

4.3.2.4. Stratégie de versioning

La gouvernance du déploiement est renforcée via les *GitHub Environments* : l'environnement production est configuré avec une règle d'approbation requérant la validation d'au moins un *reviewer* désigné avant l'exécution du job de déploiement. Ce mécanisme s'active en déclarant `environment: production` dans le job CD, ce qui interrompt le pipeline et notifie les approbateurs via l'interface GitHub. Cette fonctionnalité est particulièrement utile pour les releases formelles ou les déploiements en fenêtre de maintenance planifiée.

La stratégie retenue repose sur un mécanisme de résolution à deux niveaux. Lorsqu'un tag Git sémantique (par exemple `v1.4.2`) est associé au commit déclenché, la version correspondante est extraite et nettoyée du préfixe `v` pour constituer le tag de l'image (`1.4.2`). En l'absence de tag — cas habituel lors des intégrations continues quotidiennes — le SHA court du commit (sept caractères hexadécimaux) est utilisé comme identifiant, garantissant l'unicité absolue de chaque image produite. La résolution s'effectue via `git describe -tags -exact-match`, dont la sortie conditionne la variable `VERSION` injectée dans les étapes suivantes du pipeline (figure 46).

Cette convention assure une traçabilité bidirectionnelle : à partir du tag d'une image en production, il est toujours possible de retrouver le commit Git exact dont elle est issue, et inversement.

```

92 - name: Resolve version
93   id: version
94   run: |
95     if git describe --tags --exact-match >/dev/null 2>&1; then
96       VERSION=$(git describe --tags --exact-match | sed 's/^v//')
97     else
98       VERSION=$(git rev-parse --short HEAD)
99     fi
100
101     echo "version=$VERSION" >> $GITHUB_OUTPUT
102     echo "Resolved version: $VERSION"
103

```

FIGURE 46. Mécanisme de résolution de version : tag Git sémantique ou SHA court du commit

4.3.3. Gestion Centralisée des Versions

La mise en place d'une source de vérité unique pour les versions des services déployés en production constitue l'un des principes directeurs de l'architecture DevOps de la plateforme. Ce principe, directement inspiré des pratiques GitOps popularisées par des outils tels qu'ArgoCD ou Flux, stipule que l'état de l'infrastructure de production doit être intégralement décrit dans un repository Git, de sorte que tout changement d'état soit tracé, auditable et réversible.

Dans la plateforme décrite, ce rôle est dévolu au fichier `.env.prod` du repository `infra-deploy`, dont la structure minimaliste concentre les variables de version correspondant aux micro services de la plateforme présent à la figure 47.

```

2  GATEWAY_VERSION=b4afe72
3  AUTH_VERSION=b4afe72
4  MDX_VERSION=b4afe72
5  AI_VERSION=latest
6  CONV_VERSION=b4afe72
7  FRONTEND_VERSION=latest

```

FIGURE 47. Fichier `.env.prod` centralisant les versions des micro services dans une approche GitOps

Chaque pipeline CI, après avoir construit et publié avec succès l'image Docker de son service, exécute une séquence d'opérations sur le repository `infra-deploy` : clonage du repository

avec un jeton d'accès dédié (DEPLOY_REPO_TOKEN), modification ciblée de la variable correspondante via la commande `sed`, et commit automatique signé par l'identité du bot CI (`ci-bot@github.com`). La modification est strictement limitée à la variable du service concerné, sans affecter les versions des autres services — propriété fondamentale garantissant l'indépendance des pipelines.

Cette approche présente plusieurs propriétés remarquables. Premièrement, la traçabilité est totale : l'historique Git du fichier `.env.prod` constitue un journal chronologique complet de toutes les mises en production, avec horodatage, service concerné et version déployée. Deuxièmement, le rollback se réduit à une opération Git élémentaire — modifier la variable de version concernée et pousser le commit — déclenchant automatiquement un nouveau déploiement vers l'ancienne version.

Troisièmement, l'état de la production est à tout moment lisible et vérifiable par n'importe quel membre de l'équipe disposant d'un accès au repository, sans nécessiter d'accès direct au serveur de production.

```

143 - name: Update infra-deploy repository
144   run: |
145
146     git clone https://x-access-token:${{ secrets.DEPLOY_REPO_TOKEN }}@github.com/ZoubairMabrouk/infra-deploy.git
147
148     cd infra-deploy
149
150     sed -i "s/^${{ inputs.env-var-name }}=.*/${{ inputs.env-var-name }}=${{ steps.version.outputs.version }}/" .env.prod
151
152     git config user.email "ci-bot@github.com"
153     git config user.name "CI Bot"
154
155     git add .env.prod
156
157     git diff --staged --quiet && echo "No changes detected" && exit 0
158
159     git commit -m "chore(${{ inputs.service-name }}): bump image to ${{ steps.version.outputs.version }} [skip ci]"
160
161     git push

```

FIGURE 48. Mise à jour automatisée du repository d'infrastructure après publication des images Docker

The screenshot shows the GitHub interface for the 'infra-deploy' repository. At the top, the repository name and 'Public' status are visible. Below the repository header, there are buttons for 'Pin', 'Watch', and '0' followers. The main content area shows the repository structure with a 'main' branch, 1 branch, and 0 tags. A search bar and 'Add file' button are present. The commit history table shows a recent commit by 'CI Bot' (rag-service): bump to cbae5c9, which is checked for success. The commit details table lists three files: '.github/workflows' (exposed port by gateway permission, last week), '.env.prod' ((rag-service): bump to cbae5c9, 2 days ago), and 'docker-compose.prod.yml' (exposed port by gateway permissionv1, last week).

File	Commit Message	Time
.github/workflows	exposed port by gateway permission	last week
.env.prod	(rag-service): bump to cbae5c9	2 days ago
docker-compose.prod.yml	exposed port by gateway permissionv1	last week

FIGURE 49. Exemple de commit automatique suit un changement de version d'une image Docker

4.3.4. Déploiement Continu (CD)

4.3.4.1. Déclenchement automatique

Le pipeline de déploiement, hébergé dans le repository infra-deploy, est déclenché automatiquement par tout push sur la branche principale modifiant les fichiers `.env.prod` ou `docker-compose.prod.yml`. Ce déclenchement par contenu plutôt que par calendrier est caractéristique d'une approche GitOps : l'état du repository définit l'état souhaité de l'infrastructure, et tout écart entre les deux déclenche une réconciliation automatique.

En complément du déclenchement automatique, le workflow expose un déclenchement manuel via l'événement `workflow_dispatch`, permettant aux équipes d'exploitation d'initier un redéploiement à la demande — par exemple après une intervention manuelle sur le serveur ou lors d'un retour à l'état nominal après incident. Ce déclenchement manuel peut être soumis à une approbation humaine via les GitHub Environments, mécanisme de gouvernance décrit dans la section 6.

```
4  on:
5    push:
6      branches: [main]
7      paths:
8        - '.env.prod'
9        - 'docker-compose.prod.yml'
10     - '.github/workflows/deploy.yml'
11    workflow_dispatch:
12      inputs:
13        reason:
14          description: 'Raison du déploiement manuel'
15          required: false
16          default: 'manual trigger'
17
```

FIGURE 50. Déclenchement du pipeline de déploiement continu via modification des fichiers d'infrastructure

4.3.4.2. Déploiement sur le VPS

L'opération de déploiement proprement dite s'effectue via une connexion SSH sécurisée au VPS, établie par l'action `appleboy/ssh-action` à l'aide d'une clé SSH dédiée stockée dans les secrets GitHub. Une fois la connexion établie, le script de déploiement exécute une séquence d'opérations précisément ordonnancée, dont chaque étape est conçue pour minimiser la fenêtre d'indisponibilité et maximiser la résilience en cas d'échec.

La première opération consiste à synchroniser la configuration locale du VPS avec l'état du

repository infra-deploy via git pull, garantissant que le fichier .env.prod utilisé pour le déploiement est bien celui qui a déclenché le pipeline, et non une version antérieure potentiellement présente sur le disque. Les images Docker des services dont la version a changé sont ensuite téléchargées depuis GHCR via docker compose pull, opération qui peut s'exécuter sans interruption de service puisque les conteneurs en cours d'exécution ne sont pas encore affectés à ce stade.

Avant d'initier le redémarrage de la stack, une copie de sauvegarde du fichier .env.prod courant est effectuée (.env.prod.bak), constituant le point de restauration pour le rollback automatique. Le redémarrage de la stack s'effectue ensuite via la commande docker compose up -d --remove-orphans --wait --wait-timeout 120, dont les options méritent une attention particulière : --wait force Docker Compose à attendre que tous les conteneurs dotés d'un healthcheck atteignent l'état healthy avant de rendre la main, tandis que --remove-orphans supprime les conteneurs correspondant à des services supprimés de la configuration.

```

39 - name: Deploy via SSH
40   uses: appleboy/ssh-action@v1
41   with:
42     host:      ${ secrets.VPS_HOST }
43     username:  ${ secrets.VPS_USER }
44     key:       ${ secrets.VPS_SSH_KEY }
45     script: |
46       set -e
47       sudo mkdir -p /opt/deploy
48       cd /opt/deploy
49       echo "— Pulling latest infra config —"
50       if [ ! -d ".git" ]; then
51         echo "Repository absent → clone initial"
52         sudo git clone https://github.com/ZoubairMabrouk/infra-deploy.git .
53       else
54         echo "Repository existant → pull"
55         sudo git reset --hard
56         sudo git clean -fd
57         sudo git pull origin main
58       fi
59       echo "— Login GHCR —"
60       echo "${ secrets.GHCR_TOKEN }" | sudo docker login ghcr.io -u "${ github.actor }" --password-stdin
61       echo "— Stopping current stack —"
62       sudo docker compose --env-file .env.prod -f docker-compose.prod.yml down
63       echo "— Deleting old images —"
64       sudo docker image prune -f
65       echo "— Pulling Docker images —"
66       sudo docker compose --env-file .env.prod -f docker-compose.prod.yml pull
67       echo "— Saving current stack state for rollback —"
68       sudo cp .env.prod .env.prod.bak
69       echo "— Starting new stack —"
70       sudo docker compose --env-file .env.prod -f docker-compose.prod.yml \
71       | up -d --remove-orphans --wait --wait-timeout 120
72       echo "— Healthcheck —"
73       sleep 10
74       sudo docker compose --env-file .env.prod -f docker-compose.prod.yml ps --format json \
75       | grep -v "Health": "healthy" | grep "State": "running" \
76       && echo "All containers running" || (
77         echo "❌ Healthcheck failed — rolling back"
78         sudo cp .env.prod.bak .env.prod
79         sudo docker compose --env-file .env.prod -f docker-compose.prod.yml \
80         | up -d --remove-orphans
81         exit 1
82       )
83       echo "— Cleanup old images —"
84       sudo docker image prune -f
85

```

FIGURE 51. Exécution du déploiement sur VPS via SSH

```
47  mdx-service:
48      image: ghcr.io/zoubairmabrouk/mdx-service:${MDX_VERSION}
49      expose:
50      - "8080"
51      environment:
52          ASPNETCORE_ENVIRONMENT: Production
53          ConnectionStrings__DefaultConnection: >
54              Server=sqlserver,1433;Database=MdxDb;
55              User Id=sa;Password=$SQL_SA_PASSWORD;TrustServerCertificate=True
56          Jwt__SecretKey: "${JWT_SECRET_KEY}"
57          Jwt__Issuer: "${JWT_ISSUER}"
58          Jwt__Audience: "${JWT_AUDIENCE}"
59      networks:
60      - microservices-net
61      depends_on:
62      - sqlserver
63      restart: unless-stopped
64      healthcheck:
65          test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
66          interval: 15s
67          timeout: 5s
68          retries: 5
```

FIGURE 52. Configuration de healthcheck Docker et validation de disponibilité de micro service.

4.3.4.3. Healthchecks et validation de disponibilité

La supervision de la disponibilité des conteneurs après déploiement repose sur les mécanismes de healthcheck natifs de Docker, configurés dans le fichier `docker-compose.prod.yml` pour chaque service exposant un endpoint de santé. Pour les services .NET, l'endpoint `/health`, exposé via le middleware `HealthChecks` d'ASP.NET Core, retourne un code HTTP 200 accompagné d'un rapport détaillé de l'état des dépendances du service (connexion à la base de données, disponibilité des services aval). Docker sonde cet endpoint à intervalles réguliers (toutes les 15 secondes), et marque le conteneur comme `unhealthy` après cinq échecs consécutifs. La figure 52 illustre un extrait de cette configuration pour le service `mdx-service`, tous les autres service ont la meme configuration.

L'option `--wait` du déploiement tire parti de ces healthchecks en bloquant la progression du script tant que tous les conteneurs n'ont pas atteint l'état `healthy`, avec un délai d'expiration configurable (120 secondes dans la configuration actuelle). Ce comportement garantit que le déploiement n'est considéré comme réussi qu'une fois la disponibilité effective de tous les services confirmée, et non simplement lorsque leurs processus ont démarré — distinction fondamentale dans un contexte où les délais d'initialisation des services .NET ou de l'ORM Entity Framework peuvent atteindre plusieurs dizaines de secondes.

4.3.4.4. Notification de mis à jour

La visibilité des opérations de déploiement constitue un élément essentiel dans une architecture DevOps automatisée pour garantir la transparence et la réactivité des équipes. Afin d'améliorer le suivi des mises en production, la plateforme intègre un mécanisme de notification automatique basé sur l'API Telegram.

Ce mécanisme est déclenché à la fin du pipeline de déploiement, en fonction du statut d'exécution du job principal (deploy). Deux scénarios sont distingués : un déploiement réussi et un déploiement échoué, chacun déclenchant un workflow indépendant de notification comme il est montré la code dans la figure 53.

En cas de succès, une notification est envoyée via un appel HTTP POST vers l'API Telegram, contenant des informations clés telles que l'auteur du commit, le hash du commit, ainsi qu'un lien direct vers l'exécution du workflow GitHub Actions.

```

86 notify-success:
87   name: Telegram – Succès
88   runs-on: ubuntu-latest
89   needs: [deploy]
90   if: success()
91   steps:
92     - name: 🟢 Notification succès
93       run: |
94         curl -s -X POST \
95           "https://api.telegram.org/bot${{ secrets.TELEGRAM_TOKEN }}/sendMessage" \
96           -H "Content-Type: application/json" \
97           -d '{
98             "chat_id": "${{ secrets.TELEGRAM_CHAT_ID }}",
99             "text": " *APP Déployé*\n\n *Service* : TOUS\n 🍷 *Version* : env.AUTH_VERSION\n *Auteur* : ${{ github.actor }}\n 📄 *SHA*
100             "parse_mode": "Markdown"
101           }'
102
103 notify-failure:
104   name: 🚫 Telegram – Échec
105   runs-on: ubuntu-latest
106   needs: [deploy]
107   if: failure()
108   steps:
109     - name: 🚫 Notification échec
110       run: |
111         curl -s -X POST \
112           "https://api.telegram.org/bot${{ secrets.TELEGRAM_TOKEN }}/sendMessage" \
113           -H "Content-Type: application/json" \
114           -d '{
115             "chat_id": "${{ secrets.TELEGRAM_CHAT_ID }}",
116             "text": " 🚫 *APP – Échec Déploiement*\n\n 🍷 *Auteur* : ${{ github.actor }}\n 📄 *SHA* : `${GITHUB_SHA:0:8}`\n 🔄 *workflow
117             "parse_mode": "Markdown"

```

FIGURE 53. Implémentation du message Telegram dans GitHub Actions.

Le script utilise une requête HTTP structurée permettant d'envoyer un message formaté en Markdown, facilitant la lisibilité des informations critiques relatives au déploiement.

En cas d'échec du pipeline de déploiement, un second workflow est déclenché afin d'alerter immédiatement les équipes techniques. Ce mécanisme permet une réaction rapide en cas d'incident en production, réduisant ainsi le temps moyen de détection (MTTD).

Ces notifications, comme celle dans la figure 54, jouent un rôle clé dans la supervision opérationnelle de la plateforme, en assurant un retour immédiat sur l'état des déploiements sans nécessiter de consultation manuelle des logs GitHub Actions. Elles s'intègrent ainsi dans une

logique de *continuous feedback loop*, essentielle dans les architectures micro services modernes.

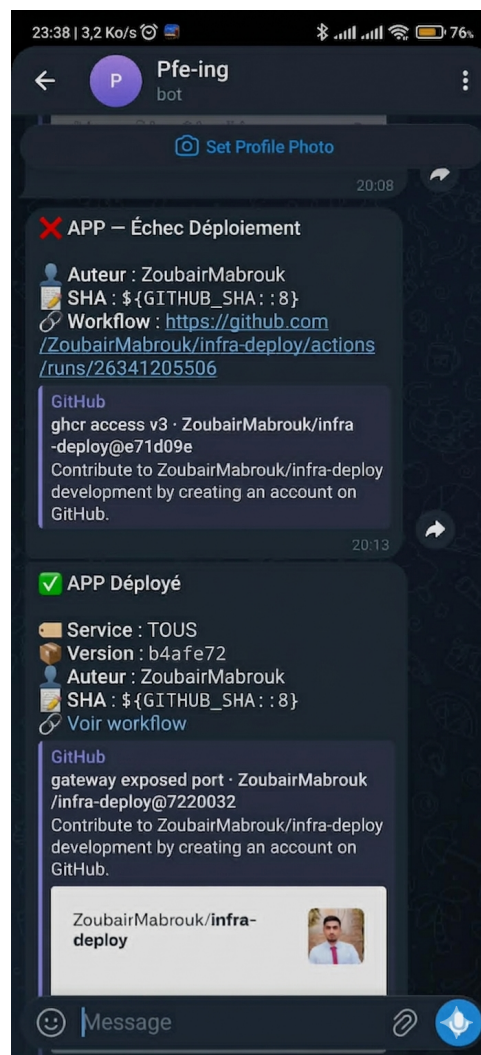


FIGURE 54. Exemple de message T  l  gram avec les deux cas : succ  es et   chec

4.3.5. S  curit   DevOps

La s  curit   de la cha  ne CI/CD est une dimension souvent n  glig  e au profit des aspects fonctionnels, alors qu'elle repr  sente un vecteur d'attaque de premier ordre. Une cha  ne CI/CD compromise peut permettre    un attaquant d'injecter du code malveillant dans des artefacts d  ploy  s en production, d'exfiltrer des secrets d'infrastructure ou de prendre le contr  le d'un serveur de production. Le dispositif mis en place int  gre plusieurs niveaux de protection compl  mentaires.

La gestion des secrets s'appuie int  gralement sur le m  canisme des **GitHub Secrets**, qui chiffre les valeurs sensibles au repos et les injecte dans les workflows comme variables d'environnement   ph  m  res, sans jamais les exposer dans les logs ou les artefacts. Deux cat  gories de secrets sont d  finies : les secrets du repository micro services (GHCR_TOKEN pour l'authentification au

GHCR, `DEPLOY_REPO_TOKEN` pour la mise à jour automatique d'infra-deploy) et les secrets du repository infra-deploy (`VPS_HOST`, `VPS_USER`, `VPS_SSH_KEY` pour la connexion SSH au serveur de production). Ces secrets sont illustrées successivement dans les figures [55, 56]

La séparation entre configuration versionnée et secrets d'exécution est rigoureusement respectée. Le fichier `.env.prod`, commité dans infra-deploy, ne contient que des variables non sensibles (versions d'images, noms d'organisation de registry, paramètres applicatifs). Les secrets de production — `JWT_SECRET_KEY` utilisée pour la signature des tokens d'authentification, et `SQL_SA_PASSWORD` pour l'accès à la base de données — sont stockés dans un fichier `.env.secrets` maintenu exclusivement sur le VPS, dans un répertoire à accès restreint, et injecté comme second fichier d'environnement lors de l'exécution de Docker Compose. Cette architecture garantit qu'aucun secret ne transite jamais par un système de contrôle de versions.

Pour les environnements nécessitant une gouvernance renforcée, GitHub Environments offre un mécanisme d'approbation humaine préalable au déploiement en production. Configuré sur le repository infra-deploy, ce mécanisme interrompt l'exécution du pipeline à l'étape de déploiement et notifie les reviewers désignés, qui doivent approuver explicitement l'opération avant sa poursuite. Cette fonctionnalité, particulièrement pertinente pour les releases formelles ou les déploiements en fenêtres de maintenance planifiées, complète le dispositif d'automatisation sans en compromettre le contrôle.

Sur le VPS, le principe du moindre privilège est appliqué à travers l'utilisation d'un utilisateur Linux dédié (`deploy`), dont les permissions sont strictement limitées aux opérations nécessaires à Docker Compose. L'accès root direct est désactivé pour les connexions SSH, et l'authentification par clé est imposée en lieu et place de l'authentification par mot de passe.

Repository secrets

[New repository secret](#)

Name 	Last updated	
 <code>DEPLOY_REPO_TOKEN</code>	3 hours ago	 
 <code>GHCR_TOKEN</code>	yesterday	 

FIGURE 55. Les secrets sensibles des micro services dans GitHub Secrets

Repository secrets New repository secret

Name 	Last updated	
 DEPLOY_PAT_TOKEN	2 hours ago	 
 GHCR_TOKEN	yesterday	 
 JWT_SECRET_KEY	now	 
 SQL_SA_PASSWORD	now	 
 TELEGRAM_CHAT_ID	3 hours ago	 
 TELEGRAM_TOKEN	3 hours ago	 
 VPS_HOST	3 hours ago	 
 VPS_SSH_KEY	3 hours ago	 
 VPS_USER	3 hours ago	 

FIGURE 56. Les secrets de l'infrastructure de déploiement dans GitHub Secrets

4.3.6. Monitoring et Supervision avec Portainer

La mise en place d'un dispositif de monitoring constitue le complément naturel de toute architecture de déploiement automatisée. Si le pipeline CI/CD garantit la cohérence et la reproductibilité des mises en production, la supervision en temps réel des conteneurs en cours d'exécution relève d'une problématique distincte : celle de la visibilité opérationnelle. C'est précisément le rôle dévolu à Portainer, interface de gestion graphique pour environnements Docker et Docker Compose, déployée sur le même VPS que les micro services de la plateforme.

Portainer se distingue des solutions de monitoring orientées métriques par sa vocation première d'interface d'administration des conteneurs. Son déploiement sous forme de conteneur Docker autonome s'intègre naturellement dans l'architecture existante, sans nécessiter de dépendances externes ni de modifications de la configuration Docker Compose des micro services.

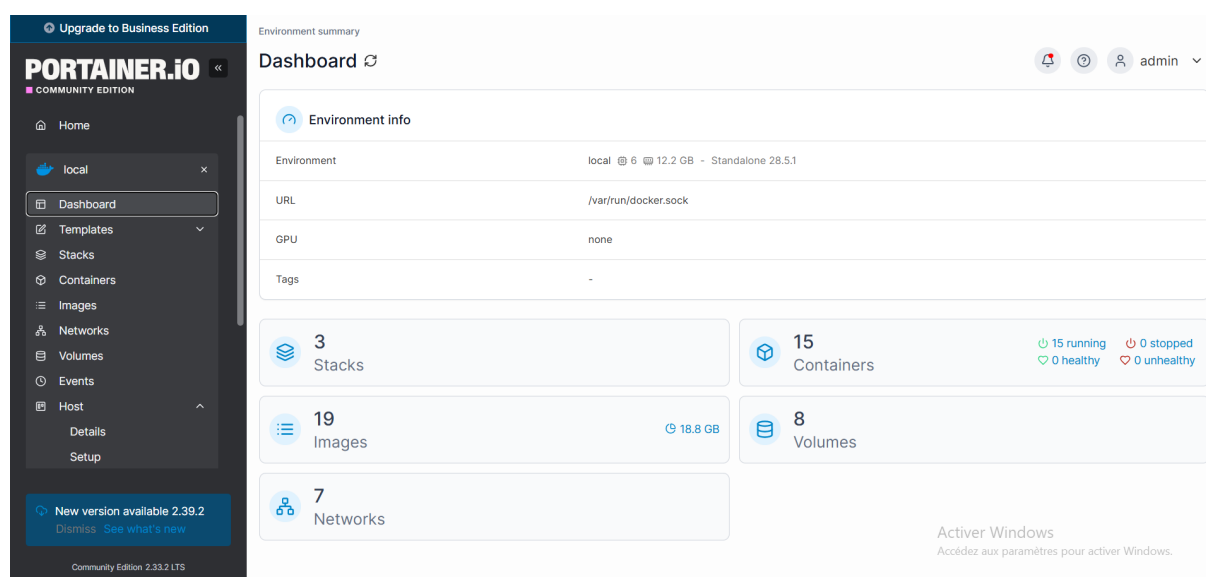


FIGURE 57. Vue d'ensemble du tableau de bord Portainer sur l'environnement de production

4.3.6.1. Supervision des conteneurs en temps réel

L'interface de Portainer offre une vue consolidée de l'état de l'ensemble des conteneurs en exécution sur le VPS, affichant pour chacun d'eux le statut courant (running, exited, unhealthy), l'image Docker utilisée avec son tag de version précis, l'uptime, ainsi que les ports exposés. Cette visibilité immédiate constitue une première ligne de supervision, permettant de détecter en un coup d'œil toute anomalie dans l'état de la stack — qu'il s'agisse d'un conteneur en état unhealthy, d'un redémarrage inopiné, ou d'une image dont la version ne correspond pas à celle attendue dans le fichier `.env.prod`.

La corrélation entre le tag d'image affiché dans Portainer et la version consignée dans *infra-deploy* constitue un mécanisme de vérification implicite de la cohérence de l'état de production : si les deux valeurs concordent, le déploiement a bien été appliqué conformément à la configuration GitOps.

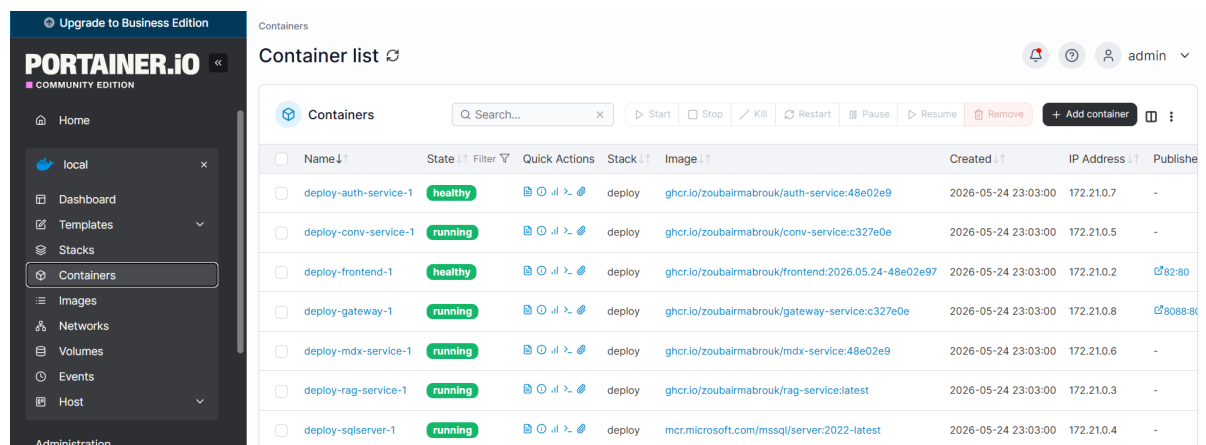


FIGURE 58. Liste des conteneurs en cours d'exécution avec leur statut et version d'image respective

4.3.6.2. Consultation des logs et diagnostics

L'une des fonctionnalités les plus déterminantes de Portainer dans un contexte opérationnel est l'accès direct aux logs de chaque conteneur, sans nécessiter de connexion SSH au serveur de production. Cette capacité réduit considérablement le temps de diagnostic lors d'un incident : un opérateur peut consulter les logs applicatifs en temps réel depuis son navigateur, filtrer les entrées par niveau de sévérité, et identifier la cause d'une anomalie sans manipuler directement le serveur.

Dans une architecture micro services où chaque service produit ses propres flux de logs, cette centralisation visuelle est d'autant plus précieuse que la corrélation manuelle des journaux inter-services peut rapidement devenir laborieuse. Portainer ne se substitue pas à une solution de centralisation des logs telle qu'un stack ELK (Elasticsearch, Logstash, Kibana), mais offre un accès opérationnel suffisant pour la majorité des interventions courantes.

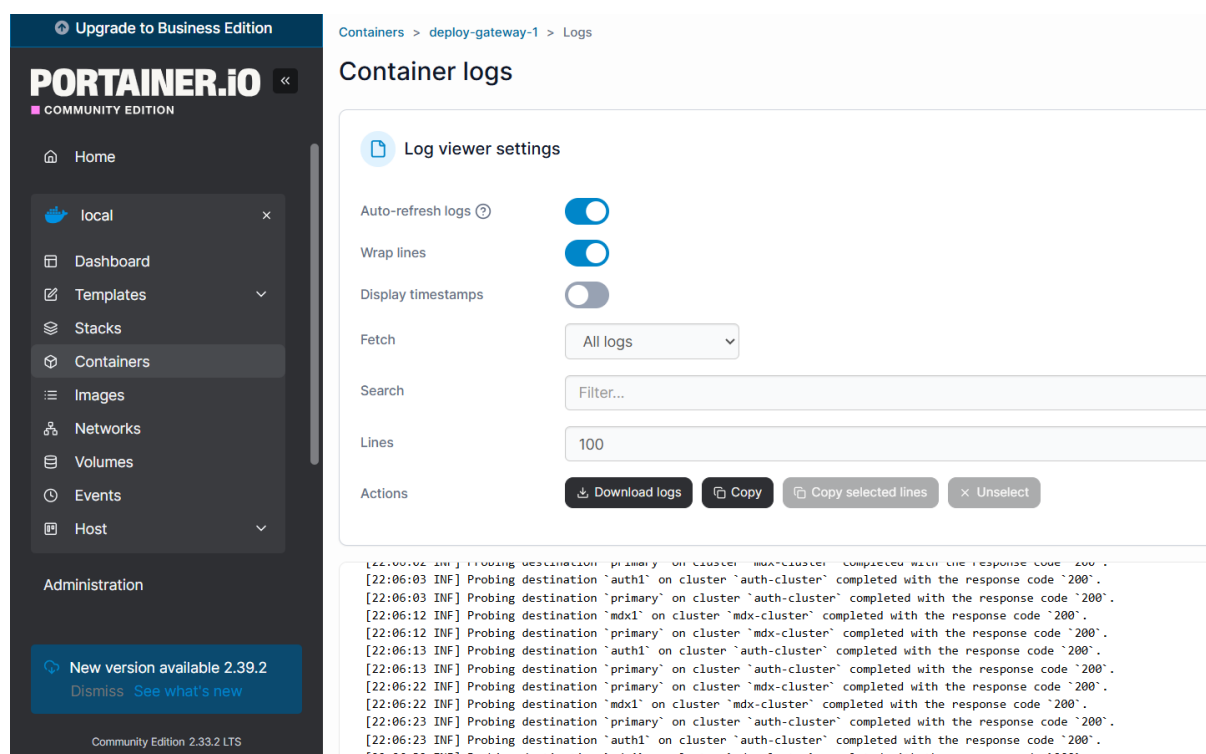


FIGURE 59. Console de logs en temps réel d'un micro service depuis l'interface Portainer

4.3.6.3. Surveillance des ressources système

Portainer expose également des métriques de consommation des ressources pour chaque conteneur actif : utilisation CPU, consommation mémoire, activité réseau en entrée et sortie, ainsi que les opérations d'entrée/sortie disque. Ces indicateurs, collectés nativement par le démon Docker via l'API `docker stats`, sont présentés sous forme de graphiques temporels permettant d'identifier visuellement les tendances de charge, les pics de consommation, et les éventuelles

fuites mémoire.

Dans un contexte où l'ensemble des micro services cohabitent sur un VPS aux ressources mutualisées, cette surveillance est essentielle pour anticiper les problèmes de saturation et dimensionner correctement les ressources allouées à chaque service. Un service dont la consommation mémoire croît continûment entre deux redémarrages est un signal d'alerte que cette interface rend immédiatement perceptible.

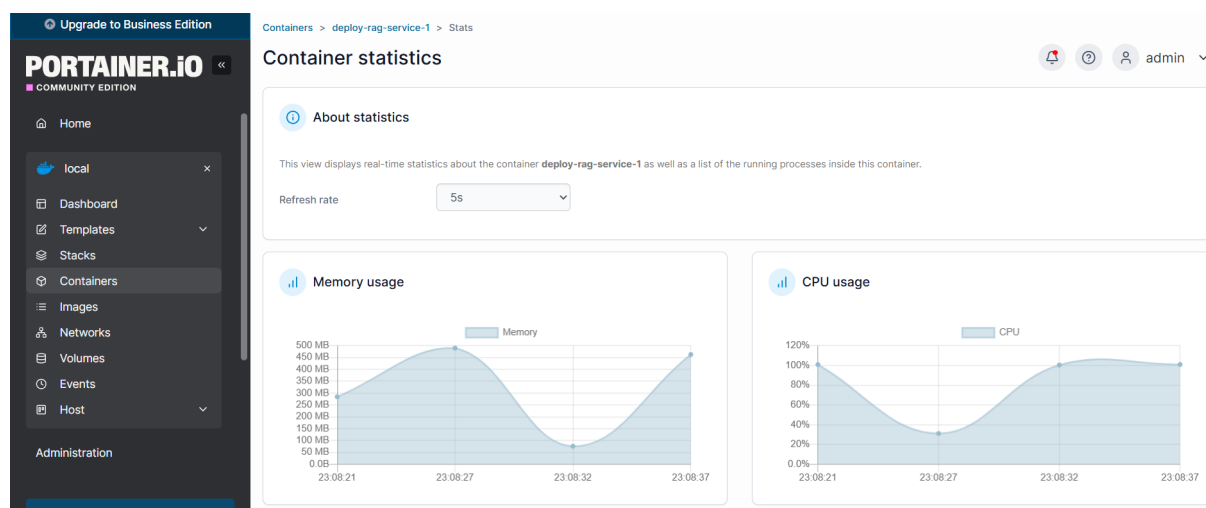


FIGURE 60. Métriques de consommation CPU et mémoire par conteneur dans Portainer - PARTIE 1



FIGURE 61. Métriques de consommation CPU et mémoire par conteneur dans Portainer PARTIE 2

4.3.6.4. Gestion des volumes et de la persistance

La supervision des volumes Docker constitue un aspect souvent négligé de la gestion opérationnelle, alors que leur intégrité conditionne directement la durabilité des données applicatives. Portainer offre une interface de gestion des volumes permettant de vérifier leur état,

d'identifier les volumes orphelins non rattachés à un conteneur actif, et de consulter les chemins de montage associés.

Dans la configuration de déploiement de la plateforme, le volume dédié à Microsoft SQL Server 2022 est particulièrement critique : sa corruption ou sa suppression accidentelle entraînerait une perte irréversible des données de production. La visibilité offerte par Portainer sur cet aspect renforce la sécurité opérationnelle du dispositif.

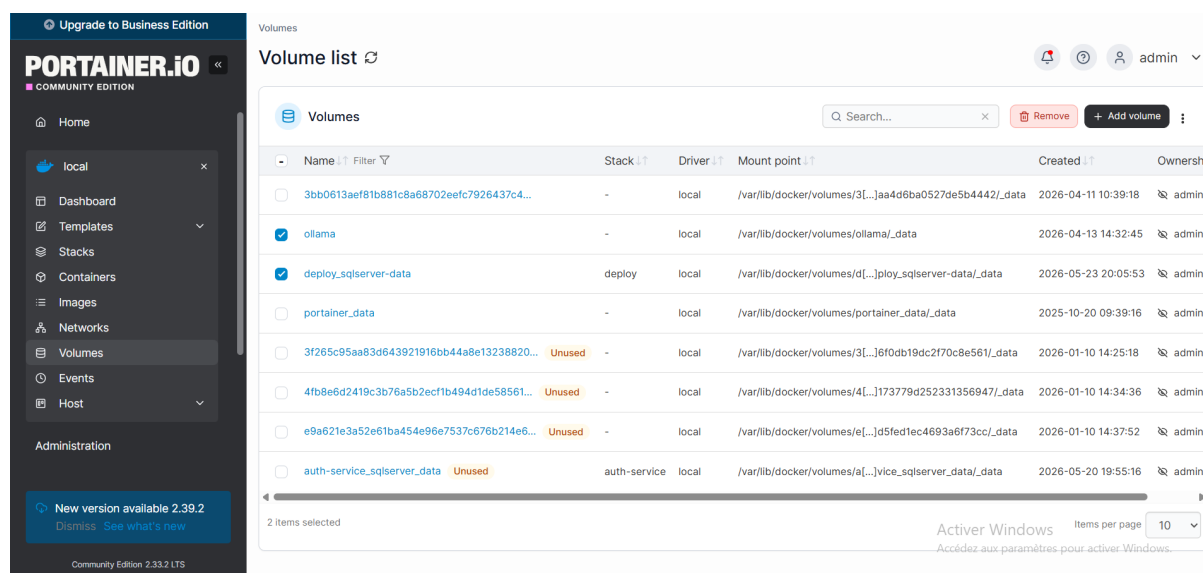


FIGURE 62. Interface de gestion des volumes Docker et supervision de la persistance des données

4.3.6.5. Gestion et inspection des réseaux Docker

La supervision des réseaux Docker constitue un aspect complémentaire à la gestion des conteneurs et des volumes, permettant de vérifier la topologie de communication inter-services au sein de la stack déployée. Portainer expose une interface dédiée à l'inspection des réseaux Docker, listant l'ensemble des réseaux définis dans la configuration Docker Compose ainsi que leurs caractéristiques : driver utilisé (bridge, overlay, host), plage d'adresses IP allouée (CIDR), passerelle par défaut, et liste des conteneurs rattachés à chaque réseau.

Cette visibilité est particulièrement utile dans une architecture micro services où plusieurs réseaux peuvent coexister afin d'isoler les flux de communication : un réseau interne réservé aux échanges inter-services, et un réseau exposé pour les conteneurs accessibles depuis le reverse proxy. L'interface Portainer permet de vérifier, sans recourir à la ligne de commande, qu'un conteneur est bien rattaché aux réseaux attendus et qu'aucun service n'est exposé sur un réseau non prévu.

La consultation des détails d'un réseau révèle également les adresses IP assignées à chaque conteneur membre, ce qui facilite le diagnostic de problèmes de connectivité entre services — notamment lorsqu'un service ne parvient pas à joindre sa dépendance par son nom DNS interne.

Dans un contexte où les noms de services Docker Compose servent de résolution DNS interne, cette transparence sur la configuration réseau constitue un levier de diagnostic non négligeable.

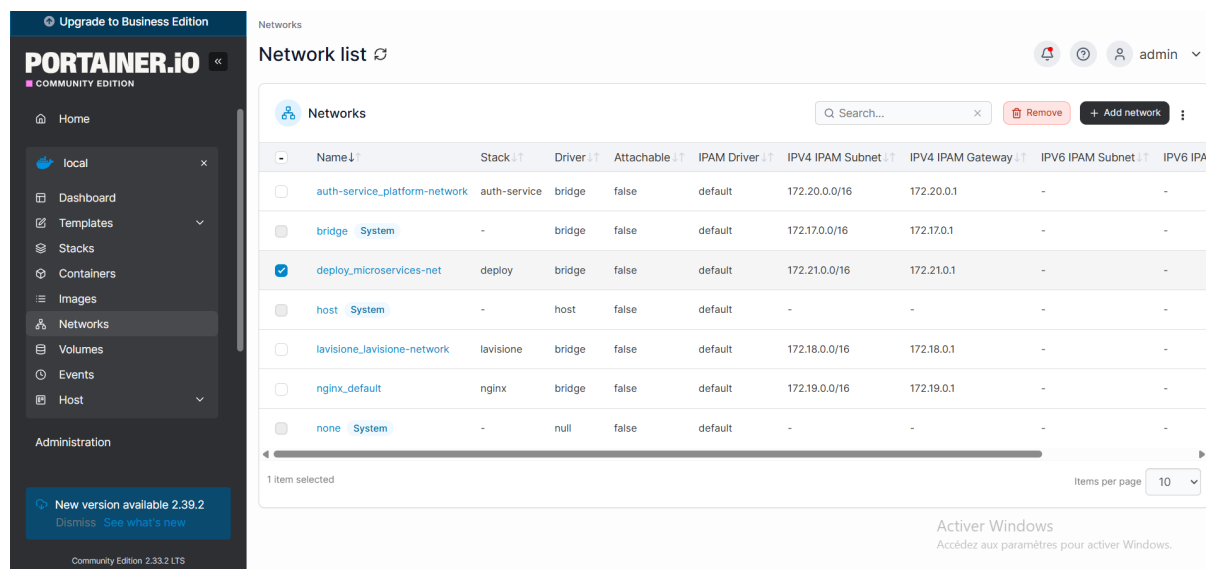


FIGURE 63. Interface de gestion et d'inspection des réseaux Docker dans Portainer

4.4. Conclusion

Cette seconde release concrétise la transition d'un prototype analytique vers une plateforme distribuée opérationnelle. Sur le plan architectural, quatre micro services indépendants — authentification, système d'audit de conversation asynchrone et API Gateway — remplacent le composant monolithique initial, avec une séparation claire des responsabilités et des interfaces contractuelles REST documentées.

Sur le plan DevOps, le pipeline CI/CD réduit le délai de mise en production d'une modification à moins de cinq minutes, de la détection du push Git au redémarrage du conteneur sur le VPS, avec rollback automatique en cas d'échec des healthchecks. La gestion GitOps des versions via infra-deploy offre une traçabilité complète et rend le rollback manuel réalisable en moins de deux minutes.

Ces orientations constituent la feuille de route naturelle d'une troisième release, qui pourrait également intégrer des capacités d'analyse prédictive et d'apprentissage automatique sur les données décisionnelles de la plateforme.

Chapitre 5

CONCLUSION GÉNÉRALE

Le projet de fin d'études présenté dans ce mémoire a porté sur la conception et l'implémentation d'une plateforme décisionnelle intelligente, intégrant un entrepôt de données multidimensionnel, des cubes OLAP, une architecture microservices et un agent conversationnel analytique basé sur un modèle de langage local. Ce travail a été réalisé au sein de la société EDI-Solutions et répond à une problématique récurrente dans les systèmes décisionnels traditionnels : la difficulté d'accès aux données analytiques pour les utilisateurs métiers, la complexité des requêtes MDX, ainsi que l'absence d'outils intelligents d'assistance à la décision.

La solution proposée repose sur une chaîne complète allant de l'intégration des données brutes jusqu'à l'interrogation en langage naturel. Le Sprint 1 a permis de construire un Data Warehouse en schéma en étoile alimenté par des processus ETL sous SSIS, puis un cube OLAP multidimensionnel sous SQL Server Analysis Services (SSAS). Un microservice MDX dédié expose les indicateurs clés via une API REST, garantissant la séparation entre les traitements transactionnels et analytiques.

Le Sprint 2 a introduit la couche intelligente via un pipeline RAG (Retrieval-Augmented Generation). Les métadonnées du cube OLAP sont vectorisées à l'aide du modèle all-MiniLM-L6-v2, indexées dans FAISS et persistées dans ChromaDB. Un modèle de langage Phi-3 Mini (3,8 milliards de paramètres), exécuté localement via Ollama, génère dynamiquement des requêtes MDX à partir d'une question en langage naturel. L'évaluation sur un corpus de 30 requêtes tests montre un taux de validité syntaxique MDX de 82% et un temps de réponse moyen de 6,1 secondes avec le paramètre $k=5$, constituant un compromis acceptable pour une infrastructure CPU contrainte.

Les Sprint 3 et Sprint 4 ont industrialisé la plateforme. L'architecture monolithique initiale a été transformée en quatre microservices indépendants (authentification JWT, exécution MDX, historique conversationnel asynchrone via RabbitMQ, et API Gateway YARP). Une chaîne CI/CD complète, basée sur GitHub Actions, automatise la construction, les tests, la conteneurisation Docker et le déploiement sur un VPS selon une approche GitOps. Les versions des services sont centralisées dans un dépôt d'infrastructure distinct, permettant une traçabilité totale et un rollback automatisé en moins de deux minutes. La supervision opérationnelle est assurée par Portainer, offrant une visibilité en temps réel sur l'état des conteneurs, les logs et la consommation des ressources.

Cette plateforme se distingue par plusieurs apports significatifs. D'une part, elle démocratise l'accès à l'analyse multidimensionnelle en permettant à des utilisateurs non techniques d'interroger le système en langage naturel, sans maîtriser le langage MDX. D'autre part, l'intégration d'un LLM open source local garantit la confidentialité des données décisionnelles et réduit la dépendance aux services cloud externes. Enfin, l'industrialisation complète du cycle de vie — de l'intégration

continue à la supervision — constitue une base solide pour un passage en production.

Malgré ces avancées, plusieurs limites ont été identifiées au cours du développement. Le système RAG actuel ne conserve aucun historique de session ; chaque requête est traitée de manière stateless, ce qui empêche le raffinement itératif des analyses. L'orchestration repose sur Docker Compose sur un seul VPS, ce qui limite la scalabilité horizontale et expose à un risque de disponibilité. La chaîne CI/CD ne comporte pas encore de tests d'intégration entre microservices, ce qui peut masquer des régressions lors de déploiements indépendants. Enfin, la supervision par Portainer reste réactive, sans mécanisme d'alerte proactive basé sur des seuils métriques.

Ces limites ouvrent plusieurs axes d'amélioration pour des travaux futurs. Une migration de l'orchestration vers Kubernetes apporterait la scalabilité automatique, la tolérance aux pannes au niveau nœud et une gestion fine des ressources par service. L'intégration d'une stack Prometheus/Grafana remplacerait la supervision manuelle par des alertes proactives configurables. L'ajout de tests de contrat entre services (consumer-driven contract testing) dans le pipeline CI renforcerait la fiabilité des déploiements indépendants.

Sur le plan de l'intelligence artificielle, l'évaluation de modèles de plus grande capacité (Mistral 7B, LLaMA 3 quantisé) pourrait améliorer la précision sémantique pour les requêtes analytiques complexes, sous réserve d'un renforcement des ressources matérielles. Enfin, l'intégration de capacités d'analyse prédictive et d'apprentissage automatique sur les données décisionnelles — par exemple pour la détection d'anomalies ou la prévision des ventes — constituerait une extension naturelle vers une plateforme d'aide à la décision véritablement proactive.

Ce projet a démontré la faisabilité et la pertinence d'une approche combinant les entrepôts de données multidimensionnels, les cubes OLAP et les modèles de langage dans une architecture microservices industrialisée. La plateforme résultante permet à des utilisateurs métiers d'interroger en langage naturel un système décisionnel complexe, tout en offrant aux équipes techniques une chaîne de déploiement fiable, traçable et supervisée. Les résultats obtenus — 82% de validité syntaxique MDX et un temps de réponse moyen de 6,1 secondes — valident le choix technique et ouvrent la voie à un enrichissement continu des fonctionnalités analytiques et intelligentes.

ANNEXE

.1. Liste des Code HTTP

TABLE 20. Principaux codes HTTP utilisés dans la plateforme

Code HTTP	Signification	Description dans le contexte du projet
200 OK	Requête réussie	La requête a été traitée avec succès.
201 Created	Ressource créée	Utilisé lorsqu'une nouvelle ressource est créée avec succès.
400 Bad Request	Requête invalide	La requête envoyée par le client contient des paramètres invalides.
401 Unauthorized	Non authentifié	Le client n'a pas fourni de token JWT valide ou n'est pas authentifié auprès du système.
403 Forbidden	Accès interdit	L'utilisateur est authentifié mais ne possède pas les permissions nécessaires pour accéder à la ressource demandée.
404 Not Found	Ressource introuvable	Le service ou la ressource demandée n'existe pas.
409 Conflict	Conflit de requête	indique que la requête ne peut pas être traitée en raison d'un conflit avec l'état actuel de la ressource.
422 Unprocessable Entity	Entité non traitable	indique que le serveur comprend le type de contenu et la syntaxe de la requête, mais ne peut pas la traiter à cause d'erreurs sémantiques.
500 Internal Server Error	Erreur interne du serveur	Une erreur inattendue s'est produite lors du traitement de logic métier.
502 Bad Gateway	Mauvaise passerelle	L'API Gateway a reçu une réponse invalide d'un service interne.
503 Service Unavailable	Service indisponible	Le service demandé est temporairement indisponible, surchargé ou en maintenance.

.2. Catalogue des Endpoints des Microservices

.2.1. Auth Service (*auth-service*)

TABLE 21. Endpoints exposés par le microservice d'authentification et de gestion des utilisateurs.

Méthode	Endpoint
POST	/api/auth/register
POST	/api/auth/login
POST	/api/auth/refresh
GET	/api/auth/me-placeholder
GET	/api/user/me
GET	/api/user
GET	/api/profile
GET	/api/users
GET	/api/users/{ id }
POST	/api/users
PUT	/api/users/{ id }
DELETE	/api/users/{ id }
PATCH	/api/users/{ id }/status
GET	/health

.2.2. Conversation History Service (*conversation-history-service*)

TABLE 22. Endpoints exposés par le microservice de gestion de l'historique des conversations.

Méthode	Endpoint
GET	/api/conversations
GET	/api/conversations/{ id }
POST	/api/conversations
POST	/api/conversations/{ id }/messages
DELETE	/api/conversations/{ id }
GET	/health

.2.3. MDX Service (*mdx-service*)

TABLE 23. Endpoints analytiques exposés par le microservice MDX pour l'interrogation du cube OLAP.

Méthode	Endpoint
GET	/api/revenue/by-year
GET	/api/revenue/by-month/{year}
GET	/api/revenue/by-quarter-doctype
GET	/api/revenue/by-client
GET	/api/revenue/top-clients
GET	/api/revenue/bottom-clients
GET	/api/revenue/concentration
GET	/api/clients/above-average
GET	/api/clients/outstanding-balance
GET	/api/clients/high-discount-low-margin
GET	/api/clients/by-margin
GET	/api/clients/avg-order-value
GET	/api/discounts/ratio-by-year
GET	/api/discounts/top-clients
GET	/api/discounts/by-document-type
GET	/api/documents/volume-by-type-quarter
GET	/api/tax/tva-by-month/{year}
GET	/api/tax/fodec-by-doctype
GET	/api/tax/burden-by-quarter
GET	/api/trends/yoy-revenue/{currentYear}/{priorYear}
GET	/api/trends/yoy-volume/{currentYear}/{priorYear}
GET	/api/trends/ytd/{year}
GET	/api/trends/qtd/{year}
GET	/api/trends/margin/{year}
POST	/api/llm
GET	/WeatherForecast

.2.4. RAG Service (*rag-service*)

TABLE 24. Endpoints exposés par le microservice RAG basé sur FastAPI pour l'interrogation intelligente des données analytiques.

Méthode	Endpoint
POST	/api/ask
POST	/api/add-external-file

.2.5. API Gateway (*api-gateway*)

TABLE 25. Routes exposées par l'API Gateway pour le routage centralisé des requêtes vers les microservices internes.

Méthode	Endpoint exposé
Toutes	/api/auth/{ catch-all }
Toutes	/api/user/{ catch-all }
Toutes	/api/users/{ catch-all }
Toutes	/api/clients/{ catch-all }
POST	/ask
Toutes	/api/conversation/{ catch-all }
GET	/health

.3. Glossaire des acronymes

TABLE 26. Glossaire des acronymes utilisés dans le mémoire

Acronyme	Signification complète	Acronyme	Signification complète
API	Application Programming Interface	ASP.NET	Active Server Pages .NET
BI	Business Intelligence	CD	Continuous Deployment
CI	Continuous Integration	CRM	Customer Relationship Management
DevOps	Development and Operations	DLQ	Dead Letter Queue
DTO	Data Transfer Object	DW	Data Warehouse

Suite à la page suivante

Acronyme

Signification complète

Table 26 – Suite du glossaire

Acronyme	Signification complète	Acronyme	Signification complète
ERP	Enterprise Resource Planning	ETL	Extract, Transform, Load
FAISS	Facebook AI Similarity Search	GHCR	GitHub Container Registry
GitOps	Git-based Operations	HTTP	HyperText Transfer Protocol
IDE	Integrated Development Environment	JSON	JavaScript Object Notation
JWT	JSON Web Token	KPI	Key Performance Indicator
LLM	Large Language Model	MDX	Multidimensional Expressions
MTTD	Mean Time To Detect	NL2MDX	Natural Language to MDX
NLP	Natural Language Processing	OLAP	Online Analytical Processing
OLTP	Online Transaction Processing	ORM	Object-Relational Mapping
RAG	Retrieval-Augmented Generation	REST	Representational State Transfer
SaaS	Software as a Service	SDK	Software Development Kit
SHA	Secure Hash Algorithm	SQL	Structured Query Language
SSAS	SQL Server Analysis Services	SSIS	SQL Server Integration Services
UI	User Interface	VPS	Virtual Private Server
XML	eXtensible Markup Language	YARP	Yet Another Reverse Proxy

Bibliographie

- [1] W. E. Deming, “Without data, you’re just another person with an opinion,” 1980. Citation populaire attribuée à W. Edwards Deming.
- [2] C. Fiorina, “The goal is to turn data into information, and information into insight,” 2004. Citation sur la valorisation des données.
- [3] OpenAI, “Openai api platform documentation.” <https://platform.openai.com/docs>, 2026. Consulté le 11 Mai 2026.
- [4] Microsoft, “Azure openai service documentation.” <https://learn.microsoft.com/en-us/azure/ai-services/openai/>, 2026. Consulté le 11 Mai 2026.
- [5] Microsoft, “Microsoft copilot.” Outil d’intelligence artificielle générative, 2026. Utilisé pour la création de rapports et l’analyse de données dans Power BI.
- [6] Basedash, “Basedash : Natural language to sql analytics platform.” <https://www.basedash.com>, 2026. Accessed : 2026-04.
- [7] M. Research, “Phi-3 technical report : A highly capable language model locally on your phone,” 2024.
- [8] M. AI, “The llama 3 herd of models,” 2024.
- [9] M. AI, “Mistral 7b,” 2023.
- [10] J. Han, M. Kamber, and J. Pei, *Data Mining : Concepts and Techniques*. Morgan Kaufmann, 3rd ed., 2011.
- [11] R. Kimball and M. Ross, *The Data Warehouse Toolkit : The Definitive Guide to Dimensional Modeling*. Wiley, 2013.
- [12] W. H. Inmon, *Building the Data Warehouse*. Wiley, 4th ed., 2005.
- [13] S. Chaudhuri and U. Dayal, *An Overview of Data Warehousing and OLAP Technology*, vol. 26. 1997.
- [14] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in Neural Information Processing Systems*, vol. 33, 2020.
- [15] Y. Gao, Y. Xiong, X. Gao, *et al.*, “Retrieval-augmented generation for large language models : A survey,” *arXiv preprint arXiv :2312.10997*, 2023.
- [16] N. Reimers and I. Gurevych, “Sentence-bert : Sentence embeddings using siamese bert-networks,” *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.

- [17] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with gpus,” *IEEE Transactions on Big Data*, 2019.
- [18] Meta, “React documentation.” <https://react.dev/>, 2024.
- [19] Microsoft, “Typescript documentation.” <https://www.typescriptlang.org/docs/>, 2024.
- [20] B. Frost, *Atomic Design*. Brad Frost Web, 2016.
- [21] Tailwind Labs, “Tailwind css documentation.” <https://tailwindcss.com/docs>, 2024.
- [22] MUI, “Material ui documentation.” <https://mui.com/material-ui/material-icons/>, 2024.
- [23] Microsoft, “Microsoft sql server documentation,” 2025.
- [24] Microsoft, “Sql server integration services documentation,” 2024. <https://learn.microsoft.com/en-us/sql/integration-services/>.
- [25] Microsoft, “Sql server analysis services documentation,” 2024. <https://learn.microsoft.com/en-us/analysis-services/>.
- [26] Microsoft, “Asp.net core documentation,” 2025.
- [27] S. Ramírez, “Fastapi documentation,” 2025.
- [28] C. Richardson, “Microservices patterns : Api gateway,” *Microservices.io*, 2023.
- [29] S. Exchange, “Dapper documentation,” 2025.
- [30] Microsoft, “Adomd.net overview,” 2024. <https://learn.microsoft.com/en-us/analysis-services/adomd/>.
- [31] Meta, “React documentation,” 2025.
- [32] Chart.js, “Chart.js documentation,” 2025.
- [33] M. Bostock, “D3.js documentation,” 2025.
- [34] T. B. Brown *et al.*, “Language models are few-shot learners,” *NeurIPS*, 2020.
- [35] Ollama, “Ollama documentation,” 2025.
- [36] Docker Inc., “Docker documentation,” 2025.
- [37] Docker Inc., “Docker compose documentation,” 2025. Consulté le 05 juin 2026.
- [38] Git Project, “Git documentation,” 2026. Consulté en mai 2026.
- [39] GitHub, “Github actions documentation,” 2025.
- [40] Portainer Ltd., “Portainer documentation,” 2026. Consulté en mai 2026.
- [41] Microsoft, “Visual studio 2022 documentation,” 2025.
- [42] Microsoft, “Visual studio code documentation,” 2025.

- [43] R. C. Martin, *Clean Architecture : A Craftsman's Guide to Software Structure and Design*. Boston, MA : Prentice Hall, 2017.
- [44] G. DeepMind, "Gemma : Open models based on gemini research and technology," 2024.
- [45] D. AI, "Deepseek-coder : When the large language model meets programming," 2024.
- [46] J. Humble and D. Farley, *Continuous Delivery : Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.